自己动手写操作系统

从零构建 x86 微内核

基于 SZY-KERNEL 项目实战

SZY

2026 年 3 月 16 日

本书随项目代码同步演进，每完成一个 Stage 即补充对应章节。

目录

Part I

准备工作

Chapter 1

前言

1.1 这本书要做什么

本书记录一个完整的 x86 微内核操作系统——SZY-KERNEL——从第一行汇编代码到能运行用户态程序的全过程。每一章对应项目的一个 Stage，包含：

- **目标**：这一步要实现什么功能
- **背景知识**：必要的硬件/CPU 原理
- **设计决策**：为什么选择这种方案而非其他
- **逐行代码讲解**：关键数据结构和实现
- **验证**：如何确认代码正确工作

1.2 目标读者

- 有 C 语言基础，了解指针、结构体、位运算
- 有基本的计算机组成原理概念（寄存器、内存、中断）
- 对”操作系统到底怎么工作的”感到好奇
- 不要求有汇编经验——书中会逐行解释每条汇编指令

1.3 项目愿景

1. 自举移植 GCC 编译器（在自己的 OS 上编译 C 代码）
2. 运行 Vim 文本编辑器

3. 支持 TCP/IP 网络（至少能 ping）

设计原则：

- **自研优先**——内存管理、分页、调度、系统调用全部自己写
- **POSIX 子集**——不追求完整，够移植 GCC/Vim 即可
- **微内核**——内核只做内存、调度、IPC，驱动尽量在用户态

1.4 全书路线图

Stage	章节	名称	里程碑
1	第 2 章	Multiboot2 引导与串口输出	A: 裸机基础
2	第 3 章	GDT：全局描述符表	A
3	第 4 章	IDT：中断描述符表	A
4	第 5 章	IRQ 与键盘驱动	A
5	第 6 章	交互式 Shell	A
6	第 7 章	PMM：物理内存管理	B: 内存管理
7	第 8 章	VMM：虚拟内存与分页	B
8	第 9 章	High-Half Kernel	C: 高级内存
9	第 10 章	内核堆 kmalloc	C
10	第 11 章	VMA：虚拟内存区域	C
11	第 12 章	TSS 与 Ring 3	D: 进程
12	第 13 章	ELF 加载器	D
13	第 14 章	系统调用	D
14	第 15 章	进程管理	D
15	第 16 章	调度器	D
16	第 17 章	VFS 虚拟文件系统	E: 文件系统
17	第 18 章	initrd / ramfs	E

1.5 如何阅读本书

建议按顺序阅读。每章末尾都有”本章小结”和”验证步骤”，确认当前 Stage 的代码能正常工作后再进入下一章。

所有代码可在 https://github.com/szylover/my_microkernel 找到，每个 Stage 对应一个 git tag 或 PR。

Chapter 2

开发环境搭建

2.1 本章目标

在动手写任何内核代码之前，先把工具链准备好：交叉编译器、汇编器、链接器、GRUB 打包工具和 QEMU 模拟器。

2.2 宿主机环境

- 操作系统：Ubuntu 22.04+ / WSL2（本书使用 WSL2 + VS Code Remote）
- 编辑器：VS Code + clangd 扩展

2.3 必要工具安装

```
sudo apt update
sudo apt install -y nasm gcc binutils grub-pc-bin \
    xorriso qemu-system-x86 gcc-multilib gdb bear
```

各工具的作用：

工具	用途
`nasm`	x86 汇编器（NASM 语法）
`gcc -m32`	C 编译器（32 位模式）
`ld`	链接器
`grub-mkrescue`	生成 GRUB 可引导 ISO
`xorriso`	ISO 9660 工具（grub-mkrescue 依赖）
`qemu-system-i386`	x86 模拟器
`gdb`	调试器

`bear`	生成 `compile_commands.json`（给 clangd）

2.4 项目目录结构

```
my_microkernel/
  src/
    boot/         # Multiboot2 引导 + 链接脚本
    include/      # 公共头文件
    kernel/       # 内核实现
  docs/           # 项目文档
  book/           # 本书 LaTeX 源码
  Makefile        # 顶层 wrapper
```

2.5 构建与运行闭环

```
cd src
make iso      # 编译内核 + 打包 ISO
make run      # QEMU 启动
make debug    # QEMU -S 等待 GDB 连接
make gdb      # 另一个终端启动 GDB
```

2.6 VS Code 调试集成

2.7 本章小结

工具链就绪后，下一章我们写第一行代码——Multiboot2 引导头和内核入口。

Part II

裸机基础设施

Chapter 3

Multiboot2 引导与串口输出

3.1 本章目标

- 理解 x86 开机流程：BIOS → GRUB → 内核 `_start`
- 掌握 Multiboot2 header 格式（magic / architecture / length / checksum）
- 理解链接脚本 `linker.ld`：物理加载地址 1 MiB、`.boot` 段、高半区布局
- 掌握 `boot.asm`：PSE 页目录构造、CR4/CR3/CR0 设置、跳转到高半区
- 理解引导期双重映射（identity + high-half）的必要性
- 理解内核栈设置与 `kmain` 的 cdecl 调用约定
- 掌握 UART 16550 寄存器与 `serial_init()` 初始化步骤
- 了解 `printk` 的 `va_list` 实现与格式说明符
- 理解 Multiboot2 info 结构的 tag 迭代解析

3.2 背景知识：x86 开机流程

当你按下电源按钮，到内核第一行 C 代码执行，中间经历了什么？

1. **BIOS/UEFI 自检**——CPU 从 `0xFFFFFFF0`（重置向量）开始执行 BIOS 固件，完成硬件自检（POST）、枚举内存及 PCI 设备。
2. **GRUB 加载**——BIOS 读取启动扇区，加载 GRUB bootloader。GRUB 解析 ELF 内核映像，找到 Multiboot2 header 确认格式正确。
3. **内核入口**——GRUB 把内核加载到 ELF `p_paddr` 指定的物理地址，设置好约定的寄存器状态，跳转到 `_start`。

CPU 状态变化

GRUB 移交控制权时的 CPU 状态：

- `EAX` = `0x36d76289`（Multiboot2 magic，用于内核验证）
- `EBX` = Multiboot2 info 结构体的**物理地址**
- `CR0.PG` = 0（分页关闭），处于 32 位保护模式
- `EIP` `0x00100000`（物理地址，内核加载位置）
- 中断已禁用（`IF=0`），无可用栈

思考 3.1

为什么内核要加载到 1 MiB（`0x100000`）而不是 0 地址？
0–`0xA0000`(640 KiB)是传统 DOS 可用内存，但其中散布着 BIOS 数据区(BDA)、中断向量表(IVT, 0–`0x3FF`)等关键结构，GRUB 运行时也在使用。`0xA0000`–`0xFFFFF` 是 VGA 缓冲区、ROM shadow 和 BIOS 扩展区。1 MiB 以上才是干净的、可自由使用的连续 RAM。

3.3 Multiboot2 Header

GRUB 扫描 OS 映像的前 32 KiB，寻找 8 字节对齐的 Multiboot2 header。header 格式如下：

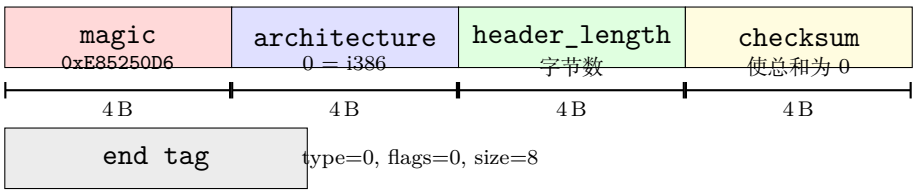

图 3.1: Multiboot2 Header 格式

校验和的计算使四个 32 位字段之和在 2^{32} 模下为 0：

$$\texttt{checksum} = \texttt{0x100000000} - (\texttt{magic} + \texttt{arch} + \texttt{length})$$

对应的汇编实现：

```
section .multiboot_header
align 8
header_start:
    dd 0xe85250d6                   ; Multiboot2 magic
```

```
    dd 0                            ; architecture: i386
    dd header_end - header_start    ; header length
    dd 0x100000000 - (0xe85250d6 + 0 + (header_end - header_start))

    ; Required end tag
    dw 0    ; type
    dw 0    ; flags
    dd 8    ; size
align 8
header_end:
```

注意

Multiboot2 header 必须位于映像的前 32 KiB 内，且 8 字节对齐。链接脚本必须确保 `.multiboot_header` 是映像中最早的段，否则 GRUB 会报“`error: no multiboot header found`”并拒绝加载。

3.4 链接脚本 `linker.ld` 详解

链接脚本控制 ELF 各段的虚拟地址（VMA）和物理地址（LMA）。对于高半区内核（high-half kernel），这两者不同：

```
OUTPUT_FORMAT(elf32-i386)
ENTRY(_start)
KERNEL_VIRT_BASE = 0xC0000000;

SECTIONS {
    . = 1M;   /* 物理加载地址从 1 MiB 开始 */
    __kernel_phys_start = .;

    /* Boot 段：物理地址链接 + 物理地址加载 */
    .boot ALIGN(4096) : {
        KEEP(*(.multiboot_header))
        *(.boot .boot.*)
    }
    .boot_bss ALIGN(4096) (NOLOAD) : {
        *(.boot_bss .boot_bss.*)
    }

    /* 切换到高半区虚拟地址 */
```

```
    . += KERNEL_VIRT_BASE;
    __kernel_virt_start = .;

    /* 内核段：虚拟地址是 0xC01xxxxx，物理地址通过 AT() 回到 1M+ */
    .text ALIGN(4096) : AT(ADDR(.text) - KERNEL_VIRT_BASE) {
        *(.text .text.*)
    }
    .rodata ALIGN(4096) : AT(ADDR(.rodata) - KERNEL_VIRT_BASE) {
        *(.rodata .rodata.*)
    }
    .data ALIGN(4096) : AT(ADDR(.data) - KERNEL_VIRT_BASE) {
        *(.data .data.*)
    }
    .bss ALIGN(4096) (NOLOAD) : AT(ADDR(.bss) - KERNEL_VIRT_BASE) {
        *(COMMON)
        *(.bss .bss.*)
    }
    __kernel_virt_end = .;
    __kernel_phys_end = __kernel_virt_end - KERNEL_VIRT_BASE;
}
```

核心要点：

- **`. = 1M;`**——位置计数器从 1 MiB 开始，GRUB 将内核加载到此处。
- **`.boot` 段**——包含 Multiboot2 header 和早期引导代码。这些代码在分页开启前执行，必须链接在物理地址上（否则 EIP 不对）。`KEEP` 确保 `.multiboot_header` 不被垃圾回收优化掉。
- **`. += KERNEL_VIRT_BASE;`**——位置计数器跳到 `0xC0000000` + 当前偏移，后续段链接在高半区虚拟地址。
- **`AT(ADDR(.text) - KERNEL_VIRT_BASE)`**——ELF 的 `p_vaddr` = `0xC01xxxxx`（链接地址），但 `p_paddr` = `0x001xxxxx`（加载地址）。GRUB 按 `p_paddr` 加载到物理 RAM，但代码中所有符号引用用的是虚拟地址——分页开启后才能正确解析。
- **`__kernel_phys_start`/`__kernel_phys_end`**——供 PMM 用来保留内核占用的物理内存，避免被分配出去。

思考 3.2

如果去掉 AT() 宏会发生什么?
ELF 的 p_paddr 和 p_vaddr 将相同（都是 0xC01xxxxx）。GRUB 会尝试把内核加载到物理地址 0xC01xxxxx 3 GiB——如果机器只有 256 MiB 内存，加载直接失败。即使内存足够，1 MiB 以下的 BIOS 区域也已经错过，浪费了低端可用 RAM。

3.5 boot.asm 详解：从物理地址到高半区

boot.asm 是内核的入口点 _start。它的核心任务是：在没有分页的情况下构建临时页目录，开启分页后跳转到高半区。

3.5.1 保存 Multiboot2 信息

```
_start:
    mov esi, eax          ; magic (0x36d76289)
    mov edi, ebx          ; mb2 info pointer (physical address)
```

GRUB 传入的 EAX（magic）和 EBX（info 指针）保存在 callee-saved 寄存器 ESI/EDI 中，以便后续传给 kmain。

3.5.2 PSE 页目录构造

```
; 清零页目录 (1024 x 4B = 4 KiB)
push edi
mov ecx, 1024
xor eax, eax
mov edi, boot_pd
rep stosd
pop edi

; 填充 PSE 映射: 256 x 4 MiB = 1 GiB
xor ecx, ecx
.fill_pse:
    mov eax, ecx
    shl eax, 22           ; eax = ecx * 4 MiB
    or  eax, 0x83         ; PS(bit7) | RW(bit1) | Present(bit0)
    mov [boot_pd + ecx*4], eax      ; identity: PD[i]
    mov [boot_pd + 768*4 + ecx*4], eax ; high-half: PD[768+i]
```

```
    inc ecx
    cmp ecx, 256
    jb .fill_pse
```

为什么？

为什么用 PSE（4 MiB 大页）而不是标准 4 KiB 页？
4 KiB 页需要分配 page table（每个 PDE 指向一个 4 KiB 页表），但此时 PMM 还没初始化——我们甚至不知道哪些物理内存可用。PSE 模式下 PDE 自身就包含 4 MiB 页的物理基地址，不需要页表。只需一个页目录（4 KiB）就能映射整个地址空间。
`vmm_init()` 之后会用 4 KiB 页表替换这些 PSE 条目，实现更细粒度的权限控制。

PSE PDE 的位格式：

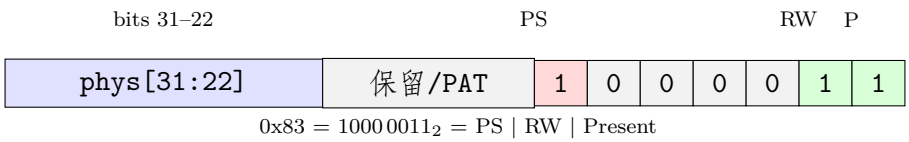

图 3.2: PSE 模式 PDE 位域（`0x83` 标志）

3.5.3 双重映射：Identity + High-Half

循环将同一物理地址同时写入两个 PD 位置：

- `PD[i]`（$i = 0..255$）——Identity 映射：虚拟 $i \times 4\text{MiB} \rightarrow$ 物理 $i \times 4\text{MiB}$
- `PD[768+i]`——High-half 映射：虚拟 `0xC0000000` $+ i \times 4\text{MiB} \rightarrow$ 物理 $i \times 4\text{MiB}$

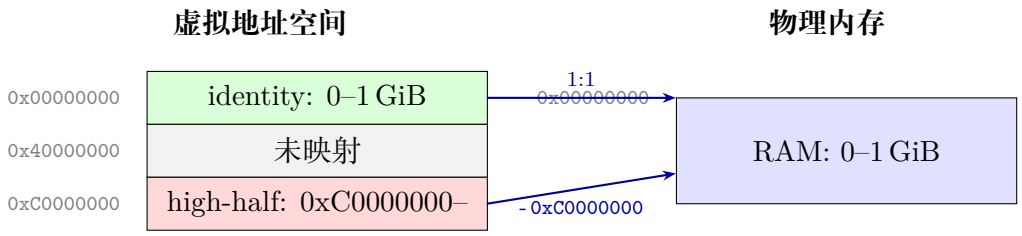

图 3.3: 引导期双重映射：identity 和 high-half 指向同一物理内存

注意

如果只建 high-half 映射而不建 identity 映射会发生什么？
开启分页后，CPU 的下一条指令在物理地址 ~`0x001xxxxx`。没有 identity 映射，

MMU 无法翻译当前 EIP → **Page Fault** → 此时 IDT 还没设置 → **Double Fault** → 没有 DF handler → **Triple Fault** → CPU 重启。
整个"死亡连锁"发生在一条指令之内，QEMU 会直接 reboot，串口什么输出都看不到——这也是引导调试最令人头疼的问题之一。

3.5.4 开启 PSE 和分页

```
; CR4.PSE = 1 (允许 4 MiB 页)
mov eax, cr4
or  eax, 0x00000010   ; bit 4 = PSE
mov cr4, eax

; CR3 <- boot_pd (物理地址)
mov eax, boot_pd
mov cr3, eax

; CR0.PG = 1 (开启分页)
mov eax, cr0
or  eax, 0x80000000   ; bit 31 = PG
mov cr0, eax
```

CPU 状态变化

三步 CR 操作后 CPU 状态的逐步变化：

1. `CR4.PSE = 1`——CPU 现在允许 4 MiB 页，但还没开启分页。
2. `CR3 = boot_pd`——告诉 CPU 页目录在哪，但分页仍关闭。
3. `CR0.PG = 1`——分页**立即生效**！从下一条指令开始，所有内存访问经过 MMU 翻译。Identity 映射保证当前 EIP 仍然有效。

3.5.5 跳转到高半区

```
mov eax, _start_high ; 链接地址 = 0xC01xxxxx
jmp eax              ; 跳到高半区虚拟地址
```

`_start_high` 在 `.text` 段中，链接在 `0xC01xxxxx`。`mov eax, _start_high` 把这个高虚拟地址加载到 `EAX`，`jmp eax` 跳转——MMU 通过 PD[768] 将 `0xC01xxxxx` 翻译回物理 `0x001xxxxx`。

从此内核运行在高半区虚拟地址空间。

3.6 栈设置与 kmain 调用

进入高半区后，第一件事是设置栈和调用 C 入口：

```
section .text
_start_high:
    mov esp, stack_top   ; 栈顶（高半区虚拟地址）
    and esp, 0xFFFFFFF0  ; 16 字节对齐（SSE 要求）

    ; cdecl 调用约定：参数从右向左压栈
    push edi             ; mb2_info（物理地址）
    push esi             ; mb2_magic
    call kmain
    add esp, 8           ; 清理参数（不会执行到这里）

.hlt:
    hlt
    jmp .hlt

section .bss
align 4096
stack_bottom:
    resb 16384           ; 16 KiB 内核栈
stack_top:
```

- 栈在 .bss 段中分配 16 KiB（resb 16384），stack_top 是栈的高地址端（x86 栈向下增长）。
- and esp, 0xFFFFFFF0 确保 16 字节对齐——System V ABI 要求调用函数前 ESP 必须 16 字节对齐。
- push edi; push esi; call kmain——cdecl 约定，参数从右向左压栈，kmain(uint32_t magic, void* mb2_info)。
- kmain 正常不会返回；如果返回则 hlt 停住 CPU。

3.7 UART 16550 串口输出

在 OS 开发中，串口是最可靠的调试输出通道——VGA 文本模式需要正确的显存映射，而串口只需要几个 I/O 端口。QEMU 可以将串口输出重定向到终端（`-serial stdio`），非常方便。

3.7.1 16550 寄存器表

COM1 的基地址为 `0x3F8`，各寄存器偏移如表 3.1：

偏移	DLAB=0	DLAB=1	说明
+0	DATA（读/写）	DLL（除数低字节）	收发数据 / 波特率低字节
+1	IER（中断使能）	DLM（除数高字节）	中断控制 / 波特率高字节
+2	IIR / FCR	—	中断识别 / FIFO 控制
+3	LCR（线路控制）	—	数据位/停止位/校验/DLAB
+4	MCR（Modem 控制）	—	RTS/DTR/OUT 控制
+5	LSR（线路状态）	—	bit 0=DataReady，bit 5=THR Empty

表 3.1: UART 16550 寄存器（COM1 = `0x3F8`）

`DLAB`（Divisor Latch Access Bit）是 LCR 的 bit 7。当 DLAB=1 时，偏移 +0 和 +1 变为波特率除数寄存器。

3.7.2 `serial_init()` 初始化步骤

```
void serial_init(void) {
    outb(COM1 + 1, 0x00); // 1. 关闭所有中断
    outb(COM1 + 3, 0x80); // 2. 设置 DLAB=1
    outb(COM1 + 0, 0x03); // 3. 除数低字节=3 -> 38400 baud
    outb(COM1 + 1, 0x00); // 4. 除数高字节=0
    outb(COM1 + 3, 0x03); // 5. 8N1, DLAB=0
    outb(COM1 + 2, 0xC7); // 6. 启用 FIFO, 清缓冲, 14B 阈值
    outb(COM1 + 4, 0x0B); // 7. RTS/DTR/OUT2 置位
}
```

逐步解析：

1. 关中断——此时 PIC/IDT 还没设好，串口中断会引发异常。
2. DLAB=1——让偏移 +0/+1 变成除数寄存器。

3. 除数 = 3——波特率 = 115200 ÷ 3 = 38400。
4. 除数高字节 = 0。
5. LCR = 0x03 = 8 数据位、无校验、1 停止位（8N1），同时 DLAB=0。
6. FCR = 0xC7 = 启用 FIFO + 清空收发缓冲区 + 14 字节触发阈值。
7. MCR = 0x0B = RTS + DTR + OUT2（OUT2 在 PC 上与 IRQ 线相连）。

3.7.3 发送一个字符

```c
void serial_putc(char c) {
    if (c == '\n') {
        while (!serial_transmit_empty()) {}
        outb(COM1, '\r');    // CRLF: 先回车
    }
    while (!serial_transmit_empty()) {} // 等 THR 空
    outb(COM1, (uint8_t)c);
}

static int serial_transmit_empty(void) {
    return (inb(COM1 + 5) & 0x20) != 0; // LSR bit5
}
```

发送前轮询 LSR 的 bit 5（Transmitter Holding Register Empty）。遇到 `'\n'` 时先发 `'\r'`（CR），确保串口终端换行后光标回到行首（否则输出会呈”阶梯状”）。

3.8 printk 实现概述

`printk` 是内核的 `printf`——唯一的格式化输出设施。所有输出最终通过 `serial_putc` 发送到串口。

3.8.1 核心结构

```c
void printk(const char* fmt, ...) {
    va_list args;
    va_start(args, fmt);
    vprintk(fmt, args);  // 内部实现
    va_end(args);
}
```

vprintk 逐字符扫描 fmt，遇到 % 时解析格式说明符。支持的格式如表 3.2：

说明符	参数类型	说明
%d	int	有符号十进制
%u	unsigned	无符号十进制
%x	unsigned	小写十六进制（无 0x 前缀）
%p	void*	指针，格式 0xXXXXXXXX（固定 8 位）
%s	char*	字符串（NULL 安全，输出 (null)）
%c	int	单个字符
%%	—	字面 %

表 3.2: printk 支持的格式说明符

3.8.2 格式化选项

vprintk 还支持：

- **宽度**——%8x 输出至少 8 个字符（右对齐，空格填充）。
- **零填充**——%08x 左侧补零。
- **左对齐**——%-16s 左对齐、右侧补空格。

设计决策

为什么从头实现 printk 而不用 libc 的 printf？
内核没有标准 C 库（freestanding 环境）。printf 依赖 FILE*、stdout、write 系统调用等基础设施——这些都是我们自己要实现的东西。内核的 printk 直接写 serial_putc，零依赖、零缓冲。

3.9 Multiboot2 信息结构解析

GRUB 在 EBX 中传递的 Multiboot2 info 是一个 tag 链表结构：

```
/* 信息结构头部 */
struct mb2_info {
    uint32_t total_size; /* 整个结构的字节数 */
    uint32_t reserved;
    /* 后面紧跟若干 tag，每个 tag 8 字节对齐 */
```

```
};

/* 每个 tag 的公共头 */
struct mb2_tag {
    uint32_t type;       /* tag 类型 */
    uint32_t size;       /* tag 字节数（含此头） */
};
```

遍历算法：从 `base + 8` 开始，每次跳过当前 tag 的 `size`（向上 8 字节对齐），直到遇到 end tag（type=0, size=8）。

对我们最重要的是 **Memory Map Tag**（type=6），它报告物理内存布局——这是 PMM 初始化的输入数据（详见第 8 章）。

```
struct mb2_mmap_entry {
    uint64_t addr;    /* 区域起始物理地址 */
    uint64_t len;     /* 区域长度 */
    uint32_t type;    /* 1=可用，2=保留，3=ACPI，... */
    uint32_t reserved;
};
```

思考 3.3

Multiboot2 info 指针是**物理地址**，但 `kmain` 在分页开启后的高半区运行。为什么仍然能直接读取 `mb2_info` 的内容？

提示：回顾第 3.5.3 节的 identity 映射——在 `vmm_unmap_identity()` 调用之前，低地址物理页同时可通过 identity 映射和 high-half 映射访问。

3.10 验证

编译并启动后，串口应出现类似如下输出：

```
[serial] init ok (COM1, 38400 baud)
[mmap] Multiboot2 Memory Map:
  [0x0 - 0x9FC00] type=1 (available) ~639 KiB
  [0x100000 - 0x7FE0000] type=1 (available) ~126 MiB
  ...
[pmm] backend: bitmap
[pmm] init ok: 32495 free pages (126 MiB)
```

- 如果启动后什么都看不到——检查 QEMU 是否加了 `-serial stdio`。

- 如果看到 VGA 上有 “HH” 但串口无输出——`serial_init()` 可能有问题，检查端口号和波特率除数。
- 如果 QEMU 反复重启——Triple Fault。检查 Multiboot2 header 和页目录构造。

3.11 思考与练习

练习 3.1

验证 Multiboot2 magic

在 `kmain` 的最开头，检查第一个参数是否等于 `0x36d76289`。如果不等，打印 “`FATAL: invalid multiboot2 magic: 0x%x`” 并 `hlt`。为什么从 GRUB 启动时这个值一定正确，但仍值得检查？(提示：考虑将来用其他 bootloader、或 chainloading 场景)。

练习 3.2

波特率计算

当前 `serial_init()` 设置除数为 3，波特率 $= 115200 \div 3 = 38400$。修改除数为 1，使波特率变为 115200。同时需要修改 QEMU 的 `-serial` 参数吗？(提示：QEMU 的虚拟串口会自动适配，但真实硬件上两端必须匹配。)

练习 3.3

增加 %b 二进制格式支持

给 `vprintk` 添加 `%b` 格式说明符，输出 32 位整数的二进制表示。例如 `printk("%b", 0x83)` 应输出 `10000011`（省略前导零）或 `00000000000000000000000010000011`（填满 32 位）。

建议参照 `fmt_u32_hex` 的实现方式，用循环从高位到低位逐位输出。宽度选项 `%032b` 应输出完整的 32 位（前导零填充）。

Chapter 4

GDT：全局描述符表

4.1 本章目标

- 理解 x86 保护模式下段式内存管理的完整机制
- 掌握 GDT 描述符的 8 字节二进制布局（逐 bit）
- 实现平坦内存模型的 GDT：Null + Kernel Code + Kernel Data
- 理解段寄存器的可见部分（selector）与不可见部分（hidden cache）
- 掌握 `lgdt` + far jump + `mov` 刷新段寄存器的完整流程

4.2 背景知识：保护模式下的段式内存管理

4.2.1 从实模式到保护模式

在实模式下（8086），CPU 使用 `segment:offset` 计算物理地址：

$$\text{物理地址} = \text{segment} \times 16 + \text{offset}$$

这种方式没有任何保护——任何代码都能访问任何内存。

保护模式改变了规则：段寄存器不再直接存储段基址，而是存储一个 16 位的 **段选择子** (selector)。CPU 用 selector 去一张全局表 (GDT) 中查找对应的**段描述符** (segment descriptor)，描述符包含段的基址、大小、权限等信息。

地址翻译变成两步：

1. **分段**：逻辑地址 (selector : offset) → 线性地址 = base + offset
2. **分页**（如果开启）：线性地址 → 物理地址

为什么？

现代 OS 都用**平坦模型**（flat model）：所有段的 base=0, limit=4GiB。这让分段**不参与地址转换**（线性地址 == offset），把控制完全交给分页。但 GDT 不能省略——x86 保护模式硬性要求每个段寄存器都指向一个有效描述符。

4.2.2 段寄存器的"双面"结构

每个段寄存器（`CS`, `DS`, `SS`, `ES`, `FS`, `GS`）实际上包含两部分：

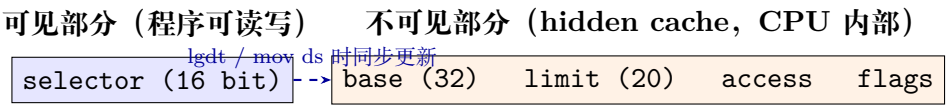

图 4.1: 段寄存器由可见的 selector 和不可见的 hidden cache 组成

当你执行 `mov ds, ax` 时，CPU 做了两件事：

1. 把 `ax`（selector）写入 `DS` 的可见部分
2. 用 selector 去 GDT 查表，把找到的描述符**缓存**到 `DS` 的 hidden part

之后的每次内存访问，CPU 直接用 hidden cache 中的 base/limit/权限，**不再查 GDT**。这就是为什么改了 GDT 后必须重新加载段寄存器——否则 hidden cache 里还是旧的描述符。

思考 4.1

如果 CPU 每次内存访问都重新查 GDT，性能会怎样？一次内存读取原本只需 1 次总线事务，现在变成 3 次（读 GDT → 读描述符 → 读目标内存）。hidden cache 就是为了避免这个开销。

4.3 段选择子格式

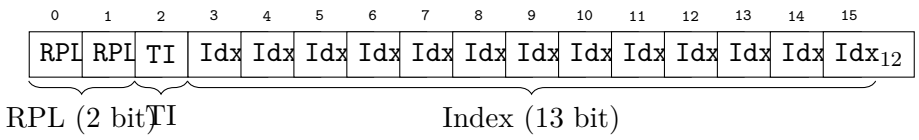

图 4.2: 段选择子的 16 位布局

`bit 0--1` RPL (Requested Privilege Level)：请求的特权级。0= 内核，3= 用户。当 selector 加载到段寄存器时，CPU 检查 $\max(\mathrm{CPL}, \mathrm{RPL}) \leq \mathrm{DPL}$。

bit 2 TI (Table Indicator)：0= 在 GDT 中查找，1= 在 LDT 中查找。我们只用 GDT。

bit 3--15 Index：GDT 中的描述符索引。GDT 条目编号从 0 开始，每个 8 字节。selector 的 Index 字段 × 8 就是 GDT 中的字节偏移。

我们的选择子取值：

Selector	二进制	Index	RPL	用途
0x00	0000000000000000	0	0	Null（无效，CPU 用于检查）
0x08	0000000000001000	1	0	内核代码段
0x10	0000000000010000	2	0	内核数据段

```
// gdt.h
enum {
    GDT_KERNEL_CODE_SEL = 0x08, // Index=1, TI=0, RPL=0
    GDT_KERNEL_DATA_SEL = 0x10, // Index=2, TI=0, RPL=0
};
```

4.4 GDT 描述符的 8 字节布局

这是 x86 最”反直觉”的数据结构之一——由于历史兼容性（286→386），字段被拆成不连续的碎片散布在 8 个字节中。

4.4.1 内存中的字节布局

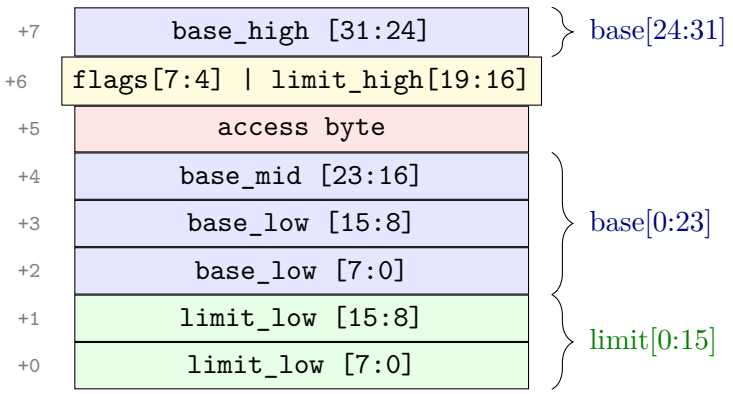

图 4.3: GDT 描述符的 8 字节内存布局（地址从下到上递增）

4.4.2 对应的 C 结构体

```c
typedef struct __attribute__((packed)) {
    uint16_t limit_low; // 字节 0-1: limit 的低 16 位 [0:15]
    uint16_t base_low; // 字节 2-3: base 的低 16 位 [0:15]
    uint8_t base_mid;  // 字节 4: base 的中间 8 位 [16:23]
    uint8_t access;    // 字节 5: 访问字节 (P/DPL/S/Type)
    uint8_t gran;      // 字节 6: 高4位=flags, 低4位=limit[16:19]
    uint8_t base_high; // 字节 7: base 的最高 8 位 [24:31]
} gdt_entry_t;
```

注意

`__attribute__((packed))` 是**必须的**。没有它，编译器可能在 `base_mid` 后面插入 padding 字节来对齐 `access`，导致整个结构体从 8 字节膨胀到 12 字节——CPU 会把 padding 当作描述符的一部分，结果完全错乱。

思考 4.2

为什么 base 被拆成三段（low/mid/high）而非一个连续的 32 位字段？
因为 286 时代 base 只有 24 位（3 字节），386 扩展到 32 位时在末尾加了 1 字节。Intel 必须向后兼容 286 的描述符格式，所以新增的 8 位只能塞到字节 7。这就是为什么 base 是”碎片化”的——历史包袱。

4.4.3 重组逻辑：gdt_set_entry

因为字段是碎片化的，填充描述符需要位操作来拆分和重组：

```c
static void gdt_set_entry(int idx, uint32_t base, uint32_t limit,
                          uint8_t access, uint8_t flags) {
    // --- limit 拆分: 低 16 位放 limit_low, 高 4 位放 gran 的低半 ---
    gdt[idx].limit_low = (uint16_t)(limit & 0xFFFF);
    //  limit = 0x000FFFFF
    //  limit & 0xFFFF = 0xFFFF → limit_low

    // --- base 拆分: 3 段 ---
    gdt[idx].base_low = (uint16_t)(base & 0xFFFF);
    //  base = 0x00000000
    //  base & 0xFFFF = 0x0000 → base_low

    gdt[idx].base_mid = (uint8_t)((base >> 16) & 0xFF);
    //  (base >> 16) & 0xFF = 0x00 → base_mid
```

```

    gdt[idx].base_high = (uint8_t)((base >> 24) & 0xFF);
    //  (base >> 24) & 0xFF = 0x00 → base_high

    // --- access 直接赋值（完整的 8 位） ---
    gdt[idx].access = access;

    // --- gran 字节：高 4 位是 flags，低 4 位是 limit 的 [16:19] ---
    gdt[idx].gran = (uint8_t)((flags & 0xF0) | ((limit >> 16) & 0x0F));
    //  flags = 0xC0 → 高 4 位 = 0xC (= 1100b → G=1, D/B=1)
    //  (limit >> 16) & 0x0F = 0x0F → 低 4 位
    //  结果：0xCF
}
```

对于我们的内核代码段（base=0, limit=`0xFFFFF`, access=`0x9A`, flags=`0xC0`），实际写入内存的 8 字节是：

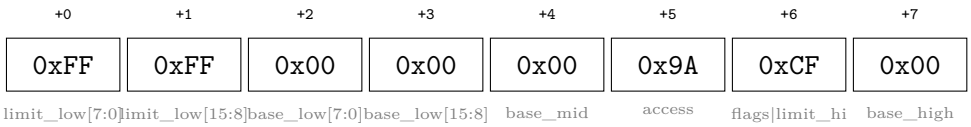

图 4.4: 内核代码段描述符在内存中的实际 8 字节

4.5 Access 字节：逐 bit 详解

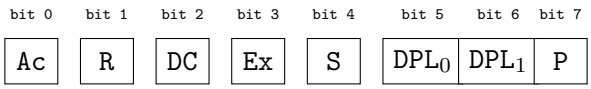

图 4.5: access 字节的 8 个 bit

bit 7 P (Present) = 1：此描述符有效。P=0 时访问该段会触发 #NP (Segment Not Present)。

bit 5--6 DPL (Descriptor Privilege Level)：段的最低访问权限。00_b = Ring 0（只有内核能访问），11_b = Ring 3（用户态可访问）。

bit 4 S (Descriptor Type) = 1：代码/数据段。S=0 表示系统段（TSS、LDT、Call Gate 等），我们将来做 TSS 时会用到。

bit 3 Ex (Executable) = 1：代码段（此段中的内容可以执行）。Ex=0：数据段（此段中的内容只能读/写，不能执行）。

bit 2 DC (Direction/Conforming)：

- 数据段：方向位。0= 向上增长（正常），1= 向下增长（用于栈段扩展）
- 代码段：一致性位。0= 非一致，1= 一致（允许低特权级代码跳入）

我们设为 0。

bit 1 RW (Readable/Writable)：

- 代码段：1= 可读（可以读取代码段中的数据），0= 仅可执行
- 数据段：1= 可写，0= 只读

我们代码段和数据段都设为 1。

bit 0 Ac (Accessed)：CPU 访问此段后自动置 1。OS 可用它做”最近使用”跟踪。初始化时设为 0 即可。

内核代码段 access = `0x9A`

$$\texttt{0x9A} = 10011010_b$$

P	DPL	DPL	S	Ex	DC	RW	Ac
1	0	0	1	1	0	1	0

含义：有效（P=1）、Ring 0（DPL=00）、代码/数据段（S=1）、可执行（Ex=1）、非一致（DC=0）、可读（RW=1）、未访问（Ac=0）。

内核数据段 access = `0x92`

$$\texttt{0x92} = 10010010_b$$

与代码段的唯一区别：Ex=0（不可执行）。RW=1 在数据段表示可写。

4.6 Flags + Limit High 字节（gran）

字节 6 的 8 bit 被分成两半使用：

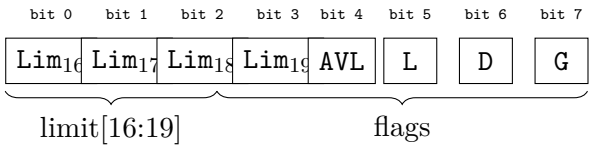

图 4.6: gran 字节的 bit 布局

bit 7 G (Granularity)：limit 的单位。G=0：limit 单位是字节（最大 $2^{20} - 1 = 1\text{MiB}$）。G=1：limit 单位是 4KiB 页（最大 $2^{20} \times 4096 = 4\text{GiB}$）。平坦模型 limit=`0xFFFFF` + G=1 覆盖完整 4GiB 地址空间。

bit 6 D/B (Default Operation Size)：D=1：32 位段（默认操作数和地址大小是 32 位）。D=0：16 位段。我们在 32 位保护模式下必须设 D=1。

bit 5 L (Long mode)：64 位代码段标志。i386 下**必须为 0**。

bit 4 AVL (Available)：CPU 不使用，软件可自由使用。我们设为 0。

bit 0--3 limit 的高 4 位 [16:19]。

我们的 flags = `0xC0`：

$$\texttt{0xC0} = 1100\ \underbrace{0000}_{\text{limit_hi}}$$

即 G=1, D/B=1, L=0, AVL=0。与 limit_high = `0x0F` 合并后 gran = `0xCF`。

4.7 GDTR 结构体

`lgdt` 指令的操作数是一个 6 字节（48 位）的结构体：

```
typedef struct __attribute__((packed)) {
    uint16_t limit; // GDT 总字节数 - 1
    uint32_t base; // GDT 数组的线性地址
} gdtr_t;
```

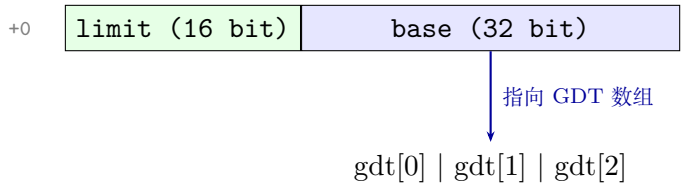

图 4.7: GDTR 的 6 字节布局

```
gdtr.limit = (uint16_t)(sizeof(gdt) - 1); // 3 * 8 - 1 = 23
gdtr.base = (uint32_t)(uintptr_t)&gdt[0]; // GDT 数组的线性地址
```

思考 4.3

为什么 limit 是 “sizeof(gdt) - 1“ 而非 “sizeof(gdt)“？这是 x86 的惯例：limit 表示最后一个有效字节的**偏移**，所以 3 个描述符 × 8 字节 = 24 字节，最后一个字节偏移是 23。如果写 24 会怎样？CPU 会认为 GDT 多了一个字节，可能读到 GDT 数组之外的垃圾数据。

4.8 gdt_init()：填充三个描述符

```c
void gdt_init(void) {
    // 描述符 0: Null 描述符 (x86 硬性要求)
    //  所有字段都是 0。
    //  如果任何段寄存器的 selector = 0x00, CPU 知道它是"空的"。
    //  在某些场景 (如加载 LDT 寄存器) CPU 会检查是否为 null selector。
    gdt_set_entry(0, 0, 0, 0, 0);

    // 描述符 1: 内核代码段 (selector = 0x08)
    gdt_set_entry(1,
        0x00000000, // base = 0 (平坦模型)
        0x000FFFFF, // limit = 0xFFFFF (配合 G=1 → 4GiB)
        0x9A,       // access: P=1, DPL=0, S=1, Ex=1, DC=0, RW=1, Ac=0
        0xC0        // flags: G=1, D/B=1, L=0, AVL=0
    );

    // 描述符 2: 内核数据段 (selector = 0x10)
    gdt_set_entry(2,
        0x00000000, // base = 0 (平坦模型)
        0x000FFFFF, // limit = 0xFFFFF
        0x92,       // access: P=1, DPL=0, S=1, Ex=0, DC=0, RW=1, Ac=0
        0xC0        // flags: G=1, D/B=1, L=0, AVL=0
    );

    gdtr.limit = (uint16_t)(sizeof(gdt) - 1); // 23
    gdtr.base = (uint32_t)(uintptr_t)&gdt[0];

    gdt_flush(&gdtr);
}
```

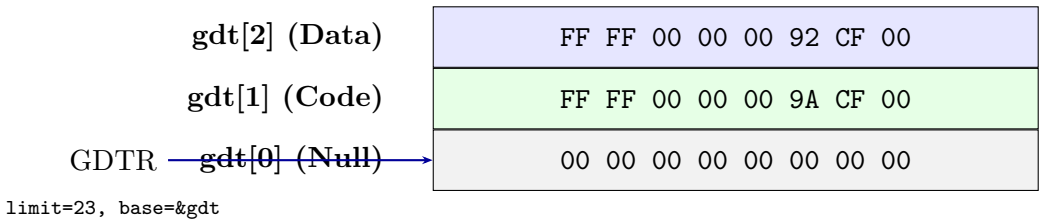

图 4.8: GDT 初始化后的内存布局（3 个描述符 = 24 字节）

设计决策

为什么 GDT 第 0 项必须是空描述符？
Intel 规范规定 selector=0 是 null selector。作用有二：

1. **防御性检查**：如果代码不小心用了未初始化的段寄存器（值为 0），CPU 在访问内存时会检测到 null selector 并触发 #GP，而非悄悄使用错误的段。
2. **LDT 相关**：加载 LDT 寄存器时，null selector 表示" 不使用 LDT"。

思考 4.4

将来做用户态（Ring 3）时，我们需要添加 2 个新描述符：

- 用户代码段（DPL=3，access=`0xFA`）
- 用户数据段（DPL=3，access=`0xF2`）

它们的 selector 分别是多少？（提示：index 3 和 4，各乘以 8。）

4.9 汇编详解：gdt_flush.asm

这是整个 GDT 初始化中最关键也最容易出错的部分。逐条指令解释：

```
bits 32
global gdt_flush

gdt_flush:
    mov eax, [esp + 4]   ; (1) 取函数参数：gdtr_ptr
    lgdt [eax]           ; (2) 加载 GDTR

    jmp 0x08:.flush_cs   ; (3) far jump 刷新 CS

.flush_cs:
    mov ax, 0x10         ; (4)
    mov ds, ax           ; (5) 刷新 DS
    mov es, ax           ; (6) 刷新 ES
    mov fs, ax           ; (7) 刷新 FS
    mov gs, ax           ; (8) 刷新 GS
    mov ss, ax           ; (9) 刷新 SS
    mov esp, esp         ; (10) SS 后的安全指令

    ret                  ; (11) 返回
```

第 5 行：`mov eax, [esp + 4]` `gdt_flush` 使用 cdecl 调用约定。进入函数时栈布局：

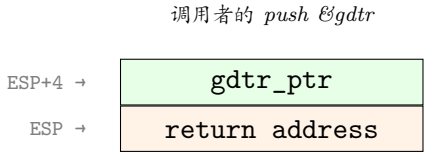

图 4.9: 进入 gdt_flush 时的栈布局

`[esp + 4]` 跳过返回地址，取到 `gdtr_ptr`。

第 6 行：`lgdt [eax]` CPU 从 `eax` 指向的 6 字节中读取 limit（2 字节）和 base（4 字节），写入 GDTR 内部寄存器。

CPU 状态变化

`lgdt` 执行后：

- GDTR.limit = 23, GDTR.base = &gdt[0]
- CPU **知道**新 GDT 在哪了
- 但 `CS`/`DS`/`SS` 的 hidden cache **仍然是旧的**
- 此时用旧 GDT 还是新 GDT 取决于 hidden cache（旧的）

第 8 行：`jmp 0x08:.flush_cs` —Far Jump 这是**最关键的一条指令**。它不是普通的 `jmp`（只改 `EIP`），而是 **far jump**：同时修改 `CS` 和 `EIP`。

- `0x08` → 写入 `CS`（新的 selector）
- `.flush_cs` → 写入 `EIP`（跳转目标）

由于 `CS` 被重新加载，CPU 用新 selector (`0x08`) 去**新 GDT** 查表，找到描述符 1（内核代码段），把 base/limit/权限缓存到 `CS` 的 hidden part。

CPU 状态变化

far jump 执行后：

- `CS` 的可见部分 = `0x08`（index=1, RPL=0）
- `CS` 的 hidden cache = GDT[1] 的完整描述符

- 后续取指令使用新的代码段描述符
- DS/SS 等仍然使用旧缓存——还需要继续刷新

为什么？

为什么刷新 CS 必须用 far jump，不能用 `mov cs, ax`?
因为 x86 **没有** `mov cs, reg` 这条指令——CS 不能被直接赋值。唯一能修改 CS 的途径是：far jump、far call、far ret、`iret`。这些指令会同时修改 CS 和 EIP，确保代码执行流不会”断裂”。

第 11–16 行：刷新数据段寄存器

```
1   mov ax, 0x10  ; selector = 0x10 = GDT[2] (内核数据段)
2   mov ds, ax    ; DS hidden cache ← GDT[2]
3   mov es, ax    ; ES hidden cache ← GDT[2]
4   mov fs, ax    ; FS hidden cache ← GDT[2]
5   mov gs, ax    ; GS hidden cache ← GDT[2]
```

每条 `mov ds, ax` 都会触发 CPU：

1. 检查 selector 是否有效（index 在 GDT 范围内、P=1、权限检查通过）
2. 将 GDT[2] 的描述符缓存到对应段寄存器的 hidden part

第 17–18 行：`mov ss, ax` + `mov esp, esp` SS 比较特殊：修改 SS 后，CPU 会在**下一条指令执行完**之前 **禁止中断和异常**（硬件行为）。这是为了保证 SS:ESP 的原子性——防止中断在 SS 改了但 ESP 还没改的”半截”状态到来。

`mov esp, esp` 是一条无害指令（空操作），目的是消耗这个”抑制窗口”。之后 `ret` 正常返回到 C 代码。

4.10 验证

```
1 # 串口输出:
2 [init] gdt ok
```

如果 GDT 有误（如 access byte 写错、base/limit 计算错误），CPU 会在 far jump 时 Triple Fault 并重启。

4.10.1 GDB 逐指令验证

```
# 终端 A
make DEBUG=1 debug

# 终端 B
make gdb
(gdb) b gdt_flush
(gdb) c
Breakpoint 1, gdt_flush () at gdt_flush.asm:21
(gdb) info reg cs ds ss
cs  0x8  8          # GRUB 设置的 CS (通常也是 0x08)
ds  0x10 16         # GRUB 设置的 DS
ss  0x10 16

(gdb) si            # lgdt [eax]
(gdb) si            # jmp 0x08:.flush_cs
(gdb) info reg cs
cs  0x8  8          # CS 重新加载, hidden cache 已更新

(gdb) si            # mov ax, 0x10
(gdb) si            # mov ds, ax
(gdb) info reg ds
ds  0x10 16         # DS 重新加载

(gdb) c             # 继续, 应该看到串口打印 [init] gdt ok
```

4.11 思考与练习

练习 4.1

在 GDB 中的 `gdt_flush` 处，执行 `lgdt` 前后分别用 `info reg cs ds` 查看段寄存器值。`lgdt` 前后 CS 的值有变化吗？`jmp 0x08:.flush_cs` 后呢？画出这三个时间点段寄存器的状态变化。

练习 4.2

把内核代码段的 access 从 `0x9A` 改为 `0x92`（变成数据段描述符）。重新编译运行。会在什么时候崩溃？（提示：far jump 时 CPU 发现 CS 指向的不是代码段……）错误是 #GP 还是 Triple Fault？

练习 4.3

在 GDT 中添加第 4 个条目：DPL=3 的用户代码段（access=`0xFA`，base=0，limit=`0xFFFFF`，flags=`0xC0`）。计算它的 selector。不需要使用它，只需确认添加后内核正常启动。这是 Stage 10 (Ring 3) 的准备。

练习 4.4

（挑战）注释掉 `gdt_flush` 中的 `mov ss, ax` 一行。运行后内核似乎仍然工作？为什么没有立刻崩溃？提示：GRUB 设置的 SS 可能和我们的 `0x10` 一样——用 GDB 验证你的猜想。如果 GRUB 的 SS 配置不同（比如用了不同的 GDT 布局），会怎样？

4.12 本章小结

本章我们完整拆解了 GDT 的每一个字节：

1. **段选择子**（16 位）= index + TI + RPL
2. **描述符**（8 字节）= 碎片化的 base/limit + access byte + flags
3. **段寄存器**的双面结构：可见的 selector + 不可见的 hidden cache
4. **gdt_flush.asm**：lgdt 加载表 → far jump 刷新 CS → mov 刷新 DS/SS

GDT 是 x86 保护模式的基础设施，也是后续 TSS（Ring 3 栈切换）和 IDT（中断门需要代码段 selector）的前置条件。

下一章我们设置 IDT，让内核能捕获 CPU 异常并打印诊断信息。

Chapter 5

IDT：中断描述符表

5.1 本章目标

- 理解 x86 中断与异常的完整机制：向量号、异常分类、错误码
- 掌握 IDT 门描述符的 8 字节二进制布局（逐 bit）
- 理解 Interrupt Gate 与 Trap Gate 的区别
- 设计 ISR 汇编 stub → C handler 的栈布局约定
- 实现 `regs_t` 结构体，与汇编 push 顺序严格对应
- 用 `int3` 自检验证整条链路

5.2 背景知识：中断与异常

CPU 在执行指令的过程中，可能被两种事件打断：

- **异常（Exception）**：**同步**事件，由 CPU 执行指令时产生（如除零、非法指令、Page Fault）。异常和造成它的指令有确定的因果关系。
- **中断（Interrupt）**：**异步**事件，由外部硬件触发（如键盘按键、定时器到期、磁盘完成 I/O）。与当前执行的指令无关。

无论是异常还是中断，CPU 的处理方式相同：

1. 用向量号 N（0–255）在 IDT 中查找第 N 个门描述符
2. 从描述符中取出 handler 的地址（`CS`:offset）
3. 在栈上保存当前的 `EFLAGS`、`CS`、`EIP`（发生特权级切换时还保存 `SS`、`ESP`）
4. 跳转到 handler 执行

5.2.1 异常的三种分类

类型	特征	例子
Fault	EIP 指向**故障指令**。handler 修复问题后 `iret` 会重新执行该指令。	#PF (Page Fault)、#G
Trap	EIP 指向故障指令**的下一条**。`iret` 继续执行后续代码。	`int3` (Breakpoint)、`int`
Abort	不可恢复的严重错误，EIP 可能无意义。	#DF (Double Fault)

思考 5.1

Page Fault 为什么是 Fault 而非 Trap?
因为 handler 需要**修复**故障（如分配物理页并建立映射），然后让 CPU **重新执行**那条触发 #PF 的指令。如果 EIP 跳过了故障指令（像 Trap 那样），CPU 就永远不会读到那个页面的数据了。这就是**按需分页**（demand paging）的基础。

5.2.2 向量号分配

向量	助记符	名称	有错误码？
0	#DE	除零	否
1	#DB	调试	否
2	NMI	不可屏蔽中断	否
3	#BP	断点 (`int3`)	否
4	#OF	溢出	否
5	#BR	BOUND 范围超出	否
6	#UD	无效操作码	否
7	#NM	设备不可用	否
8	#DF	Double Fault	**是**（总是 0）
9		协处理器段越界	否
10	#TS	无效 TSS	**是**
11	#NP	段不存在	**是**
12	#SS	栈段错误	**是**
13	#GP	通用保护	**是**
14	#PF	Page Fault	**是**
16	#MF	x87 浮点异常	否
17	#AC	对齐检查	**是**
18	#MC	机器检查	否
19	#XM	SIMD 浮点异常	否

32–47	IRQ0–15	硬件中断（PIC remap 后）	否
128	—	系统调用（`int 0x80`）	否

注意

有错误码的异常和没有错误码的异常，在 ISR 汇编 stub 中必须区别对待。如果搞混了，整个栈布局都会偏移——C handler 读到的寄存器值全是错的。

5.3 IDT 门描述符的 8 字节布局

每个 IDT 条目（gate descriptor）是 8 字节（64 位），格式和 GDT 描述符一样”碎片化”：

5.3.1 内存布局

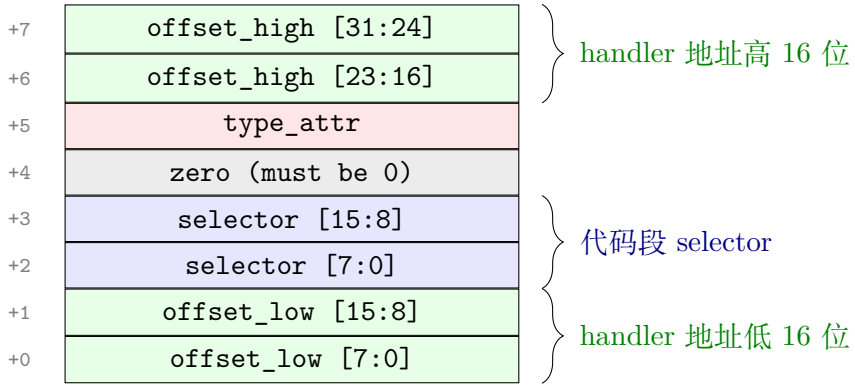

图 5.1: IDT 门描述符的 8 字节内存布局

5.3.2 C 结构体

```
typedef struct __attribute__((packed)) {
    uint16_t offset_low; // handler 地址低 16 位 [0:15]
    uint16_t selector;   // 代码段选择子 (0x08 = 内核代码段)
    uint8_t zero;        // 保留字段，必须为 0
    uint8_t type_attr;   // P(1) | DPL(2) | 0(1) | Type(4)
    uint16_t offset_high; // handler 地址高 16 位 [16:31]
} idt_entry_t;
```

思考 5.2

handler 地址为什么也被拆成 low 和 high 两段？和 GDT 描述符一样，这是 286→386 向后兼容的历史包袱。286 的 IDT 门只有 16 位 offset。

5.3.3 type_attr 字节逐 bit 详解

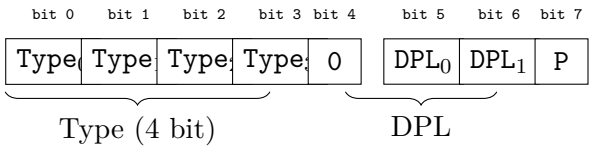

图 5.2: type_attr 字节的 8 个 bit

bit 7 P (Present) = 1：此门有效。P=0 时触发该向量会引发 #NP。

bit 5--6 DPL (Descriptor Privilege Level)：允许通过 `int N` 软件触发此门的最低特权级。DPL=0：只有 Ring 0 能用 `int N` 触发。DPL=3：Ring 3（用户态）也能触发（如 `int 0x80` 系统调用）。**注意**：硬件异常不受 DPL 限制。CPU 除零时无论 DPL 都会进入 handler。

bit 4 固定为 0（区分系统段描述符和门描述符）。

bit 0--3 Type：

- `1110b` = `0xE` = 32-bit Interrupt Gate：进入时**自动清 IF**（关中断）
- `1111b` = `0xF` = 32-bit Trap Gate：进入时**不清 IF**

我们使用 `0x8E`：

$$\texttt{0x8E} = 1\ \underbrace{00}_{DPL=0}\ \underbrace{0}\ \underbrace{1110}_{InterruptGate}$$

设计决策

Interrupt Gate vs Trap Gate——唯一区别是进入 handler 时是否自动清 `EFLAGS.IF`。

Interrupt Gate（`0x8E`）：自动关中断。适合异常处理——处理 #PF 时我们不希望被键盘中断打断，否则嵌套的中断可能在异常尚未修复时访问出错的页面。

Trap Gate（`0x8F`）：不关中断。将来 syscall gate（`int 0x80`）可以考虑用 Trap Gate，让系统调用期间仍能响应硬件中断。

5.3.4 idt_set_gate：填充一个门描述符

```
static void idt_set_gate(uint8_t vector, void (*handler)(void),
                         uint16_t selector, uint8_t type_attr) {
    uint32_t addr = (uint32_t)(uintptr_t)handler;

    idt[vector].offset_low = (uint16_t)(addr & 0xFFFF);
    //  addr = 0xC0101234
    //  addr & 0xFFFF = 0x1234 → offset_low

    idt[vector].selector = selector;
    //  0x08（内核代码段）

    idt[vector].zero     = 0;
    //  保留字段，必须为 0

    idt[vector].type_attr = type_attr;
    //  0x8E = Interrupt Gate, DPL=0, Present

    idt[vector].offset_high = (uint16_t)((addr >> 16) & 0xFFFF);
    //  (addr >> 16) & 0xFFFF = 0xC010 → offset_high
}
```

CPU 触发向量 N 时，从 IDT[N] 拼出 handler 地址：

$$\text{handler} = (\text{offset_high} \ll 16) \mid \text{offset_low}$$

然后用 selector（`0x08`）去 GDT 查代码段描述符，切换到内核代码段执行 handler。

5.3.5 IDTR 结构体

和 GDTR 格式完全相同：6 字节，limit + base。

```
typedef struct __attribute__((packed)) {
    uint16_t limit; // IDT 总字节数 - 1
    uint32_t base; // IDT 数组的线性地址
} idtr_t;
```

256 个门 × 8 字节 = 2048 字节，所以 `limit = 2047`。

5.4 ISR 汇编 Stub：isr_stubs.asm

这是 IDT 实现中最精密的部分——必须保证汇编 push 的顺序与 C 结构体**完全一致**。

5.4.1 问题：栈布局不统一

当异常发生时，CPU 自动压栈：

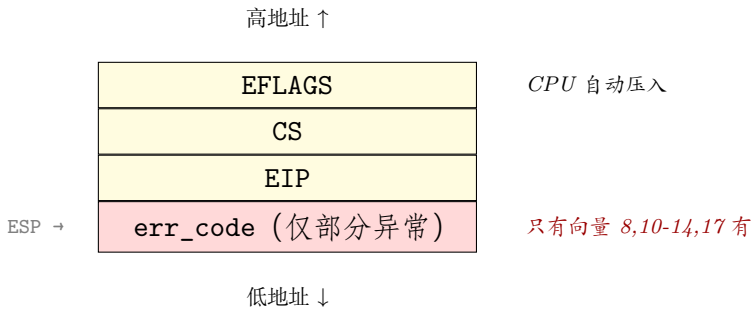

图 5.3: 异常入口时 CPU 自动压栈的内容

问题：有些异常（如 #PF）CPU 会额外压入一个 error code，有些（如 #DE）不会。如果不处理这个差异，C handler 无法用统一的结构体解析栈。

5.4.2 解决方案：ISR_NOERR 和 ISR_ERR 宏

```
; 无错误码的异常：手动 push 0 对齐
%macro ISR_NOERR 1
isr%1:
    push dword 0        ; 伪 err_code = 0
    push dword %1       ; int_no
    jmp isr_common_stub
%endmacro

; 有错误码的异常：CPU 已经 push 了 err_code
%macro ISR_ERR 1
isr%1:
    push dword %1       ; int_no（err_code 已在栈上）
    jmp isr_common_stub
%endmacro
```

效果：无论哪种异常，进入 `isr_common_stub` 时栈顶布局总是统一的：

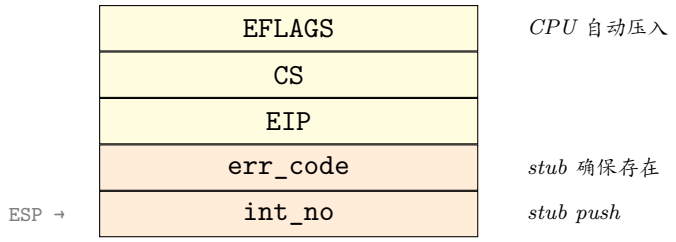

图 5.4: 进入 isr_common_stub 时的统一栈布局

5.4.3 isr_common_stub：逐指令详解

```
isr_common_stub:
    pushad              ; (1) 保存 8 个通用寄存器

    push ds             ; (2) 保存段寄存器
    push es             ; (3)
    push fs             ; (4)
    push gs             ; (5)

    mov ax, 0x10        ; (6) 切换到内核数据段
    mov ds, ax          ; (7)
    mov es, ax          ; (8)
    mov fs, ax          ; (9)
    mov gs, ax          ; (10)

    push esp            ; (11) 参数：指向 regs_t 的指针
    call isr_handler_c ; (12) 调用 C handler
    add esp, 4          ; (13) 清理参数

    pop gs              ; (14) 恢复段寄存器
    pop fs              ; (15)
    pop es              ; (16)
    pop ds              ; (17)

    popad               ; (18) 恢复通用寄存器

    add esp, 8          ; (19) 跳过 int_no 和 err_code

    iret                ; (20) 从中断返回
```

第 2 行：`pushad` `pushad` 一次性压入 8 个通用寄存器，顺序是：EAX → ECX → EDX → EBX → ESP → EBP → ESI → EDI。

注意

`pushad` 压入的 ESP 是**执行 `pushad` 之前**的值（即"当时的快照"），不是当前栈指针。这意味着 `regs_t` 中的 `esp` 字段只是参考值，不能用来恢复栈。`popad` 恢复时会跳过 ESP 字段。

第 3–6 行：保存段寄存器 `push ds/es/fs/gs` 的顺序决定了 `regs_t` 中对应字段的位置。**顺序必须严格匹配**——下面会看到 C 结构体的字段排列。

第 7–11 行：切换到内核数据段 异常可能发生在段寄存器指向非内核段的时候（将来用户态会遇到）。在调用 C handler 前，先确保 DS/ES 指向内核数据段 `0x10`。

第 12–14 行：调用 C handler `push esp` 把当前栈指针作为参数传给 `isr_handler_c`。此时 ESP 恰好指向刚才 push 的所有数据的起始位置——也就是 `regs_t*`。

第 20 行：`add esp, 8` 跳过栈上的 `int_no` 和 `err_code`（各 4 字节），让 ESP 指向 CPU 自动压入的 EIP/CS/EFLAGS。

第 21 行：`iret` `iret`（Interrupt Return）从栈上弹出 EIP、CS、EFLAGS，恢复到中断前的执行状态。

CPU 状态变化

`iret` 执行时 CPU 弹出三个值：

1. 弹出 EIP → 恢复程序计数器
2. 弹出 CS → 恢复代码段（如果 CS.RPL 变了，触发特权级切换）
3. 弹出 EFLAGS → 恢复标志位（包括 IF，如果原来中断是开的就恢复开启）

如果发生了特权级切换（如从 Ring 0 回到 Ring 3），还会额外弹出 ESP 和 SS。

5.5 完整栈布局图（ISR 入口到 C handler）

这张图展示了从异常发生到进入 C handler 时，栈上所有数据的排列：

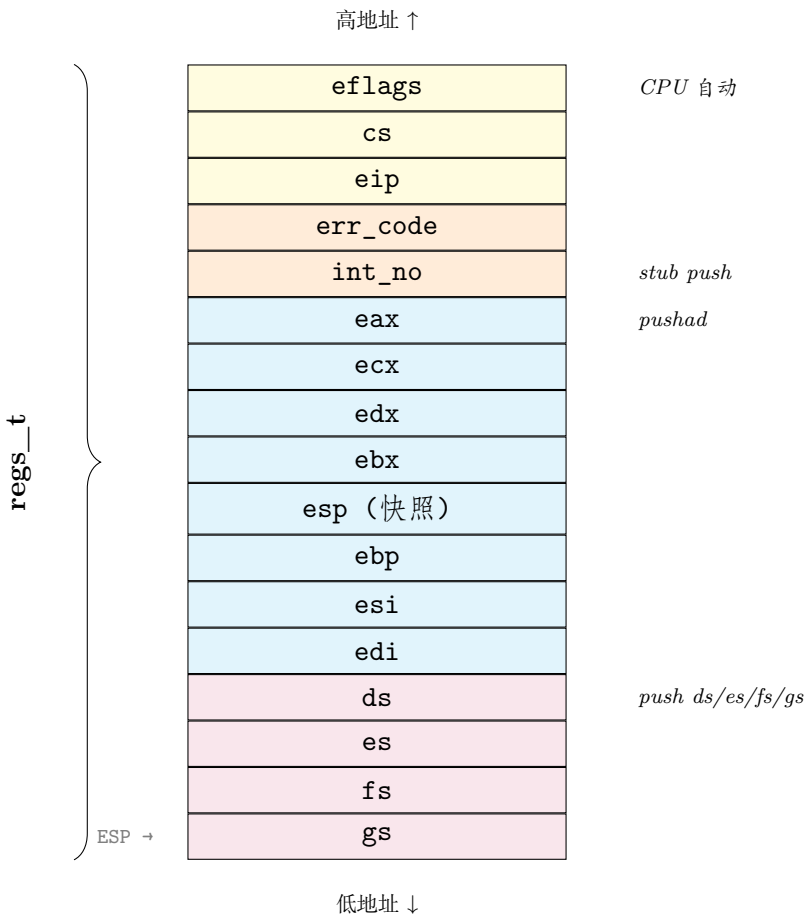

图 5.5: ISR 入口到 push esp（传给 C handler）时的完整栈布局

5.6 regs_t：与汇编栈布局一一对应的 C 结构体

```
typedef struct regs {
    // push gs/fs/es/ds (最后 push 的在最低地址 = 结构体最前面)
    uint32_t gs, fs, es, ds;

    // pushad (EAX 最先 push = 最高地址 → 结构体中排在后面)
    // pushad 的 push 顺序: EAX ECX EDX EBX ESP EBP ESI EDI
    // 栈上从高到低:      EAX ECX EDX EBX ESP EBP ESI EDI
    // 结构体从前到后:    edi esi ebp esp ebx edx ecx eax
    uint32_t edi, esi, ebp, esp, ebx, edx, ecx, eax;

    // ISR stub push 的 int_no 和 err_code
    uint32_t int_no, err_code;

    // CPU 自动压入 (最先压的在最高地址)
```

```
    uint32_t eip, cs, eflags;
} regs_t;
```

注意

regs_t 的字段顺序是由汇编 push 顺序**反向**决定的：

- 最后 push 的数据在最低地址 → 对应结构体的**第一个**字段
- 最先 push 的数据在最高地址 → 对应结构体的**最后一个**字段

如果顺序搞反，C handler 读到的 r->eip 可能是 ds 的值——这种 bug 极难排查。图 5.5 是你的" 参照表"。

5.7 C 异常处理：isr_handler_c

```c
void isr_handler_c(const regs_t* r) {
    // Page Fault 特殊路径
    if (r->int_no == 14) {
        page_fault_handler(r);
        printk("Kernel panic: #PF Page Fault\n");
        for (;;) __asm__ volatile("hlt");
    }

    // 打印异常信息
    printk("\n[isr] %s vector=%u err=0x%08x eip=0x%08x\n",
           exception_name(r->int_no),
           r->int_no, r->err_code, r->eip);

    // Breakpoint (int3) 允许返回 → 用于自检
    if (r->int_no == 3) return;

    // 其他异常不可恢复
    printk("Kernel panic: %s\n", exception_name(r->int_no));
    for (;;) __asm__ volatile("hlt");
}
```

5.7.1 Page Fault Handler：CR2 与错误码

```
static inline uint32_t read_cr2(void) {
    uint32_t val;
    __asm__ volatile("mov %%cr2, %0" : "=r"(val));
    return val;
}
```

CR2 在 #PF 发生时由 CPU 自动写入故障虚拟地址。

错误码的逐 bit 含义：

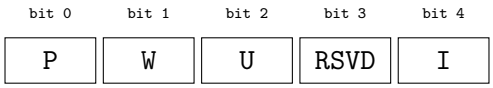

图 5.6: #PF 错误码的低 5 位

bit 0 P (Present)：0= 页不存在（not-present），1= 权限违规（页存在但不允许访问）

bit 1 W/R：0= 读操作引起，1= 写操作引起

bit 2 U/S：0= 在 Ring 0（内核态）发生，1= 在 Ring 3（用户态）发生

bit 3 RSVD：PTE 中的保留位被错误置位（硬件级错误，通常是写 PTE 的代码有 bug）

bit 4 I/D：1= 取指令（instruction fetch）引起，0= 数据访问引起

常见错误码示例：

err_code	含义
0x00	内核态读一个不存在的页
0x02	内核态写一个不存在的页
0x05	用户态读一个存在但权限不足的页
0x07	用户态写一个存在但权限不足的页

5.8 idt_init()：安装所有 gate

```
void idt_init(void) {
    // 清空 IDT（256 项全部归零）
    for (size_t i = 0; i < 256; i++) {
        idt[i] = (idt_entry_t){0};
    }

```

```
    // CPU 异常 0..31 (全部用 Interrupt Gate 0x8E)
    const uint8_t INT_GATE = 0x8E;
    idt_set_gate(0, isr0, GDT_KERNEL_CODE_SEL, INT_GATE);
    idt_set_gate(1, isr1, GDT_KERNEL_CODE_SEL, INT_GATE);
    // ... 每个向量对应自己的 stub ...
    idt_set_gate(14, isr14, GDT_KERNEL_CODE_SEL, INT_GATE); // #PF
    // ... 一直到 31 ...

    // 硬件 IRQ 0x20..0x2F (先装 gate, PIC remap 在下一章)
    idt_set_gate(0x20, irq0, GDT_KERNEL_CODE_SEL, INT_GATE);
    idt_set_gate(0x21, irq1, GDT_KERNEL_CODE_SEL, INT_GATE);
    // ... 到 0x2F ...

    // 加载 IDTR
    idtr.limit = (uint16_t)(sizeof(idt) - 1); // 256*8-1 = 2047
    idtr.base = (uint32_t)(uintptr_t)&idt[0];
    lidt(&idtr);
}
```

CPU 状态变化

`lidt` 执行后：

- IDTR.limit = 2047, IDTR.base = &idt[0]
- 从此刻起，任何异常都会通过新 IDT 分发
- 中断仍然关闭（IF=0），直到我们显式 `sti`
- 未安装的向量（32–255 中大部分）条目全为 0，触发时会引发 #GP

5.9 验证：int3 自检

```
// kmain.c
idt_init();
printk("[init] idt ok\n");
__asm__ volatile("int3"); // 触发 #BP
printk("int3 returned, IDT works!\n");
```

预期输出：

```
[init] idt ok
```

```
[isr] #BP Breakpoint vector=3 err=0x00000000 eip=0xC01xxxxx
int3 returned, IDT works!
```

如果看不到 “int3 returned”，说明 `iret` 回不来——检查栈布局。

5.9.1 GDB 逐指令验证

```
(gdb) b isr_handler_c
(gdb) c
Breakpoint, isr_handler_c (r=0xc01xxxxx)
(gdb) p r->int_no
$1 = 3
(gdb) p/x r->eip
$2 = 0xc010abcd  # int3 指令的下一条地址（Trap 语义）
(gdb) p/x r->err_code
$3 = 0x0          # int3 没有错误码
```

5.10 思考与练习

练习 5.1

在 `kmain` 中触发除零异常：

`__asm__ volatile("int $0");`

观察串口输出的 EIP。它指向哪条指令？和 `int3` 的行为有什么不同？（提示：`int N` 对所有 N 都是 Trap 语义——EIP 指向下一条。但真正的除零 `1/0` 是 Fault——EIP 指向 `div` 指令本身。）

练习 5.2

故意将 `isr_common_stub` 中 `pushad` 和 `push ds/es/fs/gs` 的顺序对调。然后触发 `int3`，看 C handler 打印的 `r->eip`。它还正确吗？参考图 5.5 分析为什么。

练习 5.3

将所有异常的 `type_attr` 从 `0x8E`（Interrupt Gate）改为 `0x8F`（Trap Gate）。运行后键盘是否还能正常输入？（提示：Trap Gate 不关中断，异常处理期间可能被键盘 IRQ 打断。在当前简单内核中可能没问题，但在更复杂的场景下会导致嵌套中断问题。）

练习 5.4

（挑战）当前 `idt_init` 中 `int3` 的 DPL=0。这意味着将来的用户态程序不能用 `int3` 做调试断点。把它改成 DPL=3（`type_attr = 0xEE`），并解释 `0xEE` 的逐 bit 含义。

5.11 本章小结

IDT 是内核的”异常/中断事件分发表”。本章涵盖了：

1. **IDT 门描述符**（8 字节）：offset 拆分 + selector + type_attr 逐 bit
2. **ISR 汇编 stub**：NOERR/ERR 宏统一栈布局 → pushad → push 段寄存器 → call C
3. **regs_t 结构体**：字段顺序严格对应汇编 push 顺序（图 5.5）
4. **Page Fault 特殊处理**：读 `CR2` + 5 bit 错误码解析
5. **int3 自检**：验证整条链路 ISR stub → C handler → iret

有了 IDT，内核不再”闷死”——任何 CPU 错误都会被捕获并打印诊断信息。下一章我们接入硬件中断：初始化 8259 PIC，让键盘能触发 IRQ 并输入字符。

Chapter 6

IRQ、PIC 与键盘驱动

6.1 本章目标

- 理解 8259 PIC 的硬件架构：主从级联、ICW 初始化序列、IRQ mask、EOI
- 理解为什么必须 remap PIC（避免与 CPU 异常冲突）
- 实现 IRQ 分发框架：函数指针数组 + 统一 EOI
- 理解 IRQ 汇编 stub 与 ISR stub 的关系
- 实现 PS/2 键盘驱动：scancode 解码、Shift 状态机、环形缓冲区
- 掌握中断上下文与主循环之间的生产者-消费者模式

6.2 背景知识：8259 PIC 硬件架构

PC 有两片级联的 8259A PIC（Programmable Interrupt Controller），
每片管理 8 条 IRQ 线，合计 16 条：

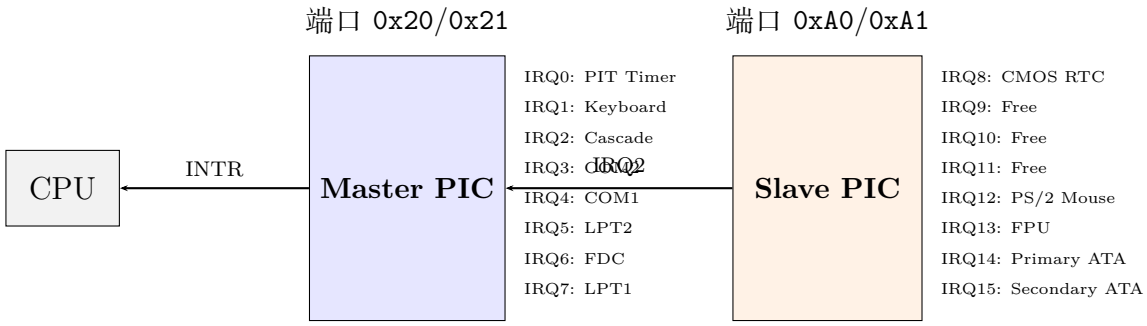

图 6.1: 8259 PIC 主从级联架构与 IRQ 分配

6.2.1 为什么必须 Remap？

PIC 上电后的默认映射：

IRQ	默认向量	冲突
IRQ0	8	#DF (Double Fault)
IRQ1	9	Coprocessor Segment Overrun
IRQ7	15	Reserved

IRQ0-7 的默认向量 8-15 和 CPU 异常**完全重叠**！如果不 remap，按键盘（IRQ1）会被 CPU 当成向量 9（Coprocessor Segment Overrun）处理。

思考 6.1

如果你忘了做 remap 就启用了 IRQ1，按键盘后会发生什么？CPU 认为向量 9 是一个异常，会调用 `isr9` 的 handler。ISR handler 打印 “Coprocessor Segment Overrun” 然后 halt。键盘表现为：按一下就死机。

6.3 PIC 初始化：ICW 序列详解

PIC 通过 4 个 ICW（Initialization Command Word）完成初始化。每个 ICW 有精确的格式和发送顺序：

```
void pic_init(void) {
    // 保存当前 mask（可选，我们会覆盖）
    uint8_t a1 = inb(PIC1_DATA);
    uint8_t a2 = inb(PIC2_DATA);
```

6.3.1 ICW1：开始初始化

```
    outb(PIC1_CMD, 0x11); // ICW1: INIT + ICW4_NEEDED
    io_wait();
    outb(PIC2_CMD, 0x11);
    io_wait();
```

`0x11` = ICW1_INIT (bit 4=1) | ICW1_ICW4 (bit 0=1)。

向命令端口写入 ICW1 告诉 PIC：” 开始初始化序列，后面会有 ICW4”。

为什么？

io_wait() 是什么？ISA 总线的某些旧设备需要在连续 I/O 操作间插入一个短延迟。常见实现是向端口 0x80（POST 诊断端口）写一个字节：

```
static inline void io_wait(void) {
    outb(0x80, 0); // 约 1 微秒延迟
}
```

6.3.2 ICW2：向量偏移

```
    outb(PIC1_DATA, 0x20); // master: IRQ0 → 向量 0x20
    io_wait();
    outb(PIC2_DATA, 0x28); // slave: IRQ8 → 向量 0x28
    io_wait();
```

这就是 **remap 的核心**：告诉每片 PIC 它管理的 IRQ 对应的起始向量号。

Remap 后的映射：

IRQ	向量	设备
0	0x20 (32)	PIT Timer
1	0x21 (33)	PS/2 Keyboard
2	0x22 (34)	Cascade（slave PIC）
8	0x28 (40)	CMOS RTC
14	0x2E (46)	Primary ATA

6.3.3 ICW3：级联关系

```
    outb(PIC1_DATA, 0x04); // master: slave 挂在 IRQ2 (bit 2 = 1)
    io_wait();
    outb(PIC2_DATA, 0x02); // slave: cascade identity = 2
    io_wait();
```

Master 的 ICW3 是一个 bitmask：bit $N = 1$ 表示 IRQN 上挂了一个 slave PIC。0x04 = bit 2 = IRQ2 上有 slave。

Slave 的 ICW3 是自身的 cascade identity：2 表示”我挂在 master 的 IRQ2 上”。

图 6.2: Master PIC 的 ICW3 = `0x04`：IRQ2 位置为 1

6.3.4 ICW4：模式设置

```
    outb(PIC1_DATA, 0x01); // 8086/88 模式
    io_wait();
    outb(PIC2_DATA, 0x01);
    io_wait();
```

`0x01` = ICW4_8086：告诉 PIC 工作在 8086 模式（相对于 8085 模式）。现代 x86 都用这个。

6.3.5 初始 Mask：全部关闭

```
    outb(PIC1_DATA, 0xFF); // mask 所有 master IRQ
    outb(PIC2_DATA, 0xFF); // mask 所有 slave IRQ
}
```

设计决策

初始化后先把所有 IRQ mask 掉。原因：

1. 此时大部分 IRQ 还没安装 handler。如果中断进来但 IDT 中没有有效的 gate，CPU 会触发 #GP。
2. 每个驱动在 init 时自己调用 `pic_clear_mask(irq)` 打开需要的 IRQ —" 谁用谁开" 原则。

6.4 EOI（End of Interrupt）

IRQ handler 执行完后**必须**向 PIC 发送 EOI，否则 PIC 认为当前中断还在处理中，不会发送同级或更低优先级的后续中断。

```
void pic_send_eoi(uint8_t irq) {
    if (irq >= 8) {
        outb(PIC2_CMD, 0x20); // 先通知 slave
```

```
    }
    outb(PIC1_CMD, 0x20);  // 再通知 master（所有 IRQ 都要）
}
```

注意

忘记发 EOI 是最常见的硬件中断 bug。表现为：第一次按键有反应，之后键盘完全死掉。因为 PIC 还在等待上一次 IRQ1 的 EOI，拒绝发送新的中断。

思考 6.2

为什么 slave IRQ（8–15）需要向**两片** PIC 都发 EOI?
因为 slave 是挂在 master 的 IRQ2 上的。一次 slave IRQ 同时占用了：

1. slave PIC 的一个 IRQ 槽位
2. master PIC 的 IRQ2 槽位

两边都要通知释放。

6.5 IRQ Mask 操作

```
void pic_clear_mask(uint8_t irq) {
    uint16_t port = (irq < 8) ? PIC1_DATA : PIC2_DATA;
    uint8_t line = (uint8_t)(irq & 7u);

    uint8_t mask = inb(port);     // 读当前 mask
    mask &= (uint8_t)~(1u << line); // 清除对应 bit（0=允许）
    outb(port, mask);              // 写回
}

void pic_set_mask(uint8_t irq) {
    // ...同理，把对应 bit 置 1（1=屏蔽）
}
```

PIC 的 mask 寄存器（IMR）是一个 8-bit 值，每 bit 对应一个 IRQ：

- bit = 0：允许该 IRQ 通过
- bit = 1：屏蔽该 IRQ

6.6 IRQ 分发框架：irq.c

6.6.1 函数指针数组

```c
static irq_handler_t g_irq_handlers[16]; // 初始全为 NULL

void irq_install_handler(uint8_t irq, irq_handler_t handler) {
    if (irq < 16) {
        g_irq_handlers[irq] = handler;
    }
}
```

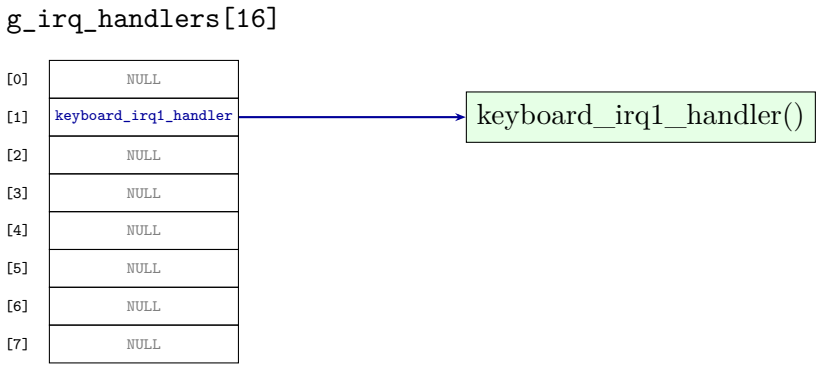

图 6.3: IRQ 分发表：g_irq_handlers 函数指针数组

6.6.2 统一 C 入口

```c
void irq_handler_c(const irq_regs_t* r) {
    uint8_t irq = (uint8_t)(r->int_no - 0x20u);

    // 调用已注册的 handler (如果有)
    if (irq < 16 && g_irq_handlers[irq]) {
        g_irq_handlers[irq](r);
    }

    // 统一发送 EOI
    pic_send_eoi(irq);
}
```

设计决策

EOI 在 dispatcher 中统一发送，而非让每个 handler 自己发。好处：

1. 驱动开发者不会忘记发 EOI
2. handler 只需专注于设备逻辑
3. 将来做更复杂的 IRQ 优先级管理时只改 dispatcher 一处

6.6.3 IRQ 汇编 Stub

IRQ stub 和 ISR stub 结构相同（push err_code=0 → push int_no → pushad → call C）：

```
%macro IRQ 2
irq%1:
    push dword 0        ; err_code = 0 (硬件 IRQ 无 error code)
    push dword %2       ; int_no = vector (0x20 + irq)
    jmp irq_common_stub
%endmacro

IRQ 0, 0x20  ; IRQ0 → 向量 0x20
IRQ 1, 0x21  ; IRQ1 → 向量 0x21 (键盘)
; ... 共 16 个 ...
```

`irq_common_stub` 和 `isr_common_stub` 完全一样（pushad → push 段寄存器 → call C → 恢复 → iret），只是调用的 C 函数不同（`irq_handler_c` vs `isr_handler_c`）。

6.7 PS/2 键盘驱动：`keyboard.c`

6.7.1 Scancode Set 1

PS/2 键盘按下键时发送一个 **make code**（0x00–0x7F），释放时发送 **break code**（make code | 0x80）。

```
// Scancode Set 1 基础映射 (部分)
static const char scancode_map[0x40] = {
    /* 0x00 */ 0, 0, '1','2','3','4','5','6','7','8','9','0','-','=',
    /* 0x0E */ '\b', '\t',
    /* 0x10 */ 'q','w','e','r','t','y','u','i','o','p','[',']','\n', 0,
    /* 0x1E */ 'a','s','d','f','g','h','j','k','l',';','\'','`', 0, '\\',
    /* 0x2C */ 'z','x','c','v','b','n','m',',','.','/', 0, 0, 0, ' ',
```

```
};
```

思考 6.3

为什么键盘发送的是 scancode 而非 ASCII?
因为键盘是一个" 物理开关阵列"——它只知道哪个位置的键被按了或松了。把 scancode 翻译成哪个字符取决于**键盘布局**（US、UK、DE、中文等）。同一个 scancode 0x10 在 QWERTY 布局是'q'，在 AZERTY 布局是'a'。翻译工作由操作系统（或键盘驱动）完成。

6.7.2 Shift 状态机

```c
static volatile uint8_t g_shift_down; // Shift 是否被按住

static void keyboard_handle_scancode(uint8_t sc) {
    // 0xE0 前缀：扩展键（方向键等），暂时忽略
    if (sc == 0xE0u) { g_e0_prefix = 1; return; }
    if (g_e0_prefix) { g_e0_prefix = 0; return; }

    uint8_t released = (sc & 0x80u) != 0; // 最高位=1 表示松开
    uint8_t code = sc & 0x7Fu;          // 低 7 位 = make code

    // Left Shift (0x2A) 或 Right Shift (0x36)
    if (code == 0x2Au || code == 0x36u) {
        g_shift_down = released ? 0 : 1;
        return;
    }

    if (released) return; // 只处理按下事件

    // 根据 Shift 状态选择映射表
    char ch = g_shift_down ? scancode_map_shift[code]
                           : scancode_map[code];
    if (ch) kbd_buf_put(ch);
}
```

6.7.3 环形缓冲区

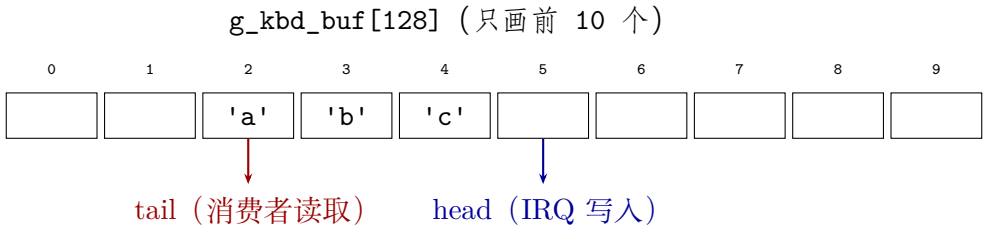

图 6.4: 键盘环形缓冲区：IRQ handler 写 head，主循环读 tail

```c
#define KBD_BUF_SIZE 128

static volatile char g_kbd_buf[KBD_BUF_SIZE];
static volatile uint8_t g_kbd_head; // 下一个写位置
static volatile uint8_t g_kbd_tail; // 下一个读位置

static void kbd_buf_put(char c) {
    uint8_t next = (g_kbd_head + 1) % KBD_BUF_SIZE;
    if (next == g_kbd_tail) return; // 满了 → 丢弃
    g_kbd_buf[g_kbd_head] = c;
    g_kbd_head = next;
}

static int kbd_buf_get(char* out) {
    if (g_kbd_tail == g_kbd_head) return 0; // 空
    *out = g_kbd_buf[g_kbd_tail];
    g_kbd_tail = (g_kbd_tail + 1) % KBD_BUF_SIZE;
    return 1;
}
```

设计决策

为什么用环形缓冲区而非普通数组？

1. **解耦生产者和消费者**：IRQ handler（生产者）在中断上下文执行，必须尽快返回。Shell 主循环（消费者）在非中断上下文运行，可以慢慢处理。
2. **避免数据拷贝**：环形缓冲区通过 head/tail 指针管理读写位置，不需要移动数组元素。
3. **线程安全**：单生产者 + 单消费者的环形缓冲区在不需要锁的情况下就是线

程安全的（原子的 head/tail 更新）。

思考 6.4

g_kbd_buf 和 g_kbd_head/tail 为什么要声明为 volatile?
因为它们在两个"执行流"中被访问：

- IRQ handler（中断上下文）写 g_kbd_head
- keyboard_getc（主循环）读 g_kbd_head

没有 volatile，编译器可能把 g_kbd_head 缓存在寄存器中，导致主循环永远看到旧值——缓冲区"永远为空"。

6.7.4 公开 API

```
// 阻塞等待一个字符（hlt 直到 IRQ 把字符放入缓冲区）
char keyboard_getc(void) {
    char c;
    while (!keyboard_try_getc(&c)) {
        __asm__ volatile("hlt"); // 等下一个中断
    }
    return c;
}

// 非阻塞尝试读取（0=无字符，1=读到一个）
int keyboard_try_getc(char* out) {
    return kbd_buf_get(out);
}
```

为什么？

为什么用 hlt 而非空循环?
hlt 让 CPU 进入低功耗状态直到下一个中断。空循环（while(1){}）让 CPU 100% 占用，白白浪费电力和 QEMU 宿主机 CPU 资源。

6.7.5 keyboard_init：注册 + 开 IRQ

```
void keyboard_init(void) {
    irq_install_handler(1, keyboard_irq1_handler);
```

```
    pic_clear_mask(1); // 允许 IRQ1 通过 PIC
}
```

注意此时 EFLAGS.IF 仍然是 0（中断关闭）。IRQ handler 不会触发，直到 `kmain` 末尾执行 `__asm__ volatile("sti")`。

6.8 完整中断链路图

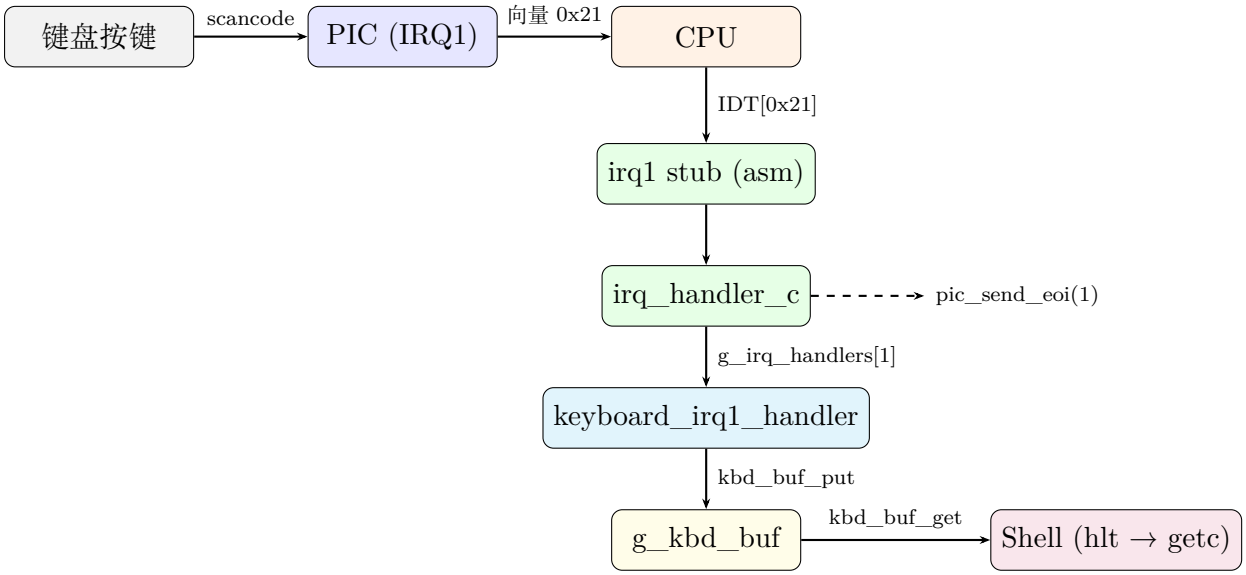

图 6.5: 从键盘按键到 Shell 读取的完整中断链路

6.9 验证

```
make iso && make run
# 在 QEMU 窗口中按键，串口应看到字符回显
# 按 Shift+a 应显示 'A'
# 快速连续打字不应丢字符（除非超过 128 个未消费的字符）
```

6.10 思考与练习

练习 6.1

注释掉 `irq_handler_c` 中 `pic_send_eoi(irq)` 的调用。重新运行，第一次按键后再按键还有反应吗？恢复后试试只给 master 发 EOI （不给 slave），IRQ8–15 的设备是否受影响？

练习 6.2

修改 `KBD_BUF_SIZE` 为 4（极小缓冲区）。快速连续打字 10 个字符，`keyboard_getc` 最多能读到几个？改成 2 呢？思考：环形缓冲区的有效容量是 `size - 1` 还是 `size`？（提示：head == tail 表示空，那满的时候 head 和 tail 是什么关系？）

练习 6.3

在键盘驱动中添加对 Caps Lock 的支持。Caps Lock 的 make code 是 `0x3A`。它是一个" 切换键"——按一次开启，再按一次关闭。提示：维护一个 `g_caps_lock` 状态，按下时 toggle。Caps Lock 只影响字母键的大小写，不影响数字和符号。

练习 6.4

（挑战）当前 `0xE0` 前缀（方向键等）被直接忽略。实现方向键的处理：上箭头（`0xE0 0x48`）、下箭头（`0xE0 0x50`）、左箭头（`0xE0 0x4B`）、右箭头（`0xE0 0x4D`）。把它们翻译成 ANSI 转义序列（如 `\x1b[A` 表示上箭头），放入键盘缓冲区。Shell 可以用这些序列实现光标移动。

6.11 本章小结

本章完成了硬件中断的完整链路：

1. **PIC 初始化**：ICW1-4 序列完成 remap（IRQ → 向量 0x20+），默认 mask 全关
2. **IRQ 分发**：函数指针数组 + 统一 EOI
3. **键盘驱动**：scancode → Shift 状态机 → ASCII → 环形缓冲区
4. **生产者-消费者**：IRQ handler 写 head，主循环读 tail，`volatile` 保证可见性

内核现在能接收用户输入了。下一章我们用这个能力构建交互式 Shell。

Chapter 7

交互式 Shell

7.1 本章目标

- 设计统一的 Console 输入抽象层（键盘 IRQ + 串口轮询）
- 实现行编辑器：回显、退格（`\b`）、回车
- 实现命令解析器：空格分词 → `argc/argv`
- 设计可扩展的命令注册表（`cmd_t` 结构体 + 静态数组）
- 实现第一批命令：`cls`、`shutdown`、`cmds`

7.2 架构设计

Shell 采用五层架构，各层通过清晰的函数接口解耦：

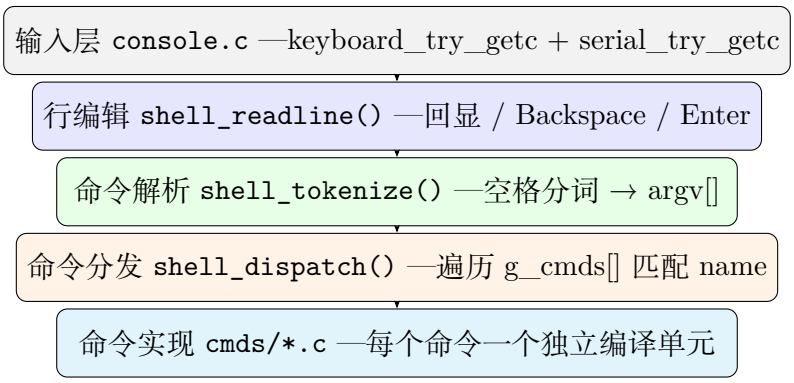

图 7.1: Shell 五层架构

思考 7.1

为什么要把输入抽象成 Console 层？如果 Shell 直接调 `keyboard_getc()`，将来要支持串口远程调试（QEMU `-serial stdio`）时需要改 shell.c。有了 Console 层，输入源的增减只改 console.c 一处。

7.3 Console 输入层：console.c

```c
int console_try_getc(char* out) {
    // 优先查键盘 IRQ 缓冲区（非阻塞）
    if (keyboard_try_getc(out)) return 1;

    // 其次查串口（轮询，非阻塞）
    if (serial_try_getc(out)) {
        if (*out == '\r') *out = '\n'; // 串口终端发 CR，归一化为 LF
        return 1;
    }
    return 0; // 无输入
}

char console_getc(void) {
    for (;;) {
        char c;
        if (console_try_getc(&c)) return c;
        __asm__ volatile("pause"); // 让出 CPU 一个周期
    }
}
```

为什么？

为什么 `console_getc` 用 `pause` 而非 `hlt`？

`hlt` 会让 CPU 完全停止直到下一个中断。但 `serial_try_getc` 是轮询模式（不依赖中断），`hlt` 后如果没有键盘中断就永远不会被唤醒。`pause` 只是一个短暂提示（用于 spin-wait），CPU 仍然继续执行循环。

更好的方案是在每次 `hlt` 前 check 一次串口，但当前简单实现已够用。

7.4 命令接口：cmd.h

```c
typedef int (*cmd_fn_t)(int argc, char** argv);

typedef struct {
    const char* name; // 命令名（如 "cls"）
    const char* help; // 帮助文本
    cmd_fn_t fn;      // 处理函数
} cmd_t;
```

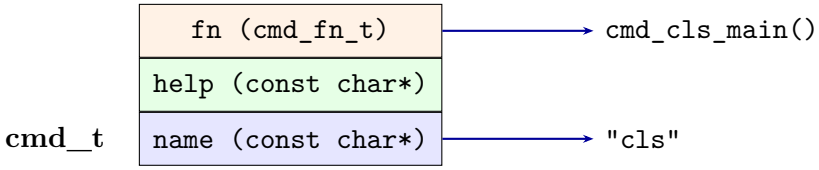

图 7.2: cmd_t 结构体的内存布局与指针指向

每个命令做成独立编译单元（一个 .c 文件），在文件末尾导出一个 const cmd_t：

```c
// cmd_cls.c
static int cmd_cls_main(int argc, char** argv) {
    (void)argc; (void)argv;
    printk("\x1b[2J\x1b[H"); // ANSI 清屏 + 光标归位
    return 0;
}

const cmd_t cmd_cls = {
    .name = "cls",
    .help = "Clear screen (serial ANSI)",
    .fn   = cmd_cls_main,
};
```

设计决策

为什么用 const cmd_t（全局常量）而非注册函数？

1. 编译期确定：不需要运行时动态分配内存（此时堆还没初始化）
2. 链接器自动包含：只要 .o 文件被链接进来，cmd_t 就存在
3. Shell 只需 extern 声明 + 加入数组即可

7.5 命令注册表与分发

7.5.1 注册表：静态指针数组

```
// shell.c 中
extern const cmd_t cmd_cls;
extern const cmd_t cmd_shutdown;
extern const cmd_t cmd_cmds;
// ... 每加一个命令就加一行 extern ...

static const cmd_t* g_cmds[] = {
    &cmd_cls,
    &cmd_shutdown,
    &cmd_cmds,
    &cmd_mmap,
    &cmd_free,
    &cmd_pmm,
    &cmd_vmm,
    &cmd_heap,
    &cmd_vma,
};
```

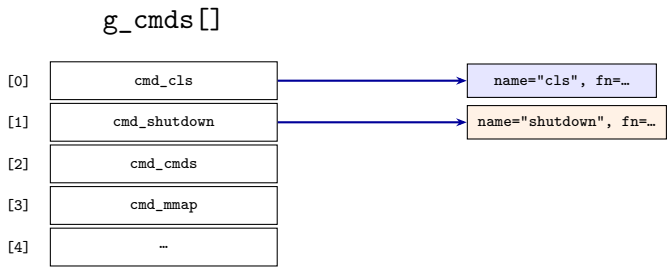

图 7.3: g_cmds[] 是指向各 cmd_t 常量的指针数组

7.5.2 分发逻辑

```
static void shell_dispatch(char* line) {
    if (!line || !*line) return;

    char* argv[16];
    int argc = shell_tokenize(line, argv, 16);
    if (argc <= 0) return;

```

```
    // 遍历注册表，匹配 name
    for (size_t i = 0; i < sizeof(g_cmds)/sizeof(g_cmds[0]); i++) {
        if (streq(argv[0], g_cmds[i]->name)) {
            g_cmds[i]->fn(argc, argv);
            return;
        }
    }

    printk("Unknown command: %s\n", argv[0]);
}
```

思考 7.2

当前遍历是 $O(n)$——对几十个命令完全够用。如果将来命令数量上千（不太可能在内核态），可以改为排序数组 + 二分查找或哈希表。但这是过度工程化——**YAGNI**（You Aren't Gonna Need It）。

7.6 行编辑器：shell_readline

```
static size_t shell_readline(char* buf, size_t cap) {
    size_t len = 0;
    for (;;) {
        char c = console_getc(); // 阻塞等待一个字符

        if (c == '\n') {
            printk("\n");
            buf[len] = '\0';
            return len;
        }

        // Backspace (0x08) 或 DEL (0x7F)
        if (c == '\b' || (unsigned char)c == 0x7F) {
            if (len > 0) {
                len--;
                printk("\b \b"); // 退格 + 空格覆盖 + 再退格
            }
            continue;
        }

```

```
        // 忽略其他控制字符 (< 0x20)
        if ((unsigned char)c < 0x20) continue;

        // 可见字符: 存入缓冲区 + 回显
        if (len + 1 < cap) {
            buf[len++] = c;
            printk("%c", c);
        }
    }
}
```

退格的三步序列 \b \ \b 详解:

szy-kernel > helo	按 *Backspace* 前
szy-kernel > hel␣	*\b* 退格 → 空格覆盖 *'o'*
szy-kernel > hel	再 *\b* 退格到正确位置

图 7.4: 退格三步序列: \b (退格) → ' ' (覆盖) → \b (归位)

7.6.1 命令分词: shell_tokenize

```
static int shell_tokenize(char* line, char** argv, int argv_cap) {
    int argc = 0;
    char* p = line;
    while (*p) {
        while (*p && is_space(*p)) p++; // 跳过空白
        if (!*p) break;
        if (argc >= argv_cap) break;
        argv[argc++] = p;           // 记录 token 起始
        while (*p && !is_space(*p)) p++; // 跳过非空白
        if (*p) { *p = '\0'; p++; } // 用 NUL 截断 token
    }
    return argc;
}
```

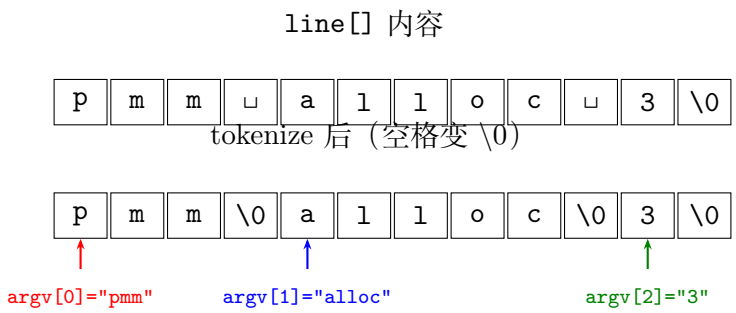

图 7.5: tokenize 将空格替换为 \0，argv[] 指向各 token 起始

注意

shell_tokenize 会**修改**输入字符串（用 \0 替换空格）。这是 C 中常见的”零拷贝分词”技巧——不分配新内存，但调用者不能再使用原始字符串。

7.7 主循环：shell_run

```
void shell_run(void) {
    printk("\nInteractive shell ready.\n");
    char line[SHELL_LINE_MAX]; // 128 字节栈上缓冲区

    for (;;) {
        shell_print_prompt();              // "szy-kernel > "
        shell_readline(line, sizeof(line)); // 阻塞读一行
        shell_dispatch(line);               // 分词 + 匹配 + 执行
    }
}
```

7.8 命令实现示例

7.8.1 cmd_cls：清屏

```
printk("\x1b[2J\x1b[H");
```

- \x1b[2J：ANSI 转义序列——清除整个屏幕
- \x1b[H：光标移到左上角 (1,1)

这只对支持 ANSI 的串口终端有效。QEMU -serial stdio 的宿主终端通常支持。

7.8.2 cmd_shutdown：关机

```
outw(0x604, 0x2000); // QEMU ACPI 关机
outw(0xB004, 0x2000); // Bochs 关机
cpu_halt_forever(); // cli + hlt 循环
```

为什么？

为什么要尝试两个不同的端口？因为 QEMU 和 Bochs 的 ACPI 关机端口不同。如果两个都无效（真实硬件），就退化为 `cli; hlt`——CPU 停止，电源不会关闭但系统不再响应。

7.9 添加新命令的完整流程

1. 创建 `src/kernel/cmds/cmd_xxx.c`
2. 实现 handler 函数并导出：

```
const cmd_t cmd_xxx = {
    .name = "xxx", .help = "...", .fn = cmd_xxx_fn
};
```

3. 在 `shell.c` 中添加 `extern const cmd_t cmd_xxx;` 并加入 `g_cmds[]`
4. 无需修改 Makefile——`kernel/cmds/*.c` 被自动发现编译

7.10 验证

```
szy-kernel > cmds
Commands (9):
cls      shutdown cmds    mmap     free
pmm      vmm      heap    vma
szy-kernel > cls
# 屏幕清空
szy-kernel > shutdown
Shutting down...
# QEMU 窗口关闭
```

7.11 思考与练习

练习 7.1

添加一个 `echo` 命令，将 `argv[1..argc-1]` 用空格连接后打印。示例：`echo hello world` 输出 `hello world`。

练习 7.2

当前 `shell_readline` 的缓冲区是 128 字节。如果用户连续输入 200 个字符（不按回车），超出部分会怎样？修改为在超出时发出 `\a`（BEL，蜂鸣器）提醒。

练习 7.3

实现命令历史：用上箭头键（ANSI `\x1b[A`）回显上一条命令。提示：维护 `history[8][128]` 环形数组。键盘驱动需要先支持方向键（ch05 的挑战练习）。

练习 7.4

（挑战）让 `streq` 支持前缀匹配：输入 `shut` 自动匹配到 `shutdown`（如果只有一个候选）。如果有多个候选，打印所有匹配项。这就是简单的 Tab 补全。

7.12 本章小结

Shell 的核心数据结构：

1. **`cmd_t`**：name + help + fn 函数指针，每个命令一个独立编译单元
2. **`g_cmds[]`**：静态指针数组，编译期确定，遍历分发
3. **行缓冲**：栈上 128 字节，tokenize 原地修改（零拷贝）
4. **Console 层**：统一键盘 + 串口输入，屏蔽输入源差异

Shell 是内核开发阶段最重要的调试工具——后续每实现一个子系统都会立即添加对应的调试命令。下一章进入内存管理：PMM（物理内存分配器）。

Part III

内存管理

Chapter 8

PMM：物理内存管理

8.1 本章目标

- 解析 Multiboot2 Memory Map Tag，获取物理内存布局
- 设计可插拔 PMM 后端接口（`pmm_ops_t` 函数指针表）
- 实现 `pmm.c` dispatch 层
- 实现 bitmap 分配器后端：多 region、位图放置算法、round-robin 分配
- 实现 buddy 分配器后端：order 0–10 分裂/合并
- Shell 命令：`free`、`mmap`、`pmm`

8.2 背景知识：物理内存布局

BIOS 通过 Multiboot2 Memory Map Tag（type=6）报告物理内存区域。一台典型的 256MiB QEMU 虚拟机的内存映射如下：

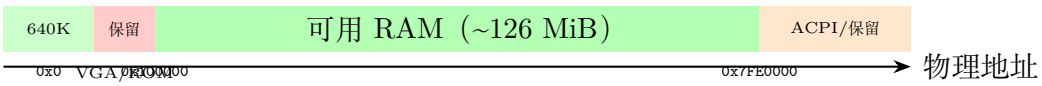

图 8.1: 典型物理内存布局（256MiB QEMU VM）

8.2.1 Multiboot2 Memory Map Tag 格式

```
struct mb2_tag_mmap {
    uint32_t type;       // = 6 (MB2_TAG_TYPE_MMAP)
    uint32_t size;       // tag 总字节数
    uint32_t entry_size; // 每个 entry 的字节数（通常 24）
```

```
    uint32_t entry_version; // = 0
    // 后面紧跟 N 个 mb2_mmap_entry
};

struct mb2_mmap_entry {
    uint64_t addr;       // 区域起始物理地址
    uint64_t len;        // 区域长度（字节）
    uint32_t type;       // 1=Available, 2=Reserved, 3=ACPI, ...
    uint32_t reserved;
};
```

思考 8.1

addr 和 len 是 64 位的。在 32 位内核中，我们只能访问低 4GiB 的物理内存。如何处理报告了 4GiB 以上 RAM 的机器？（答：截断到 0xFFFFFFFF——安全忽略高于 4GiB 的区域。）

8.3 可插拔后端架构

8.3.1 pmm_ops_t：后端操作表

```
typedef struct pmm_ops {
    const char* name;               // "bitmap" / "buddy"

    void (*init)(void);             // 解析 mmap → 建立数据结构
    void* (*alloc_page)(void);      // 分配一个物理页，返回物理地址
    void (*free_page)(void* page);  // 释放一个物理页

    unsigned (*total_pages)(void);  // 管理的总页数
    unsigned (*free_pages)(void);   // 当前空闲页数
    unsigned (*managed_base)(void); // 管理区域的物理起始地址

    unsigned (*page_addr)(unsigned idx); // 页索引 → 物理地址
    int (*page_is_used)(unsigned idx); // 页索引 → 使用状态
} pmm_ops_t;
```

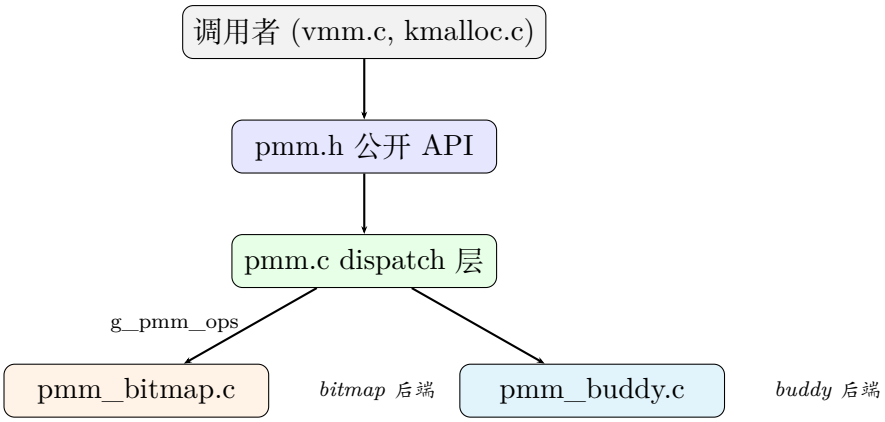

图 8.2: PMM 三层可插拔架构

8.3.2 Dispatch 层：pmm.c

```
static const pmm_ops_t* g_pmm_ops = NULL;

void pmm_register_backend(const pmm_ops_t* ops) {
    g_pmm_ops = ops;
}

void pmm_init(void) {
    if (!g_pmm_ops) g_pmm_ops = pmm_bitmap_get_ops(); // 默认 bitmap
    printk("[pmm] backend: %s\n", g_pmm_ops->name);
    g_pmm_ops->init();
}

void* pmm_alloc_page(void) {
    if (!g_pmm_ops) return NULL;
    return g_pmm_ops->alloc_page();
}

void pmm_free_page(void* page) {
    if (!g_pmm_ops) return;
    g_pmm_ops->free_page(page);
}
// ... 其他函数同理 ...
```

设计决策

这套"ops 函数指针表 + dispatch 薄层" 的模式在本项目中反复出现：

- PMM: `pmm_ops_t` → `pmm.c`
- Heap: `heap_ops_t` → `kmalloc.c`
- VMA: `vma_ops_t` → `vma.c`

这是 Linux 内核 `file_operations` 模式的简化版——接口固定，实现可替换，调用者零改动。

8.4 Bitmap 分配器：pmm_bitmap.c

8.4.1 核心数据结构

```
struct pmm_region {
    uint32_t base;            // 区域起始物理地址
    uint32_t pages;           // 管理的总页数
    uint8_t* bitmap;          // 位图指针（每 bit 对应一页）
    uint32_t bitmap_bytes;    // 位图有效字节数 = ceil(pages/8)
    uint32_t bitmap_storage_bytes; // 实际分配字节数（对齐后）
    uint32_t bitmap_pages;    // 位图本身占用的页数
    uint32_t free;            // 当前空闲页数
};

static struct pmm_region g_regions[PMM_MAX_REGIONS]; // 最多 32 个区域
static uint32_t g_region_count = 0;
```

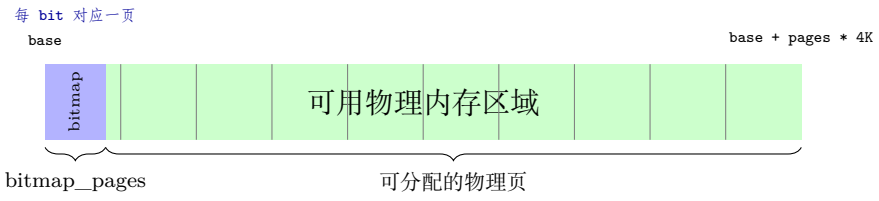

图 8.3: Bitmap 位图放置在区域开头，自身占用几页物理内存

8.4.2 页 → bit 的映射

每个物理页对应位图中的 1 bit：

- bit = 1：页被占用（used/reserved）
- bit = 0：页空闲（free）

```
// 测试第 bit 位是否为 1
static int bitmap_test_r(const struct pmm_region* r, uint32_t bit) {
    uint32_t byte_idx = bit / 8;
    uint8_t bit_mask = (uint8_t)(1u << (bit % 8));
    return (r->bitmap[byte_idx] & bit_mask) != 0;
}

// 置位（标记为 used）
static void bitmap_set_r(struct pmm_region* r, uint32_t bit) {
    r->bitmap[bit / 8] |= (uint8_t)(1u << (bit % 8));
}

// 清位（标记为 free）
static void bitmap_clear_r(struct pmm_region* r, uint32_t bit) {
    r->bitmap[bit / 8] &= (uint8_t)~(1u << (bit % 8));
}
```

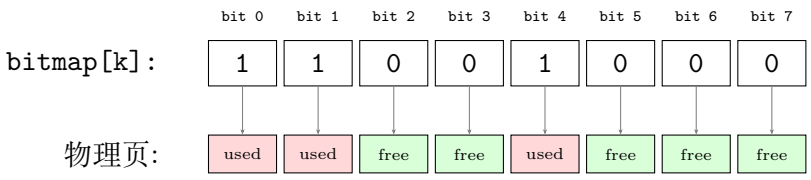

图 8.4: 位图的一个字节（8 bit）对应 8 个连续的物理页

8.4.3 分配算法：Round-Robin + 线性扫描

```
void* pmm_bitmap_alloc_page(void) {
    // Round-robin: 从上次分配的 region 开始，避免总是扫描第一个 region
    for (uint32_t tries = 0; tries < g_region_count; tries++) {
        uint32_t ri = (g_region_rr + tries) % g_region_count;
        struct pmm_region* r = &g_regions[ri];
        if (r->free == 0) continue;

        uint32_t idx;
        if (pmm_find_free_index(r, &idx)) {
            bitmap_set_r(r, idx);   // 标记为 used
```

```
            r->free--;
            g_pmm_total_free--;
            g_region_rr = ri;        // 记住本次从哪个 region 分配的
            return (void*)(uintptr_t)region_index_to_addr(r, idx);
        }
    }
    return NULL; // OOM
}
```

pmm_find_free_index 的核心：先按字节扫描（跳过全 0xFF 的字节），找到有空位的字节后再逐 bit 检查。这把线性扫描的常数因子降低了 8 倍。

思考 8.2

Round-robin 跨 region 分配有什么好处？如果总是从 region 0 开始扫描，region 0 会很快碎片化，而 region 1 始终空闲——分配不均匀。Round-robin 均衡了各 region 的使用率。

8.4.4 释放算法

```
void pmm_bitmap_free_page(void* page) {
    uint32_t addr = (uint32_t)(uintptr_t)page;
    // 安全检查：对齐？在管理范围内？
    if (!pmm_addr_is_page_aligned(addr)) return;

    // 找到包含该地址的 region
    for (uint32_t i = 0; i < g_region_count; i++) {
        struct pmm_region* r = &g_regions[i];
        if (!region_contains_addr(r, addr)) continue;

        uint32_t idx = region_addr_to_index(r, addr);
        if (!bitmap_test_r(r, idx)) {
            printk("[pmm] double free: 0x%08x\n", addr); // 防御
            return;
        }
        bitmap_clear_r(r, idx);
        r->free++;
        g_pmm_total_free++;
        return;
    }
```

```
}
```

注意

Double-free 检测：如果 bit 已经是 0 还要清，说明同一个页被 free 了两次。这是严重 bug，打印警告但不执行清除（避免 free count 错乱）。

8.4.5 初始化流程

`pmm_bitmap_init()` 的完整步骤：

1. 解析 Multiboot2 mmap tag，找出所有 type=1（Available）的区域
2. 对每个区域：截断到 $\geq$ 1MiB、$\leq$ 4GiB、页对齐
3. 计算位图大小和放置位置（`pmm_compute_region_layout`）——位图本身也要占物理内存，需要避开内核镜像和 mb2 info
4. 清零位图 → 将位图自身占用的页标记为 used
5. 保留内核镜像区域（`__kernel_phys_start` 到 `__kernel_phys_end`）
6. 保留 Multiboot2 info 区域
7. 重新计算 free count
8. Selftest：分配 4 页 → 写入 pattern → 读回验证 → 释放 → 检查 free 恢复

设计决策

位图放置是初始化中最复杂的部分。位图需要放在可用物理内存中，但它不能覆盖内核镜像或 Multiboot2 info。`pmm_compute_region_layout` 最多尝试 4 次不同的放置方案，直到找到一个不和任何已有数据重叠的位置。

8.5 Buddy 分配器：`pmm_buddy.c`

8.5.1 核心思想

Buddy system 将物理内存分成 2^k 大小的块（order $k = 0, 1, \ldots, 10$）。每个 order 维护一个**空闲链表**：

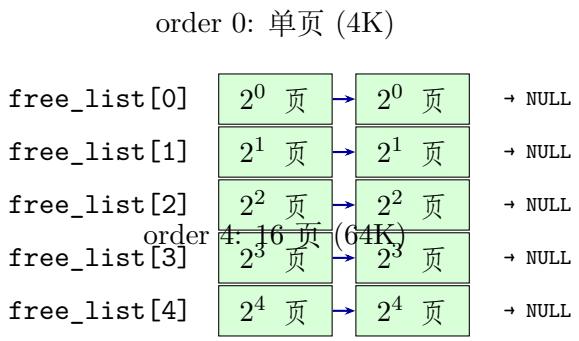

图 8.5: Buddy allocator 的 free_list 数组（每个 order 一个链表）

8.5.2 分配：分裂（Split）

如果请求 order k 但 `free_list[`k`]` 为空，就从更高 order 取一个块并**分裂**：

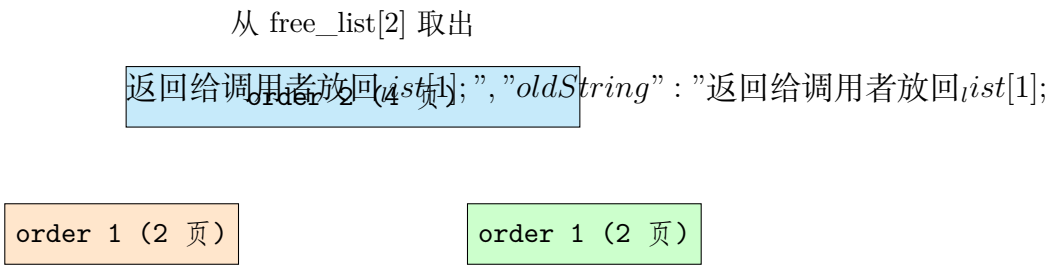

图 8.6: Buddy 分裂：order 2 块拆成两个 order 1 块

8.5.3 释放：合并（Coalesce）

释放一个 order k 块后，检查它的 **buddy** 是否也空闲。如果是，合并为 order $k+1$：

```
// 给定页帧号 P 和 order k, buddy 的帧号:
uint32_t buddy = page_frame ^ (1u << k);
```

思考 8.3

为什么 XOR 能找到 buddy？假设 order 1（2 页），页帧号 4 和 5 是一对，6 和 7 是一对。$4 \oplus 2 = 6$（不是 buddy!）。实际上 buddy 是以**对齐的块**为单位的：order k 块的起始帧号必须是 2^k 的整数倍，所以 $P \oplus 2^k$ 恰好切换到同级的另一半。例如帧 4 的 order-1 buddy $= 4 \oplus 2 = 6$，帧 6 的 buddy $= 6 \oplus 2 = 4$。

8.6 后端切换：kconfig.h

```
// kconfig.h
```

```c
#define KCONFIG_PMM_BACKEND 0 // 0=bitmap, 1=buddy

// kmain.c
#if KCONFIG_PMM_BACKEND == 0
    pmm_register_backend(pmm_bitmap_get_ops());
#elif KCONFIG_PMM_BACKEND == 1
    pmm_register_backend(pmm_buddy_get_ops());
#endif
pmm_init();
```

8.7 验证

```
szy-kernel > mmap
Available RAM: 0x00000000 - 0x0009FC00 (0MB)
Available RAM: 0x00100000 - 0x07FE0000 (126MB)

szy-kernel > free
PMM: free 32395 / 32768 pages
PMM: free 129580KiB, used 1492KiB

szy-kernel > pmm state
PMM: base=0x00100000, page_size=4096
PMM: total=32768, free=32395, used=373
PMM: backend=bitmap

szy-kernel > pmm alloc 3
pmm alloc: 0x07FC1000
pmm alloc: 0x07FC2000
pmm alloc: 0x07FC3000

szy-kernel > pmm freeall
pmm freeall: freed 3 pages

szy-kernel > pmm dump 0 32
################################ idx 0 addr 0x00100000
```

8.8 思考与练习

练习 8.1

切换到 buddy 后端（`KCONFIG_PMM_BACKEND=1`），运行 `pmm state`。对比 bitmap 和 buddy 的 `free` 数量。buddy 为什么可能更少？（提示：buddy 的元数据——`free_list` 节点和 order 标记——也占物理页。）

练习 8.2

运行 `pmm alloc 100` → `pmm state` → `pmm freeall` → `pmm state`。free 是否完全恢复？如果差了几页，用 `pmm dump` 查看哪些页没被释放。

练习 8.3

在 `pmm_bitmap_alloc_page` 中注释掉 round-robin 逻辑（`g_region_rr = ri`），改为总是从 region 0 开始扫描。连续分配 1000 页后用 `pmm dump` 查看各 region 的分布。和 round-robin 对比有什么不同？

练习 8.4

（挑战）bitmap 分配器不支持连续多页分配。实现一个 `pmm_bitmap_alloc_pages(unsigned count)` 函数：在位图中找到 `count` 个连续的 0 bit，全部置 1 并返回起始物理地址。这比 buddy 的 order 分配更灵活，但扫描开销更大。

8.9 本章小结

PMM 的核心数据结构和算法：

1. **`pmm_ops_t`**：9 个函数指针，定义后端接口
2. **`pmm.c`**：薄 dispatch 层，`g_pmm_ops` 指针转发
3. **Bitmap**：`pmm_region` 结构体 × 多区域，每页 1 bit，线性扫描分配
4. **Buddy**：`free_list[MAX_ORDER+1]`，2^k 分裂/合并
5. **初始化**：解析 mmap → 计算位图放置 → 保留内核 + mb2 → selftest

下一章在 PMM 之上构建虚拟内存管理器（VMM）——两级页表和分页机制。

Chapter 9

VMM：虚拟内存管理

9.1 本章目标

- 理解 x86 32 位两级页表的硬件翻译流程
- 掌握 Page Directory Entry (PDE) 和 Page Table Entry (PTE) 的位域格式
- 实现 `vmm_init`：用 4 KiB 页表替换 `boot.asm` 的临时 PSE 映射
- 实现 `vmm_map_page` / `vmm_unmap_page`：基本映射原语
- 理解 TLB 缓存与 `invlpg` 刷新机制
- 实现 High-Half Kernel 地址转换宏 `PHYS_TO_VIRT` / `VIRT_TO_PHYS`
- 实现 `vmm_alloc_pages` / `vmm_free_pages`：批量虚拟页分配

9.2 背景知识：x86 两级页表

当 `CR0` 的 PG 位（bit 31）被置 1 后，CPU 的每一次内存访问都会经过 **MMU**（Memory Management Unit）进行虚拟 → 物理地址翻译。x86 32 位模式使用**两级页表**结构：

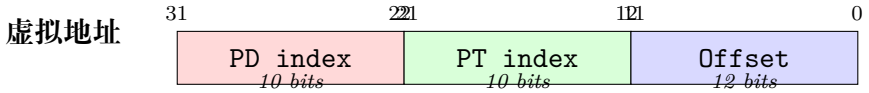

图 9.1: 32 位虚拟地址的三段划分

翻译流程如下：

1. CPU 从 `CR3` 取得 **Page Directory** 的物理基地址。

2. 用虚拟地址的高 10 位（PD index）索引 Page Directory，得到一个 PDE。
3. PDE 的高 20 位指向一个 **Page Table** 的物理地址。
4. 用虚拟地址的中 10 位（PT index）索引 Page Table，得到一个 PTE。
5. PTE 的高 20 位是目标**物理页帧**地址，拼上低 12 位 Offset，即为最终物理地址。

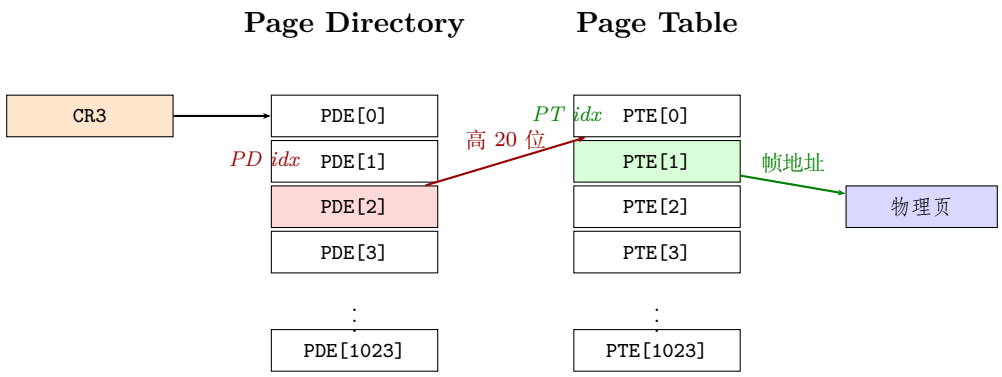

图 9.2: 两级页表翻译流程：CR3 → PDE → PTE → 物理页

为什么？

为什么用两级而不是一级平坦页表?
一级页表需要 2^{20} 个 PTE = 4 MiB 连续内存，即使地址空间大部分为空也要全部分配。两级结构只需 1 个 Page Directory（4 KiB），空闲区域的 PDE 标记为”不存在”，对应的 Page Table 根本不需要分配。大多数进程只用几十个 Page Table，远小于 4 MiB。

9.3 PDE 与 PTE 位域格式

PDE 和 PTE 共享相同的 32 位格式，仅 bit 6–7 的含义略有区别：

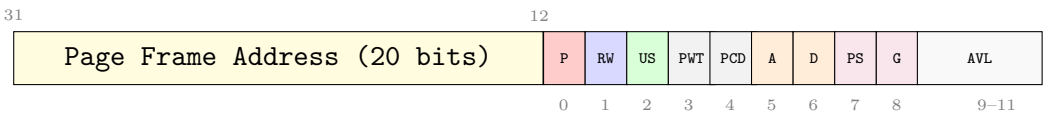

图 9.3: PDE/PTE 32 位格式

Bit	名称	说明
0	P (Present)	1 = 条目有效；0 = 访问触发 #PF (Page Fault)
1	R/W	1 = 可读写；0 = 只读

2	U/S	1 = Ring 3 可访问；0 = 仅 Ring 0
3	PWT	Write-Through 缓存策略（通常置 0）
4	PCD	Cache Disable（通常置 0）
5	A (Accessed)	CPU 访问此页后自动置 1
6	D (Dirty)	CPU 写入此页后自动置 1（PTE 有意义）
7	PS/PAT	PDE: PS=1 启用 4 MiB 大页；PTE: PAT 位
8	G (Global)	1 = TLB 不随 CR3 切换刷新（需 CR4.PGE）
9–11	AVL	软件可用（OS 自定义用途）
12–31	Frame	物理页帧地址的高 20 位（低 12 位隐含为 0，即 4 KiB 对齐）

我们的内核目前只用到三个 flag 宏：

```
#define PTE_PRESENT  0x001u /* bit 0: 条目有效 */
#define PTE_WRITABLE 0x002u /* bit 1: 可读写 */
#define PTE_USER     0x004u /* bit 2: 用户态可访问 */

#define PDE_PRESENT  0x001u
#define PDE_WRITABLE 0x002u
#define PDE_USER     0x004u
```

设计决策

PDE flags 遵循”宽松上限”原则：PDE 给足权限（Present + Writable），精细控制放在 PTE 层。例如 PDE 设 Writable 但 PTE 设 Read-Only，最终该页只读；反之 PDE 设 Read-Only，即使 PTE 设 Writable，该页也只读。所以我们在 PDE 层统一设 `PDE_PRESENT | PDE_WRITABLE`，具体的只读/用户态权限在 PTE 中控制。

9.4 数据结构

9.4.1 page_directory_t 与 page_table_t

```
typedef struct {
    uint32_t entries[1024]; /* 1024 个 PDE，每个 4 字节 */
} __attribute__((aligned(4096))) page_directory_t;

typedef struct {
    uint32_t entries[1024]; /* 1024 个 PTE，每个 4 字节 */
```

```
} __attribute__((aligned(4096))) page_table_t;
```

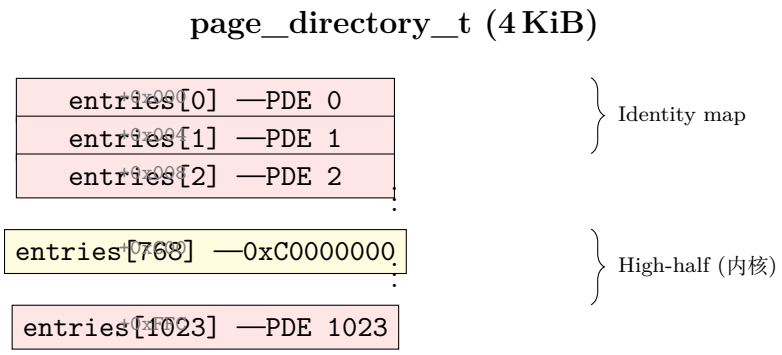

图 9.4: page_directory_t 内存布局——PDE[768] 对应 0xC0000000

为什么？

为什么 page_directory_t 用 __attribute__((aligned(4096)))？
CR3 要求页目录物理地址必须 4 KiB 对齐（低 12 位为 0）。用 aligned 属性声明的静态变量由链接器保证对齐。静态变量放在 BSS 段，启动时自动清零——所有 PDE 初始 Present=0，安全。

思考 9.1

每个 PDE 覆盖 $1024 \times 4\,\text{KiB} = 4\,\text{MiB}$ 的虚拟空间。PDE[768] 对应虚拟地址 $768 \times 4\,\text{MiB} =$ 0xC0000000 ——正好是 High-Half Kernel 的起始地址。这是巧合吗？（答：不是。0xC0000000 被选为内核基地址，正因为它恰好对齐到 PDE 边界，简化了地址空间划分。）

9.5 地址提取辅助函数

虚拟地址需要拆成 PD index 和 PT index：

```
static inline uint32_t pd_index(uint32_t virt) {
    return (virt >> 22) & 0x3FFu; // 高 10 位
}

static inline uint32_t pt_index(uint32_t virt) {
    return (virt >> 12) & 0x3FFu; // 中 10 位
}
```

例如，虚拟地址 0xC0100000：

- pd_index = 0xC0100000 » 22 = 768 = PDE[768]

- pt_index = (0xC0100000 » 12) & 0x3FF = 256 = PTE[256]
- offset = 0xC0100000 & 0xFFF = 0

9.6 High-Half 地址转换宏

```
#define KERNEL_VIRT_OFFSET 0xC0000000u

#define PHYS_TO_VIRT(paddr) ((void*)((uint32_t)(paddr) + KERNEL_VIRT_OFFSET))
#define VIRT_TO_PHYS(vaddr) ((uint32_t)(vaddr) - KERNEL_VIRT_OFFSET)
```

注意

这两个宏**仅适用于内核直接映射区域**——即 vmm_init 中建立的 virt = phys + 0xC0000000 映射。对于动态分配的虚拟地址（如 vmm_alloc_pages 返回的地址），不能使用这两个宏，必须查页表才能得到物理地址。

9.7 核心函数：vmm_ensure_page_table

在写入 PTE 之前，必须确保对应的 Page Table 存在。这是”按需创建”的关键：

```
static page_table_t* vmm_ensure_page_table(uint32_t pd_idx) {
    uint32_t pde = g_kernel_pd.entries[pd_idx];

    if (pde & PDE_PRESENT) {
        // PDE 已有效，取出页表物理地址
        uint32_t pt_phys = pde & ~0xFFFu;
        return (page_table_t*)PHYS_TO_VIRT(pt_phys);
    }

    // PDE 不存在 -> 分配新页表
    uint32_t pt_phys = (uint32_t)(uintptr_t)pmm_alloc_page();
    if (!pt_phys) return NULL;

    // 通过虚拟地址清零（所有 PTE 的 Present=0）
    page_table_t* pt_virt = (page_table_t*)PHYS_TO_VIRT(pt_phys);
    memzero_page(pt_virt);

    // 填入 PDE: 物理地址 | flags
    g_kernel_pd.entries[pd_idx] = pt_phys | PDE_PRESENT | PDE_WRITABLE;
```

```

    return pt_virt;
}
```

步骤详解：

1. 读取 `g_kernel_pd.entries[pd_idx]`，检查 Present 位。
2. **已存在**：用 `& ~0xFFF` 取出高 20 位物理地址，`PHYS_TO_VIRT` 转为虚拟地址后返回。
3. **不存在**：调用 `pmm_alloc_page()` 获取一页物理内存。
4. 用 `memzero_page()` 清零——确保所有 1024 个 PTE 的 Present=0。
5. 将物理地址与 flags 组合后写入 PDE。

注意

清零新页表是**绝对必要**的。未清零的页表中可能残留垃圾数据，CPU 可能把任意非零值当作 Present=1 的 PTE，导致虚拟地址被映射到随机物理页——轻则数据损坏，重则 Triple Fault。

9.8 核心函数：vmm_map_page

```
int vmm_map_page(uint32_t virt, uint32_t phys, uint32_t flags) {
    // 对齐检查
    if ((virt & 0xFFFu) || (phys & 0xFFFu)) {
        printk("[vmm] map: unaligned virt=0x%08x phys=0x%08x\n",
               virt, phys);
        return -1;
    }

    uint32_t pdi = pd_index(virt);
    uint32_t pti = pt_index(virt);

    page_table_t* pt = vmm_ensure_page_table(pdi);
    if (!pt) return -1; // PMM OOM

    // 写入 PTE: 高 20 位 = 物理帧地址，低 12 位 = flags
    pt->entries[pti] = (phys & ~0xFFFu) | (flags & 0xFFFu);

```

```
    // 分页已开启则刷新 TLB
    if (g_vmm_ready) {
        vmm_invlpg(virt);
    }

    return 0;
}
```

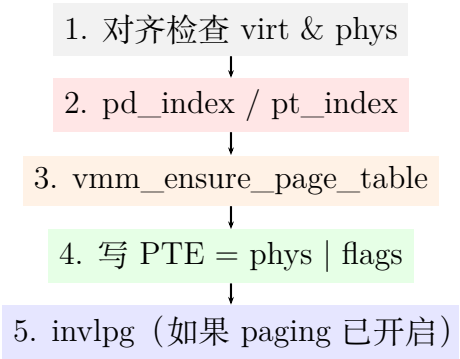

图 9.5: `vmm_map_page` 执行流程

CPU 状态变化

调用 `vmm_map_page(0xC0200000, 0x200000, PTE_PRESENT | PTE_WRITABLE)` 后：

- PDE[768] 的 Present 位被置 1（如果页表刚创建）
- PDE[768] 指向的 Page Table 的 PTE[512] = `0x200003`（`0x200000` | 0x3）
- 如果 TLB 中缓存了 `0xC0200000` 的旧条目，`invlpg` 使其失效
- 此后访问虚拟地址 `0xC0200000` 将读写物理地址 `0x200000`

9.9 TLB 与 invlpg

TLB（Translation Lookaside Buffer）是 CPU 内部的**页表翻译缓存**。每次虚拟地址翻译都需要两次内存访问（读 PDE + 读 PTE），TLB 将最近的翻译结果缓存起来，命中时可以跳过页表查找。

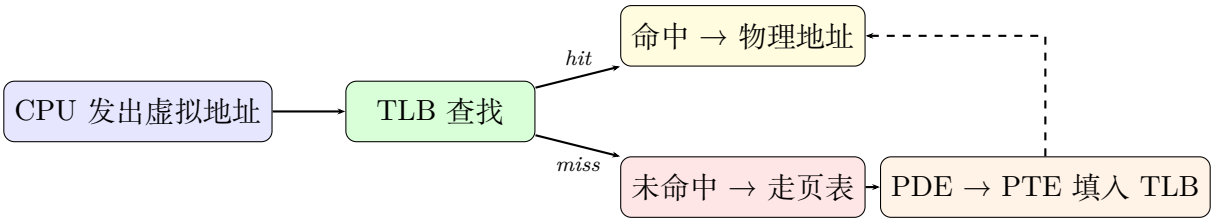

图 9.6: TLB 命中与未命中流程

当我们修改了一个 PTE（改映射或取消映射），TLB 中可能还缓存着**过时**的翻译。必须用 `invlpg` 指令使对应条目失效：

```
; void vmm_invlpg(uint32_t virt_addr);
vmm_invlpg:
    mov eax, [esp + 4]    ; eax = virt_addr
    invlpg [eax]          ; 使该虚拟地址的 TLB 条目失效
    ret
```

为什么？

为什么不每次都重新加载 `CR3`（刷新**全部** TLB）？
重新加载 CR3 会使**所有**非 Global 的 TLB 条目失效，代价远大于 `invlpg` 只失效一条。在高频映射操作中（如缺页处理），`invlpg` 的性能优势很明显。但在大批量拆除映射时（如 `vmm_unmap_identity`），重新加载 CR3 反而更简单高效。

思考 9.2

`vmm_unmap_identity()` 清除了数百个 PDE 后调用 `vmm_load_page_directory()` 重新加载 CR3。如果改成对每个被清除的 PDE 覆盖的虚拟地址逐页 `invlpg`，需要多少次 `invlpg` 调用？(提示：每个 PDE 覆盖 1024 页。假设清除了 16 个 PDE...)

9.10 汇编辅助：paging_flush.asm

```
; void vmm_load_page_directory(uint32_t pd_phys_addr);
;   [CPU STATE] CR3 <- pd_phys_addr
;   副作用：刷新整个 TLB（除 Global 页）
vmm_load_page_directory:
    mov eax, [esp + 4]    ; eax = pd_phys_addr
    mov cr3, eax          ; CR3 <- 页目录物理地址
    ret

; void vmm_enable_paging(void);
```

```
;   [CPU STATE] CR0.PG <- 1, 之后所有内存访问经过 MMU
vmm_enable_paging:
    mov eax, cr0            ; 读出当前 CR0
    or  eax, 0x80000000     ; 设置 bit 31 (PG)
    mov cr0, eax            ; 写回 -> 分页立即生效
    ret
```

CPU 状态变化

`vmm_load_page_directory(pd_phys)` 执行后：

- `CR3` = `pd_phys`（页目录物理地址）
- 整个 TLB 被刷新（非 Global 条目全部失效）
- 下一条指令的取指已经使用新页目录翻译

`vmm_enable_paging()` 执行后：

- `CR0` 的 bit 31 (PG) 从 0 变为 1
- 从 `mov cr0, eax` 的下一条指令 (`ret`) 起，每次内存访问都经过 MMU 翻译
- 如果当前 EIP 所在物理地址没有被 identity map → **Triple Fault**

9.11 vmm_init：初始化全流程

`vmm_init()` 负责用正式的 4 KiB 页表替换 `boot.asm` 的临时 4 MiB PSE 映射。

9.11.1 进入时的 CPU 状态

CPU 状态变化

- `CR0`.PG = 1（`boot.asm` 已开启分页，使用 4 MiB PSE 大页）
- `CR3` 指向 `boot_pd`（临时页目录）
- EIP 在虚拟地址 `0xC01xxxxx`（high-half）
- PMM 已初始化完毕，可以调用 `pmm_alloc_page()`

9.11.2 核心步骤

1. **清零页目录**：`memzero_page(&g_kernel_pd)`
   所有 1024 个 PDE 初始化为 0（Present=0）。

2. **计算映射范围**：取内核物理结束地址、PMM 管理范围上限，至少 16 MiB，上限 1 GiB。

3. **建立双重映射**：对 $[0, \text{map_end})$ 的每个 4 KiB 页：

   - Identity mapping: `vmm_map_page(addr, addr, ...)`
   - High-half mapping: `vmm_map_page(addr + 0xC0000000, addr, ...)`

4. **切换 CR3**：`vmm_load_page_directory(VIRT_TO_PHYS(&g_kernel_pd))`
   从 boot 临时页目录切换到 `g_kernel_pd`。

5. **标记就绪**：`g_vmm_ready = 1`

6. **初始化 bump allocator**：设置 `g_next_vaddr` 为直接映射区上方（对齐到 4 MiB）。

```c
void vmm_init(void) {
    printk("[vmm] init: replacing boot PSE with 4KiB page tables...\n");
    memzero_page(&g_kernel_pd);

    // 计算映射范围
    uint32_t kernel_end = (uint32_t)(uintptr_t)__kernel_phys_end;
    uint32_t map_end = kernel_end;
    uint32_t min_map = 16u * 1024u * 1024u;
    if (map_end < min_map) map_end = min_map;
    uint32_t pmm_end = pmm_managed_base()
                    + pmm_total_pages() * VMM_PAGE_SIZE;
    if (pmm_end > map_end) map_end = pmm_end;
    if (map_end > 0x40000000u) map_end = 0x40000000u;

    // 逐页建立双重映射
    for (uint32_t addr = 0; addr < map_end; addr += VMM_PAGE_SIZE) {
        vmm_map_page(addr, addr, PTE_PRESENT | PTE_WRITABLE);
        vmm_map_page(addr + KERNEL_VIRT_OFFSET, addr,
                    PTE_PRESENT | PTE_WRITABLE);
    }

    // 切换到新页目录
```

```
    uint32_t pd_phys = VIRT_TO_PHYS(
        (uint32_t)(uintptr_t)&g_kernel_pd);
    vmm_load_page_directory(pd_phys);
    g_vmm_ready = 1;

    // 初始化 bump allocator 水位线
    uint32_t direct_map_top = KERNEL_VIRT_OFFSET + map_end;
    uint32_t align_4m = 0x400000u;
    g_next_vaddr = (direct_map_top + align_4m - 1u)
                & ~(align_4m - 1u);
}
```

设计决策

为什么保留 identity mapping？

`vmm_init` 运行时，内核已经在 high-half（`0xC0000000+`），按说不需要低地址映射。但多处仍依赖 identity mapping：

- VGA text buffer 在物理 `0xB8000`，identity map 下可直接写 `(char*)0xB8000`
- Multiboot2 info 指针是物理地址
- PMM 内部 bitmap 的物理地址直接当指针用

将来 `vmm_unmap_identity()` 会在所有指针转换完成后拆除低地址映射，把 `[0, 0xBFFFFFFF]` 留给用户态进程。

9.12 vmm_unmap_page 与 vmm_unmap_identity

9.12.1 解除单页映射

```
void vmm_unmap_page(uint32_t virt) {
    if ((virt & 0xFFFu)) return;      // 对齐检查

    uint32_t pdi = pd_index(virt);
    uint32_t pde = g_kernel_pd.entries[pdi];
    if (!(pde & PDE_PRESENT)) return; // 页表不存在

    uint32_t pt_phys = pde & ~0xFFFu;
    page_table_t* pt = (page_table_t*)PHYS_TO_VIRT(pt_phys);
    pt->entries[pt_index(virt)] = 0; // PTE 置 0
```

```

    if (g_vmm_ready) vmm_invlpg(virt); // 刷新 TLB
}
```

注意

vmm_unmap_page 只断开虚拟 → 物理的映射，**不释放物理页**。物理页可能被多处引用（如共享内存），释放是调用者的责任。典型模式：先 vmm_get_physical() 拿到物理地址，再 vmm_unmap_page()，最后 pmm_free_page() 归还物理页。

9.12.2 拆除 Identity Mapping

```
void vmm_unmap_identity(void) {
    if (!g_vmm_ready) return;

    uint32_t first_kernel_pde = pd_index(KERNEL_VIRT_OFFSET); // 768
    uint32_t cleared = 0;

    for (uint32_t i = 0; i < first_kernel_pde; i++) {
        if (g_kernel_pd.entries[i] & PDE_PRESENT) {
            g_kernel_pd.entries[i] = 0;
            cleared++;
        }
    }

    // 重新加载 CR3 做全局 TLB 刷新
    uint32_t pd_phys = VIRT_TO_PHYS(
        (uint32_t)(uintptr_t)&g_kernel_pd);
    vmm_load_page_directory(pd_phys);
}
```

CPU 状态变化

vmm_unmap_identity() 执行后：

- PDE[0]–PDE[767] 全部清零
- 虚拟地址 0x00000000–0xBFFFFFFF 访问将触发 #PF
- CR3 被重新加载，TLB 全局刷新

9.13 查询辅助函数

VMM 提供几个查询函数供 Shell 命令和上层代码使用：

```
// 查询虚拟地址是否已映射（返回 1=已映射，0=未映射，-1=错误）
int vmm_is_mapped(uint32_t virt);

// 查询虚拟地址对应的物理地址
int vmm_get_physical(uint32_t virt, uint32_t* phys);

// 直接读取 PDE / PTE 原始值（供调试命令使用）
uint32_t vmm_get_pde(uint32_t pd_idx);
uint32_t vmm_get_pte(uint32_t virt);
```

9.14 Shell 验证

编译并启动 QEMU 后，可以用 vmm 命令验证页表状态：

```
$ make iso && make run
...
[vmm] init: replacing boot PSE with 4KiB page tables...
[vmm] mapping phys 0x00000000 - 0x10000000
[vmm] page directory: virt=0xc01xx000 phys=0x001xx000
[vmm] loading new CR3...
[vmm] 4KiB paging active! mapped 262144 KiB (131072 pages, 128 page tables)
[vmm] identity: 0x00000000 - 0x10000000
[vmm] high-half: 0xc0000000 - 0xd0000000
[vmm] alloc area: 0xd0000000 - 0xFFFFFFFF

> vmm
  PD index Phys PT addr Flags
  0        0x001xx000  P W
  1        0x001xx000  P W
  ...
  768      0x002xx000  P W
  ...

> vmm 0xC0000000
  virt=0xC0000000 -> phys=0x00000000 flags=P W
```

启动日志确认：

- 物理 `0x00000000`–`0x10000000`（256 MiB）被双重映射
- 分配了 128 个 Page Table（identity 64 + high-half 64）
- bump allocator 从 `0xD0000000` 开始

9.15 思考与练习

练习 9.1

手动页表查找：给定虚拟地址 `0xC0B8000`，手工计算 PD index、PT index 和 offset。如果该地址通过 `vmm_init` 被映射到 `phys = virt − 0xC0000000`，写出该 PDE 和 PTE 的完整 32 位值（假设 flags = `PTE_PRESENT | PTE_WRITABLE`）。

练习 9.2

页表内存开销分析：假设 `vmm_init` 需要映射 256 MiB 物理内存（identity + high-half 双重映射），需要多少个 Page Table？这些页表本身占多少物理内存？与 4 MiB 的单级平坦页表相比，节省了多少？（提示：256 MiB / 4 MiB-per-PDE = 64 个 Page Table × 2 = 128 × 4 KiB = 512 KiB）

练习 9.3

拆除 identity mapping 后的陷阱：在 `vmm_unmap_identity()` 执行后，如果内核代码中仍有一处 `*(char*)0xB8000 = 'X';` 尝试直接写 VGA buffer，会发生什么？如何修改这行代码让它在 high-half 环境下正确工作？

练习 9.4

添加只读映射支持：修改 `vmm_map_page`，当 flags 中不包含 `PTE_WRITABLE` 时，映射的页面应该是只读的。编写一个测试：映射一个只读页，尝试写入，观察是否触发 Page Fault。（提示：当前 `CR0`.WP=0 时 Ring 0 不受 Read-Only 限制，需要在 `vmm_enable_paging` 中同时设置 WP 位。）

Chapter 10

High-Half Kernel：高半区内核

10.1 本章目标

- 理解 High-Half Kernel 的地址空间布局：用户态 0–3 GiB / 内核态 3–4 GiB
- 掌握直接映射（Direct Mapping）的原理与公式 virt = phys + `0xC0000000`
- 理解 `boot.asm` 中 PSE 临时映射为何需要同时建立 identity 与 high-half
- 掌握 `PHYS_TO_VIRT` / `VIRT_TO_PHYS` 宏的使用场景与限制
- 实现 `vmm_unmap_identity()`：拆除 identity mapping，释放低地址空间
- 理解指针转换的必要性：`g_mb2_info`、PMM bitmap 等指针的迁移
- 掌握拆除后的 CPU 状态变化与 TLB 刷新

10.2 背景知识：地址空间布局

在 32 位 x86 系统中，虚拟地址空间共 4 GiB（`0x00000000`–`0xFFFFFFFF`）。High-Half Kernel 的经典划分方式是：

- **低 3 GiB**（`0x00000000`–`0xBFFFFFFF`）：用户态进程
- **高 1 GiB**（`0xC0000000`–`0xFFFFFFFF`）：内核

图 10.1: High-Half Kernel 地址空间布局（32 位 x86）

为什么？

为什么内核占高 1 GiB 而不是低 1 GiB?
将内核放在高地址有几个好处：

1. 用户进程总是从 `0x00000000` 附近加载（ELF 默认基址 `0x08048000`），不用跳过内核区域
2. 内核切换进程时只需替换低 3 GiB 的页表（PD[0..767]），高 1 GiB（PD[768..1023]）在所有进程中共享，切换代价小
3. Linux 2.x/3.x、Windows NT 均采用此方案，是业界惯例

10.3 直接映射区（Direct Mapping）

内核启动后，需要频繁访问物理内存（页表、PMM bitmap、设备寄存器等）。**直接映射**将一段连续的物理内存”平移”到内核虚拟地址空间中：

$$\text{virt} = \text{phys} + \texttt{0xC0000000} \tag{10.1}$$

例如：

- 物理 `0x00000000` → 虚拟 `0xC0000000`
- 物理 `0x000B8000`（VGA text buffer）→ 虚拟 `0xC00B8000`
- 物理 `0x00100000`（内核加载地址）→ 虚拟 `0xC0100000`

直接映射区的大小取决于物理内存的总量（但不超过 1 GiB，否则会占满整个内核地址空间）。我们的 `vmm_init()` 在建立映射时会计算这个范围：

Listing 10.1: 直接映射范围计算（vmm.c 节选）

```
uint32_t map_end = kernel_end;

/* 至少映射 16 MiB（覆盖 VGA、BIOS 区域等） */
if (map_end < 16u * 1024u * 1024u)
    map_end = 16u * 1024u * 1024u;

/* 覆盖 PMM 管理的全部物理页 */
uint32_t pmm_end = pmm_managed_base()
                + pmm_total_pages() * VMM_PAGE_SIZE;
if (pmm_end > map_end)
    map_end = pmm_end;

/* 不超过 1 GiB */
if (map_end > 0x40000000u)
    map_end = 0x40000000u;
```

10.4 PHYS_TO_VIRT 与 VIRT_TO_PHYS 宏

在 vmm.h 中定义了两个核心转换宏：

Listing 10.2: 地址转换宏（vmm.h）

```
#define KERNEL_VIRT_OFFSET 0xC0000000u

#define PHYS_TO_VIRT(paddr) \
    ((void*)((uint32_t)(paddr) + KERNEL_VIRT_OFFSET))

#define VIRT_TO_PHYS(vaddr) \
    ((uint32_t)(vaddr) - KERNEL_VIRT_OFFSET)
```

作用域限制

这两个宏**仅适用于内核直接映射区域**。
对于通过 vmm_alloc_pages() 动态映射的虚拟地址（如内核堆），物理页可能是不连续的，virt − 0xC0000000 ≠ phys。这些地址需要通过查询页表（vmm_get_physical()）来获取物理地址。
简单规则：

- PMM 返回的物理地址 → 用 PHYS_TO_VIRT 转换后解引用

- PDE/PTE 中存储的始终是物理地址（给 CPU/MMU 用）
- 内核堆指针（`kmalloc` 返回值）→ 不能用 `VIRT_TO_PHYS`

10.5 boot.asm 中的 PSE 临时映射

在第 ?? 章中我们已经知道，`boot.asm` 使用 4 MiB PSE 大页建立临时映射。这里我们详细分析为什么需要**同时**建立 identity mapping 和 high-half mapping。

Listing 10.3: boot.asm：构建双重 PSE 映射

```
; PSE PDE: [31:22]=phys_base | PS(bit7) | RW(bit1) | P(bit0)
;   flags = 0x83
xor ecx, ecx            ; ecx = 0..255
.fill_pse:
    mov eax, ecx
    shl eax, 22          ; eax = ecx * 4 MiB
    or  eax, 0x83        ; PS | RW | Present
    mov [boot_pd + ecx*4],     eax  ; identity: PD[i]
    mov [boot_pd + 768*4 + ecx*4], eax ; high-half: PD[768+i]
    inc ecx
    cmp ecx, 256
    jb  .fill_pse
```

这段代码会产生如下映射：

PDE 范围	虚拟地址	映射到物理地址
PD[0..255]	0x00000000–0x3FFFFFFF	0x00000000–0x3FFFFFFF
PD[768..1023]	0xC0000000–0xFFFFFFFF	0x00000000–0x3FFFFFFF

为什么？

为什么不能只建 high-half mapping，跳过 identity mapping?
因为在开启分页的那一瞬间，`EIP` 仍然指向物理地址 ~`0x001xxxxx`（`boot.asm` 的 `.boot` 段链接在低地址）。如果没有 identity mapping，`CR0.PG` 置 1 后 CPU 取下一条指令时会 Page Fault → Double Fault → Triple Fault → 重启。

时序回顾：

1. `mov cr0, eax` —开启分页，此时 EIP ≈ `0x001xxxxx`
2. CPU 取下一条指令 → 虚拟地址 `0x001xxxxx` → identity mapping 翻译 → 物理 `0x001xxxxx`

3. `jmp _start_high` —跳转到虚拟地址 `0xC01xxxxx`
4. CPU 取指 → `0xC01xxxxx` → high-half mapping 翻译 → 物理 `0x001xxxxx`
5. 从此以后，代码运行在 high-half，identity mapping 变成了”遗产”

开启分页的关键指令序列：

Listing 10.4: boot.asm：开启分页并跳转到 high-half

```
; 加载页目录物理地址到 CR3
mov eax, boot_pd
mov cr3, eax

; 开启分页 (CR0.PG = 1)
; [CPU STATE] 从下一条指令开始，所有内存访问经过 MMU
mov eax, cr0
or  eax, 0x80000000   ; bit 31 = PG
mov cr0, eax

; --- 分页已开启，identity mapping 保证 EIP 仍然有效 ---

; 跳转到 high-half 虚拟地址
mov eax, _start_high ; _start_high 链接在 0xC01xxxxx
jmp eax
```

CPU 状态变化

执行 `mov cr0, eax`（PG=1）后：

- CR3 → `boot_pd`（物理地址）
- EIP 仍在低地址（~`0x001xxxxx`），通过 PD[0] 的 identity mapping 继续运行
- `jmp _start_high` 后，EIP 切换到 `0xC01xxxxx`，通过 PD[768] 的 high-half mapping 运行
- 此后内核栈、全局变量等均位于 `0xC0xxxxxx`+ 虚拟地址

10.6 vmm_init 后的双重映射

经过第 ?? 章的 `vmm_init()`，临时 PSE 映射被 4 KiB 细粒度页表替换，但 identity mapping 仍然保留。此时每个物理页 P 有两个虚拟地址：

映射类型	虚拟地址
Identity	P（虚拟 = 物理）
High-half	$P + \texttt{0xC0000000}$

这意味着同一块物理内存可以通过两个不同的虚拟地址访问——这是一种浪费，也为将来的用户态进程留下了安全隐患。下一步就是拆除 identity mapping。

10.7 指针转换：拆除前的准备

注意

在调用 `vmm_unmap_identity()` 之前，必须将所有仍指向低地址的指针转换为 high-half 地址。否则拆除 identity mapping 后，解引用这些指针会触发 Page Fault。

在我们的内核中，需要转换的关键指针包括：

10.7.1 g_mb2_info：Multiboot2 信息结构

GRUB 传入的 Multiboot2 info 指针是物理地址（由 `boot.asm` 从 `EBX` 保存）。在 `vmm_init()` 建立 high-half mapping 之后，需要将其转换：

Listing 10.5: kmain.c：转换 Multiboot2 指针

```
/* vmm_init 之后、vmm_unmap_identity 之前 */
g_mb2_info = (const void*)PHYS_TO_VIRT(
    (uint32_t)(uintptr_t)g_mb2_info);
```

如果忘记转换会怎样?

- `g_mb2_info` 仍指向 ~`0x00010000`（物理地址）
- 此时 identity mapping 还在，暂时不会出错
- 但 `vmm_unmap_identity()` 之后，PD[0] 被清除
- 下次 shell 命令访问 `g_mb2_info` → `0x00010000` → Page Fault!

10.7.2 PMM bitmap 指针

PMM bitmap 后端在 `pmm_init()` 时将 bitmap 放置在物理内存中。由于 bitmap 后端内部存储的是虚拟地址指针（通过 identity mapping 期间可以等价于物理地址），在 `vmm_init()` 建立 high-half mapping 后，PMM 内部的 bitmap 指针需要通过直接映射区的 high-half 地址来访问。

思考 10.1

如果 PMM bitmap 存储在物理地址 `0x00200000`，在 identity mapping 拆除后，内核应该通过哪个虚拟地址访问它？

答：`0xC0200000`（= `0x00200000` + `0xC0000000`）

10.8 vmm_unmap_identity 实现

一切指针转换完毕后，调用 `vmm_unmap_identity()` 拆除低地址映射：

Listing 10.6: vmm_unmap_identity() 完整实现

```
void vmm_unmap_identity(void) {
    /*
     * 拆除 identity mapping (虚拟地址 0x00000000-0xBFFFFFFF)
     *
     * [WHY] identity mapping 只是 boot→high-half 过渡期的临时需要。
     *   现在内核完全运行在 high-half (0xC0000000+),
     *   所有指针已通过 PHYS_TO_VIRT 转换，可以安全拆除。
     *   拆除后，低地址空间留给将来的用户态进程。
     */

    if (!g_vmm_ready)
        return;

    uint32_t first_kernel_pde = pd_index(KERNEL_VIRT_OFFSET);
    /* first_kernel_pde = 768 (0xC0000000 >> 22) */
    uint32_t cleared = 0;

    /* 步骤 1: 清零 PD[0..767] 中所有有效条目 */
    for (uint32_t i = 0; i < first_kernel_pde; i++) {
        if (g_kernel_pd.entries[i] & PDE_PRESENT) {
            g_kernel_pd.entries[i] = 0;
            cleared++;
```

```
        }
    }

    /* 步骤 2: 重新加载 CR3 做全局 TLB 刷新 */
    uint32_t pd_phys = VIRT_TO_PHYS(
        (uint32_t)(uintptr_t)&g_kernel_pd);
    vmm_load_page_directory(pd_phys);

    printk("[vmm] identity mapping removed "
           "(%u PDEs cleared)\n", (unsigned)cleared);
}
```

逐步分析：

1. **计算边界**：`KERNEL_VIRT_OFFSET` = `0xC0000000`，右移 22 位得到 PD index = 768。所有低于 768 的 PDE 属于用户态地址空间。

2. **清零 PDE**：遍历 PD[0..767]，将所有 Present 的条目置 0。置 0 后 CPU 查页表会发现 Present=0，触发 #PF。

3. **TLB 刷新**：向 `CR3` 重新写入页目录物理地址。x86 规范：写 CR3 会使整个 TLB 无效化（除 Global 页外）。这比逐页调用 `invlpg` 更高效，因为我们一次性清除了几百个 PDE。

CPU 状态变化

执行 `vmm_unmap_identity()` 后：

- PD[0..767] 全部为 0（Present=0）
- PD[768..1023] 不变（high-half 映射完好）
- TLB 已刷新，旧的 identity mapping 缓存被清除
- 访问 `0x00000000`–`0xBFFFFFFF` 中任何地址 → #PF
- 内核代码继续通过 `0xC0000000`+ 正常运行

注意

必须在调用此函数**之前**完成所有指针转换！
内核的初始化顺序严格为：

1. `vmm_init()` —建立 identity + high-half 双重映射

2. 转换所有低地址指针 → high-half 地址（如 `g_mb2_info`）
3. `vmm_unmap_identity()` —拆除 identity mapping

如果顺序颠倒（先拆除再转换），转换代码本身不会崩溃（因为 `PHYS_TO_VIRT` 只是做加法运算），但在拆除之后访问未转换的旧指针就会 Page Fault。调试这类问题非常困难，因为症状可能延迟很久才出现。

10.9 VGA text buffer 的特殊处理

思考 10.2

VGA text buffer 位于物理地址 `0xB8000`。拆除 identity mapping 后，通过 `0xB8000` 访问 VGA 会触发 Page Fault。内核应该如何继续输出到屏幕？

方案分析：

- 方案 A：通过 high-half 直接映射访问 `0xC00B8000`（= `0xB8000`+`0xC0000000`）
- 方案 B：放弃 VGA，只使用串口（serial port）输出

我们的内核选择了方案 B：`printk` 完全通过串口（I/O 端口 `0x3F8`）输出。串口使用 `in/out` 指令访问 I/O 端口空间，不经过内存映射，因此不受 identity mapping 拆除的影响。

如果将来需要 VGA 输出，只需将 VGA 基地址改为 `0xC00B8000` 即可。这正是直接映射区的方便之处——任何物理设备地址加上 `0xC0000000` 就能访问。

10.10 kmain 中的初始化顺序

我们来看 `kmain.c` 中与本章相关的关键代码片段：

Listing 10.7: kmain.c：high-half 切换的完整流程

```
/* Stage 2: 物理内存管理器 */
pmm_init();

/* Stage 3: 虚拟内存管理器（identity + high-half 双重映射）*/
vmm_init();

/*
 * [CRITICAL] 转换阶段
 * PMM init 已结束，将 g_mb2_info 从物理地址转换为虚拟地址。
```

```
 * 之后 shell 命令通过 high-half 地址访问 Multiboot2 info。
 */
g_mb2_info = (const void*)PHYS_TO_VIRT(
    (uint32_t)(uintptr_t)g_mb2_info);

/* 拆除 identity mapping，低地址空间留给将来的用户态 */
vmm_unmap_identity();

/* Stage 4: 内核堆初始化 (kmalloc) */
kmalloc_init();
```

注意这段代码的三个关键步骤形成了一个”安全窗口”：

1. `vmm_init()` 建立双重映射—此后 high-half 和 identity 都可用
2. 指针转换—所有低地址指针切换到 high-half 地址
3. `vmm_unmap_identity()` —关闭窗口，低地址不再可用

10.11 验证：查看映射状态

拆除 identity mapping 后，可以用 shell 命令验证效果。通过 `vmm_is_mapped()` 查询特定虚拟地址的映射状态：

Listing 10.8: 验证 identity mapping 已拆除

```
> vmm lookup 0x00000000
  virt=0x00000000 -> NOT MAPPED

> vmm lookup 0x000B8000
  virt=0x000B8000 -> NOT MAPPED

> vmm lookup 0xC0000000
  virt=0xC0000000 -> phys=0x00000000 [P RW]

> vmm lookup 0xC00B8000
  virt=0xC00B8000 -> phys=0x000B8000 [P RW]

> vmm lookup 0xC0100000
  virt=0xC0100000 -> phys=0x00100000 [P RW]
```

可以看到：

- 低地址（0x00000000、0x000B8000）全部显示 NOT MAPPED
- 对应的 high-half 地址（加 0xC0000000）映射正常
- 物理地址 = 虚拟地址 − 0xC0000000，符合直接映射公式

10.12 完整地址空间示意图

拆除 identity mapping 后，内核的地址空间最终布局如下：

图 10.2: 拆除 identity mapping 后的最终地址空间布局

10.13 思考与练习

练习 10.1

手动计算 PDE index

给定虚拟地址 0xC0100000（内核入口点），请手工计算：

1. PD index（高 10 位）的值是多少？
2. 它落在 PD[768..1023] 中的哪个位置？
3. 如果物理地址是 0x00100000，该 PTE 的内容（含 flags Present+RW）应该是什么？

提示：0xC0100000 ≫ 22 = 0x300 = 768。PT index = (0xC0100000 ≫ 12) & 0x3FF = 0x100 = 256。PTE = 0x00100000 | 0x03 = 0x00100003。

练习 10.2

模拟遗忘转换的后果

假设开发者在 `kmain.c` 中忘记了这一行：

```
g_mb2_info = (const void*)PHYS_TO_VIRT(
    (uint32_t)(uintptr_t)g_mb2_info);
```

请分析：

1. 在 `vmm_unmap_identity()` 调用之前，访问 `g_mb2_info` 会崩溃吗？为什么？
2. 调用之后呢？具体会触发什么异常？CPU 的错误码（error code）会包含哪些信息？
3. 如果使用 GDB 调试，应该在哪里设置断点来定位此问题？

练习 10.3

VGA text mode 输出的 high-half 适配

当前内核的 `printk` 通过串口输出，不受 identity mapping 拆除影响。请设计并实现一个简单的 VGA text mode 驱动，要求：

1. 定义 VGA buffer 的 high-half 虚拟地址常量
2. 实现 `vga_putchar(char c)` 和 `vga_clear()`
3. 考虑：如果 VGA 所在的物理页 `0xB8000` 没有被直接映射覆盖（map_end < `0xB9000`），你的驱动还能工作吗？如何修复？

提示：我们 `vmm_init()` 至少映射 16 MiB，`0xB8000` < 16 MiB，所以直接映射一定覆盖了 VGA 区域。

Chapter 11

Kernel Heap：kmalloc 与 kfree

11.1 本章目标

- 理解内核为什么需要字节级堆分配器——PMM 只能以 4 KiB 页为粒度
- 掌握堆虚拟地址区域规划：`0xE0000000`–`0xF0000000`，256 MiB
- 理解 `heap_ops_t` 可插拔后端接口（与 `pmm_ops_t` 同一模式）
- 实现 First-Fit 空闲链表：`block_header_t`、分配拆分、释放合并
- 了解 Slab 分配器后端：固定大小 cache + bitmap
- 实现 `kmalloc_init`：映射物理页并初始化后端
- 掌握 `kheap_stats_t` 统计信息的含义

11.2 背景知识：为什么需要内核堆？

PMM 以 4 KiB 页为单位分配，VMM 的 `vmm_alloc_pages` 同样以页为粒度。但内核中大量数据结构远小于一页——进程控制块 128 B、VMA 节点 32 B、命令表项十几字节。若每个小对象都分配整页，内部碎片率高达 $1 - 128/4096 \approx 96.9\%$。

为什么？

为什么不直接用全局数组（静态分配）?
静态数组在编译时就固定了容量，无法按运行时需求伸缩；为”以防万一”必须预留最大容量，浪费严重；每新增一种数据结构就要新增一个数组，代码耦合也越来越重。内核堆（`kmalloc`/`kfree`）就是给内核提供任意大小、动态生命周期的分配能力——类比用户态的 `malloc`/`free`。

11.3 堆的虚拟地址区域规划

内核堆需要一段专属连续虚拟地址。在第 ?? 章高半区布局基础上细分：

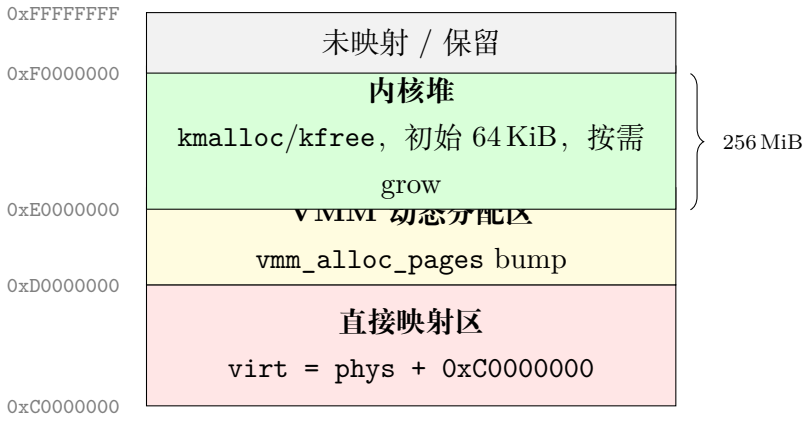

图 11.1: 内核虚拟地址空间细分布局

vmm.h 中定义的三个宏：

```
#define KHEAP_START  0xE0000000u
#define KHEAP_MAX_SIZE (256u * 1024u * 1024u) /* 256 MiB */
#define KHEAP_END    (KHEAP_START + KHEAP_MAX_SIZE) /* 0xF0000000 */
```

设计决策

堆区域必须和 vmm_alloc_pages 的地址空间分开。后者内部用 bump allocator（g_next_vaddr 单调递增），如果堆也用同一空间，两个分配器会互相踩踏。隔离到不同区间后各管各的，互不干扰。

11.4 可插拔后端：heap_ops_t 接口

与第 8 章的 pmm_ops_t 相同模式——接口与实现解耦：

```
typedef struct heap_ops {
    const char* name;
    void (*init)(void* start, size_t size);
    void* (*alloc)(size_t size);
    void (*free)(void* ptr);
    void (*stats)(kheap_stats_t* out);
} heap_ops_t;
```

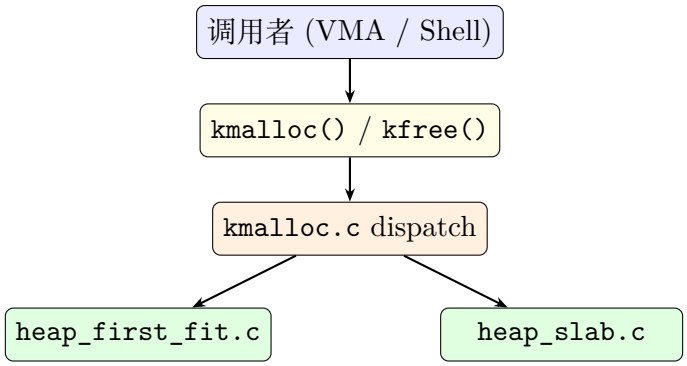

图 11.2: kmalloc 可插拔后端调用链

kconfig.h 编译期选择后端（0=first_fit，1=slab），kmain.c 启动时注册并调用 kmalloc_init()。

思考 11.1

"dispatch + 后端" 模式在 Linux 内核中随处可见（VFS、I/O 调度器、协议栈）。如果不用函数指针表，而在每个调用点用 #if 选择实现，会有什么问题？（提示：代码膨胀与维护成本）

11.5 First-Fit 空闲链表

11.5.1 block_header_t 结构

First-Fit 后端把堆区域切成若干块，每块前面有小 header，所有块串成双向链表：

```
typedef struct block_header {
    size_t size;              /* 用户数据区大小（不含 header） */
    int is_free;              /* 0=已分配，1=空闲 */
    struct block_header* next;
    struct block_header* prev;
} block_header_t;
#define HEADER_SIZE ((sizeof(block_header_t) + 7u) & ~7u) /* 16B */
#define ALIGN8(x) (((x) + 7u) & ~7u)
```

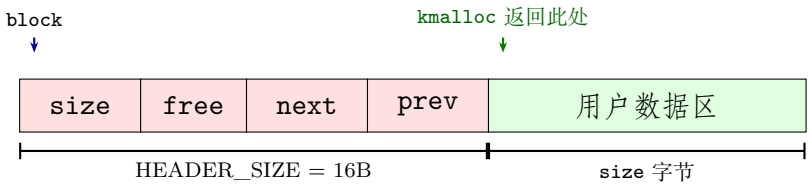

图 11.3: block_header_t 与用户数据区的内存布局

kmalloc 返回 block + HEADER_SIZE；kfree 时往回退 16 字节即找到 header。32 位下 sizeof = 4+4+4+4 = 16，天然 8 字节对齐。

11.5.2 分配：遍历 + 拆分

heap_first_fit_alloc(size) 步骤： 对齐 ALIGN8(size)； 从头遍历找第一个 is_free && size >= need； 若块比请求大得多（余 > HEADER_SIZE + 8）则拆分；is_free = 0，返回数据区指针。

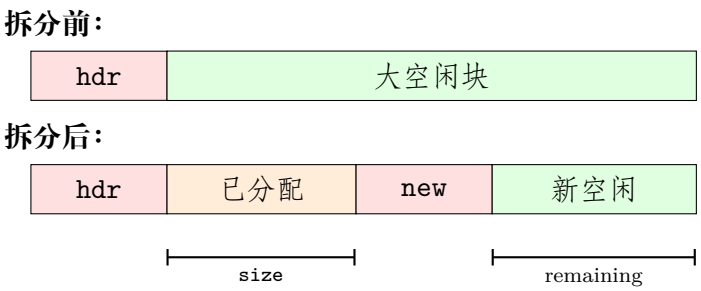

图 11.4: First-Fit 分配时的块拆分

核心拆分代码：

```
if (curr->size - size > HEADER_SIZE + 8) {
    block_header_t* new_block =
        (block_header_t*)((uint8_t*)curr + HEADER_SIZE + size);
    new_block->size = curr->size - size - HEADER_SIZE;
    new_block->is_free = 1;
    new_block->next = curr->next; new_block->prev = curr;
    if (curr->next) curr->next->prev = new_block;
    curr->size = size;
    curr->next = new_block;
}
curr->is_free = 0;
return (void*)((uint8_t*)curr + HEADER_SIZE);
```

思考 11.2

拆分阈值 HEADER_SIZE + 8 的含义：剩余空间必须够放 header（16 B）加最小数据（8 B），否则拆出的碎片永远不可用。若改成 HEADER_SIZE + 1，返回的指针将不满足 8 字节对齐——许多 CPU 指令（如 SSE）会因此触发异常。

11.5.3 释放与合并

`heap_first_fit_free(ptr)` 步骤：`block = ptr - HEADER_SIZE`；范围检查；`is_free = 1`；向后合并（while `next` 也 free 就 `merge_with_next`）；向前合并（若 `prev` 也 free 让其吞并自己）。

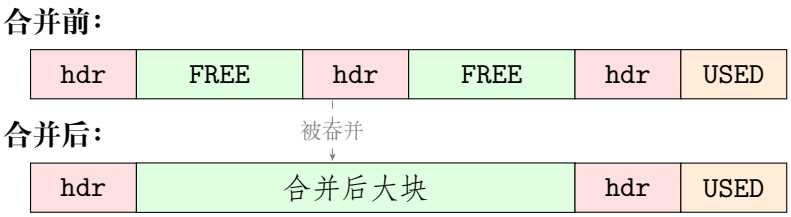

图 11.5: 相邻空闲块的合并

```
static void merge_with_next(block_header_t* block) {
    block_header_t* victim = block->next;
    block->size += HEADER_SIZE + victim->size;
    block->next = victim->next;
    if (victim->next) victim->next->prev = block;
}
```

为什么？

为什么需要双向链表?
单向链表向后合并（吃 next）没问题，但向前合并需要从 block 找前驱——单向链表要 $O(n)$ 遍历。双向链表多花 4 B 存 `prev`，换来 $O(1)$ 向前合并，是经典的空间换时间。

11.6 堆的初始化与增长

11.6.1 `kmalloc_init`：映射初始物理页

`kmalloc_init()` 在内核启动时调用：逐页从 PMM 分配物理页 → VMM 映射到 `0xE0000000` → 调后端 `init`。

```
void kmalloc_init(void) {
    size_t initial_size = KHEAP_INITIAL_PAGES * VMM_PAGE_SIZE;
    for (unsigned i = 0; i < KHEAP_INITIAL_PAGES; i++) {
        uint32_t virt = KHEAP_START + i * VMM_PAGE_SIZE;
        uint32_t phys = (uint32_t)(uintptr_t)pmm_alloc_page();
        vmm_map_page(virt, phys, PTE_PRESENT | PTE_WRITABLE);
```

```
    }
    g_heap_ops->init((void*)(uintptr_t)KHEAP_START, initial_size);
}
```

初始页数由 `kconfig.h` 配置：`KCONFIG_HEAP_INITIAL_PAGES = 16`（64 KiB）。

11.6.2 heap_brk：按需增长

First-Fit 后端用 `heap_brk` 跟踪已提交上界：

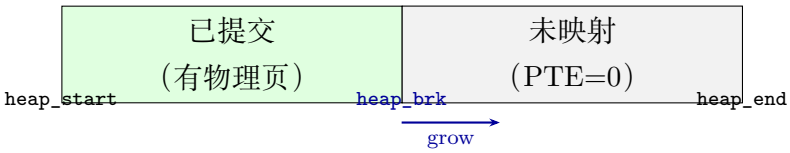

图 11.6: `heap_brk`：已提交区域上界

分配找不到合适块时调 `heap_grow(min_bytes)`——从 PMM 拿页、VMM 映射、推进 `heap_brk`，语义类似用户态 `sbrk()`。

11.7 Slab 分配器概述

First-Fit 每次分配 $O(n)$ 遍历且易碎片化。**Slab 分配器**（源自 Solaris / Linux）针对这两个弱点：按固定大小预切 slot，相同大小 $O(1)$ 取。

11.7.1 7 级 Cache

编号	大小	每页 Slot 数	适用场景
0	32 B	≈ 126	小结构体、链表节点
1	64 B	≈ 63	字符串缓冲区
2	128 B	≈ 31	进程控制块
3	256 B	≈ 15	中型数据
4	512 B	≈ 7	大结构体
5	1024 B	3	页级缓冲
6	2048 B	1	最大 cache

`kmalloc(50)` → cache-64，内部碎片 = 14 B。

11.7.2 Slab 内存布局

每个 slab 占一页（4 KiB），页头放 `slab_header_t`（含 128 位 bitmap）：

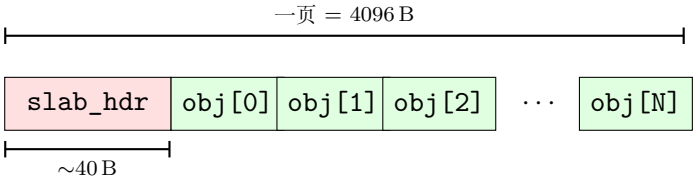

图 11.7: Slab 内存布局

分配：`bitmap_find_first_set` → 清位 → 计算地址，$O(1)$。释放：`ptr & ~0xFFF` 找到页头 header，算 slot index → 置位，$O(1)$。

设计决策

`slab_header_t` 放在页头而非单独分配的三个原因：避免鸡生蛋——slab 自己就是分配器，metadata 不能再 `kmalloc`；`kfree` 时页对齐即得 header，$O(1)$；每页只损失 ~40 B（< 1%）。

11.8 统计信息：kheap_stats_t

```
typedef struct kheap_stats {
    size_t  heap_size;   /* 已提交区域总大小 */
    size_t  used_bytes;  /* 已分配字节（含 header 开销） */
    size_t  free_bytes;  /* 空闲字节 */
    unsigned alloc_count; /* 活跃分配数 */
    unsigned free_count; /* 空闲块数 */
} kheap_stats_t;
```

注意 `used_bytes + free_bytes ≠ heap_size`——差值是空闲块 header 的开销。`free_count` 很大但 `free_bytes` 很小说明碎片化严重。

11.9 验证：heap 命令

Shell 输入 `heap` 查看堆状态：

```
> heap
=== Kernel Heap Status ===
  backend:   first_fit
  heap size: 65536 bytes (64 KiB)
  used:      0 bytes (0 allocations)
  free:      65520 bytes (1 free blocks)
```

初始：整个 64 KiB 是一个大空闲块（65536 − 16 = 65520）。执行 `heap test`：

```
> heap test
[heap] alloc: req=100 aligned=104
[heap] alloc: split -> used 104 + free 65400 @ 0xE0000078
[heap] alloc: return 0xE0000010
test: alloc 100B -> 0xE0000010
...
[heap] free: merge forward ...
test: free all -- OK
```

可验证：100 B 对齐到 104（$\lceil 100/8 \rceil \times 8$）；返回地址 = 0xE0000000 + 16；合并正确执行。

11.10 初始化顺序

```
pmm_init(mmap);      /* 1. 物理内存管理器 */
vmm_init();          /* 2. 虚拟内存（页表）*/
vmm_unmap_identity(); /* 3. 拆除 identity mapping */
kmalloc_init();      /* 4. 内核堆 — 依赖 PMM + VMM */
vma_init();          /* 5. VMA — 依赖 kmalloc */
```

依赖链：PMM → VMM → kmalloc → VMA。VMA 在堆之后——因为 VMA 节点需要 kmalloc 分配。

11.11 思考与练习

练习 11.1

实现 Best-Fit 分配算法

复制 heap_first_fit.c 为 heap_best_fit.c，修改 alloc：遍历**整个**链表，找 size 差值最小的空闲块。在 kconfig.h 新增 KCONFIG_HEAP_BACKEND == 2。测试：反复分配 32B/128B/64B，释放 128B，再分配 100B——对比两种算法行为。思考：Best-Fit 一定减少碎片吗？在什么负载下反而更差？

练习 11.2

堆增长（heap_grow）集成

当前 First-Fit 找不到块就返回 NULL。扩展它：调 heap_grow(size + HEADER_SIZE)；找链表最后一个块 last，算末尾 last_end；若 last 是 free 就扩 size，否则在 last_end 建新空闲块；递归重试分配。

练习 11.3

碎片率统计

给 `kheap_stats_t` 新增 `fragmentation` 字段：

$$\text{fragmentation} = \left(1 - \frac{\max(\text{单块 free size})}{\sum \text{free_bytes}}\right) \times 100\%$$

碎片率 $= 0$ 表示所有空闲内存连续；趋近 100% 表示碎片化严重。在 `heap` 命令中显示该指标。

练习 11.4

Slab 的 `kfree` 流程分析

阅读 `heap_slab.c`，画出 `kfree` 完整流程图（标注数据结构）：`ptr & ~0xFFF` → 读 `cache_idx/obj_size` → 计算 slot index → `bitmap_set_bit` → 若从 full 变非满则移到 partial。

进阶：当 slab 全空（`free_count == capacity`）时，是否应把整页还给 PMM？好处和风险各是什么？

Chapter 12

VMA：虚拟内存区域管理

12.1 本章目标

- 理解 VMA 的核心价值——让 Page Fault handler 区分”合法的按需分页”与”非法访问”
- 掌握 `vma_ops_t` 可插拔后端接口（与 `pmm_ops_t`/`heap_ops_t` 同一模式）
- 理解 `vm_area_t` 结构体中各字段的含义
- 掌握 VMA 权限标志（`VMA_READ`/`WRITE`/`EXEC`）与 PTE 标志位的映射关系
- 理解 `vma.c` dispatch 层的转发逻辑
- 了解红黑树后端的核心思想：5 条性质、旋转操作、$O(\log n)$ 保证
- 理解静态节点池的必要性（打破 kmalloc 循环依赖）
- 掌握 Page Fault 集成：三种诊断结果
- 理解引导阶段区域注册与 `vmm_alloc_pages` 的 VMA 自动跟踪

12.2 背景知识：为什么需要 VMA？

在第 ?? 章我们实现了二级页表：CPU 通过 PDE → PTE → 物理页完成地址翻译。当 PTE 的 Present 位为 0 时，CPU 触发 Page Fault（#PF, ISR 14），但 *页表本身不记录这个虚拟地址”本来应不应该”被映射*。

考虑以下两种引发 #PF 的场景：

1. **合法的按需分页**——程序声明了一段堆区域，但尚未分配物理页。缺页时内核应分配物理页并建立映射，让程序继续运行。

2. **非法访问**——野指针写到 0x00000000，或跳转到一段没有任何映射的地址。内核应杀死进程或触发 panic。

为什么？

为什么不在 Page Fault handler 里扫描页表来区分?
页表只有两种状态：映射了（Present=1）或没映射（Present=0）。”没映射”既可能是”还没来得及映射”，也可能是”根本不该映射”。VMA 显式注册每个合法的虚拟地址区间及其权限，使 #PF handler 有了第二层判据——*该地址属不属于某个已注册的 VMA？*

VMA（Virtual Memory Area）的职责就是 ** 维护一张虚拟地址区间注册表 **。每个区间记录 [start，end)、权限标志和可读名称。Page Fault handler 查 VMA 表，即可做出三种判断（详见第 ?? 节）。

12.3 vm_area_t 结构体

VMA 系统中最基本的数据结构是 vm_area_t——描述一段虚拟内存区域：

```
typedef struct vm_area {
    uint32_t  start;     /* 起始虚拟地址（包含，页对齐） */
    uint32_t  end;       /* 结束虚拟地址（不包含，页对齐） */
    uint32_t  flags;     /* VMA_READ | VMA_WRITE | VMA_EXEC */
    const char* name;    /* 人类可读名称，如 "direct-map", "kheap" */
} vm_area_t;
```

约定与 Linux 一致：[start，end) 左闭右开，start 和 end 都必须页对齐，不同 VMA 之间不允许地址重叠。

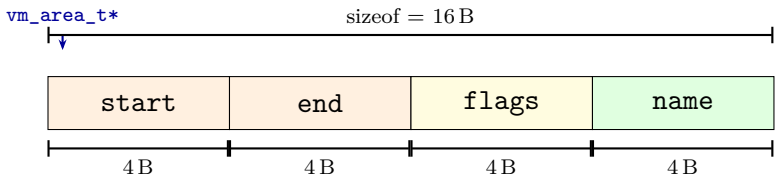

图 12.1: vm_area_t 内存布局（32 位平台）

注意：vm_area_t 是公开的只读视图，不包含后端内部管理字段（如红黑树颜色、父/子指针等）。这保持了接口与实现的解耦——调用者只读 vm_area_t，后端的内部节点可以包含任意附加字段。

12.4 VMA 权限标志与 PTE 标志的关系

```
#define VMA_READ 0x01u  /* 可读 */
#define VMA_WRITE 0x02u /* 可写 */
#define VMA_EXEC 0x04u  /* 可执行（配合 NX 位使用） */
```

VMA 标志描述的是*语义层级*的权限意图；PTE 标志是*硬件层级*的访问控制。它们之间的映射关系如表 ?? 所示。

VMA 标志	对应 PTE 操作	说明
VMA_READ	PTE_PRESENT	页面存在即可读
VMA_WRITE	PTE_WRITABLE	设置 R/W 位
VMA_EXEC	清除 NX 位（PAE/64-bit）	32 位无 NX，留作扩展

表 12.1: VMA 权限标志到 PTE 标志的映射

思考 12.1

在 32 位非 PAE 模式下，x86 没有 NX（No-Execute）位——所有可读页面都可以执行。这意味着 VMA_EXEC 在当前实现中其实不会产生硬件层面的约束。那为什么还要定义它？
提示：考虑将来迁移到 PAE 或 64 位模式、以及软件层面的审计需求。

12.5 可插拔后端：vma_ops_t 接口

与 PMM（第 8 章）和 kmalloc（第 ?? 章）完全相同的"dispatch + 后端" 模式：

```
typedef struct vma_ops {
    const char* name;
    void (*init)(void);
    int (*add)(uint32_t start, uint32_t end,
               uint32_t flags, const char* name);
    int (*remove)(uint32_t start);
    const vm_area_t* (*find)(uint32_t addr);
    void (*dump)(void);
    unsigned (*count)(void);
} vma_ops_t;
```

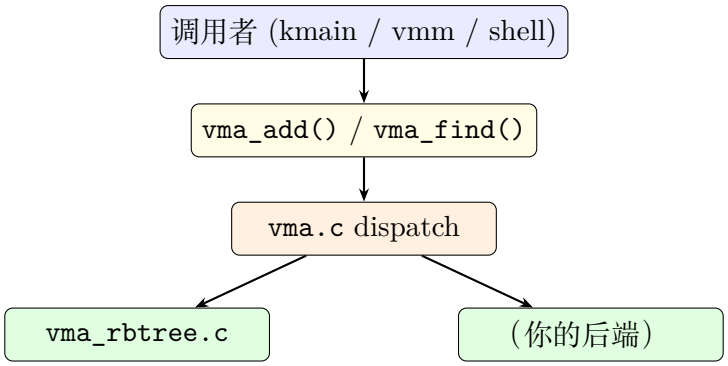

图 12.2: vma_ops_t 可插拔后端调用链

12.6 Dispatch 层：vma.c 实现

vma.c 的职责非常简单——校验后端是否已注册，然后转发：

```
static const vma_ops_t* g_vma_ops = NULL;
static int g_vma_ready = 0;

void vma_register_backend(const vma_ops_t* ops) {
    g_vma_ops = ops;
}

void vma_init(void) {
    if (!g_vma_ops) {
        printk("[vma] no backend registered\n");
        return;
    }
    g_vma_ops->init();
    g_vma_ready = 1;
    printk("[vma] backend: %s, init ok\n", g_vma_ops->name);
}

int vma_add(uint32_t start, uint32_t end,
            uint32_t flags, const char* name) {
    if (!g_vma_ops || !g_vma_ready) return -1;
    return g_vma_ops->add(start, end, flags, name);
}
```

其余 vma_remove/vma_find/vma_dump/vma_count 格式完全一致——if (!ready) return fallback; return g_vma_ops->xxx(...);。

设计决策

Dispatch 层看似”什么都没做”，但它带来三个好处：

1. **编译期解耦**——调用者只 `#include "vma.h"`，不需要知道后端头文件；
2. **运行时切换**——`vma_register_backend` 在 init 前随时可换后端；
3. **安全守卫**——未初始化时所有 API 返回安全默认值，不会空指针崩溃。

12.7 红黑树后端概述

VMA 后端需要高效地支持三种操作：

操作	链表	排序数组	红黑树
`find`(addr)	$O(n)$	$O(\log n)$	$O(\log n)$
`add`	$O(n)$	$O(n)$	$O(\log n)$
`remove`	$O(n)$	$O(n)$	$O(\log n)$
`dump`（遍历）	$O(n)$	$O(n)$	$O(n)$

表 12.2: 不同数据结构的 VMA 操作复杂度对比

内核随进程增多，VMA 数量会快速增长（Linux 进程平均 65+ VMAs）。红黑树在所有单次操作上都保持 $O(\log n)$，是最优选择。

12.7.1 红黑树的 5 条性质

红黑树是一种自平衡二叉搜索树，每个节点带一个颜色位（红/黑），且满足以下 5 条性质：

1. 每个节点要么是红色，要么是黑色。
2. 根节点是黑色。
3. 所有叶子节点（NIL 哨兵）是黑色。
4. 红色节点的两个子节点必须是黑色（不能有连续的红节点）。
5. 从任意节点到其所有后代叶子节点的路径上，黑色节点数相同（黑高相等）。

性质 4 + 5 共同保证了树的高度不超过 $2\log_2(n+1)$，从而使查找、插入、删除都是 $O(\log n)$。

12.7.2 旋转操作

当插入或删除破坏了红黑性质时，需要通过**旋转**（rotation）恢复平衡。旋转分为左旋和右旋，它们互为镜像操作：

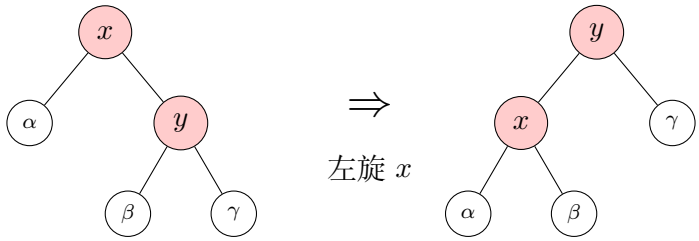

图 12.3: 红黑树左旋示意图（右旋为其镜像）

旋转只改变指针指向，不改变 BST 的中序遍历顺序——因此旋转前后所有 VMA 的地址排序仍然正确。

思考 12.2

如果 VMA 后端改用简单的有序链表（插入时遍历找位置），在 VMA 数量为 $n = 200$ 时，`find` 的最坏情况要遍历多少个节点？红黑树呢？（提示：$\lceil \log_2 201 \rceil = 8$）在什么场景下链表反而比红黑树更合适？（提示：VMA 数量极少且变化频繁——旋转的常数开销）

12.8 静态节点池：打破循环依赖

回顾初始化顺序（第 ?? 章）：

```
pmm_init();      /* 1. 物理内存 */
vmm_init();      /* 2. 页表 */
kmalloc_init();  /* 3. 内核堆 */
vma_init();      /* 4. VMA — 依赖 kmalloc */
```

VMA 初始化在 `kmalloc_init()` 之后——所以后端可以用 `kmalloc` 分配内部节点。但存在一个微妙问题：

为什么？

如果 `kmalloc` 的堆增长（`heap_grow`）需要调 `vmm_alloc_pages`，而 `vmm_alloc_pages` 内部想注册 VMA（跟踪新分配的虚拟地址区间），VMA 的 `add` 又要 `kmalloc` 分配红黑树节点——循环依赖！

解决方案：红黑树后端使用**静态节点池**（编译期 `static` 数组），预分配固定数量的节点（如 256 个），完全绕开 `kmalloc`。代价是 VMA 上限固定，但内核启动阶

段区域数量很少，256 足够。

静态节点池的设计非常简单：

```
#define VMA_MAX_NODES 256

typedef struct rb_node {
    vm_area_t area;  /* 公开视图 */
    int       color;  /* 0=BLACK, 1=RED */
    struct rb_node* parent;
    struct rb_node* left;
    struct rb_node* right;
} rb_node_t;

/* 静态数组——不依赖 kmalloc */
static rb_node_t node_pool[VMA_MAX_NODES];
static unsigned pool_used = 0;
```

分配节点：`pool_used < VMA_MAX_NODES ? &node_pool[pool_used++] : NULL`。释放节点：标记为已回收（简单实现可放入 freelist；最简实现不回收）。

12.9 Page Fault 集成

#PF handler(ISR 14)收到故障地址 `cr2` 和错误码 `err_code` 后，调用 `vma_find(cr2)` 可以产生三种诊断结果：

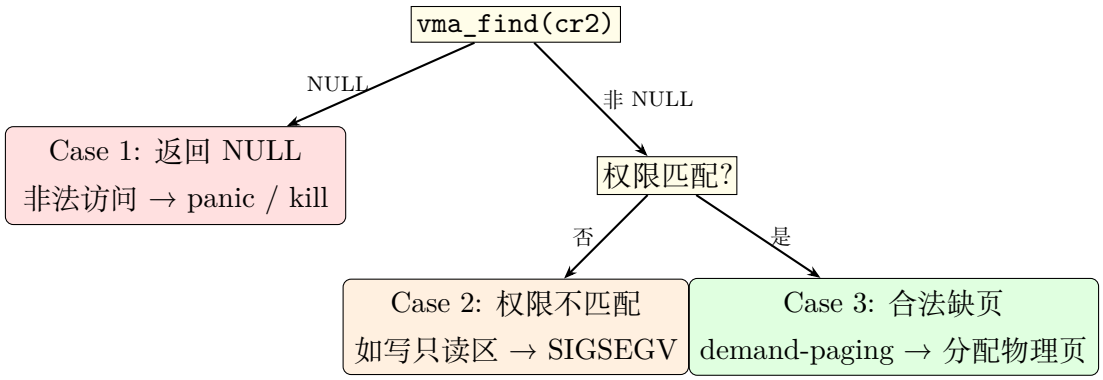

图 12.4: #PF handler 中的 VMA 三种诊断

1. **Case 1: 无 VMA**（`vma_find` 返回 NULL）——该地址不属于任何已注册区域，是真正的非法访问。内核态触发则 panic；用户态触发则杀死进程。

2. **Case 2: 权限不匹配**——地址属于某个 VMA，但操作违反权限。例如写入一个仅 `VMA_READ` 的区域。对应 Linux 的 `SIGSEGV`。

3. **Case 3: 合法缺页（demand paging candidate）**——地址属于 VMA 且权限匹配，只是物理页尚未分配。分配物理页 → 建立映射 → 返回，程序继续运行。

当前实现中我们尚未启用 demand paging，所有映射在 `vmm_alloc_pages` 时一步到位。但 VMA 框架为将来的按需分页奠定了基础。

12.10 引导阶段区域注册

在 `kmain.c` 中，VMA 初始化后立即注册两个引导时建立的内核区域：

```
vma_init();

if (vma_is_ready()) {
    /* 1. 直接映射区：内核代码/数据 + 直接映射的物理内存 */
    vma_add(KERNEL_VIRT_OFFSET, vmm_direct_map_end(),
            VMA_READ | VMA_WRITE | VMA_EXEC, "direct-map");

    /* 2. 内核堆区 */
    vma_add(KHEAP_START,
            KHEAP_START + KCONFIG_HEAP_INITIAL_PAGES * VMM_PAGE_SIZE,
            VMA_READ | VMA_WRITE, "kheap");
}
```

- `direct-map`：覆盖 `0xC0000000` 到 `vmm_direct_map_end()`，包含内核代码（需要 EXEC）、只读数据和读写数据。权限给足 `rwx`，更细粒度的分段保护留给将来的 `.text`/`.rodata` 拆分。

- `kheap`：覆盖 `0xE0000000` 开始的初始堆区域。堆只做数据 `rw`，无需 `x`。堆增长时应同步调用 `vma_add` 或更新 end 来跟踪新区域。

设计决策

注册 VMA 的时机是”谁建立映射，谁负责注册”：

- `vmm_init()` 建立直接映射 → `kmain` 注册 `direct-map`
- `kmalloc_init()` 建立堆映射 → `kmain` 注册 `kheap`
- `vmm_alloc_pages()` 将来可在内部自动调用 `vma_add`

这比集中在一处注册更不容易遗漏——映射与注册在同一上下文中完成。

12.11 vmm_alloc_pages/free_pages 的 VMA 自动跟踪

当前 vmm_alloc_pages 只做 PMM 分配 + 页表映射，不通知 VMA。理想的流程是在分配/释放时自动跟踪：

```
void* vmm_alloc_pages(unsigned count) {
    uint32_t vaddr = bump_next();          /* 选虚拟地址 */
    for (unsigned i = 0; i < count; i++) {
        uint32_t phys = pmm_alloc_page();
        vmm_map_page(vaddr + i * PAGE_SIZE, phys,
                     PTE_PRESENT | PTE_WRITABLE);
    }
    /* 自动注册 VMA */
    if (vma_is_ready()) {
        vma_add(vaddr, vaddr + count * PAGE_SIZE,
                VMA_READ | VMA_WRITE, "vmm-alloc");
    }
    return (void*)vaddr;
}
```

vmm_free_pages 的逆操作则调用 vma_remove(vaddr)。这样 Page Fault handler 对所有动态映射都有完整的 VMA 覆盖，不会误判合法区域为非法访问。

12.12 验证：Shell 命令

12.12.1 vma list——列出所有 VMA

```
> vma list
=== VMA list (rbtree, 2 entries) ===
  [0xC0000000, 0xD0000000) rwx direct-map (256 MiB)
  [0xE0000000, 0xE0010000) rw- kheap      (64 KiB)
```

初始状态有两个 VMA：直接映射区和内核堆。地址按升序排列（红黑树中序遍历），权限用 rwx 紧凑表示。

12.12.2 vma find <addr>——查找指定地址

```
> vma find 0xC1000000
0xC1000000 -> VMA [0xC0000000, 0xD0000000) 'direct-map'
  size: 262144 KiB, offset: 0x1000000, flags: rwx
```

```
> vma find 0xA0000000
0xA0000000 -> no VMA (unmapped / untracked)
```

12.12.3 vma test——自测试

```
> vma test
[vma-test] === VMA selftest ===
[vma-test] test 1: add + find
[vma-test] PASS
[vma-test] test 2: find outside
[vma-test] PASS
[vma-test] test 3: overlap detection
[vma-test] PASS
[vma-test] test 4: remove + find
[vma-test] PASS
[vma-test] === ALL PASS ===
```

测试使用 0xF1000000–0xF1010000（64 KiB）作为临时区域，验证 add/find/overlap/remove 四个基本操作的正确性。

12.13 内核虚拟地址空间全景与 VMA 覆盖

将第 ?? 章的地址空间图补充 VMA 信息后：

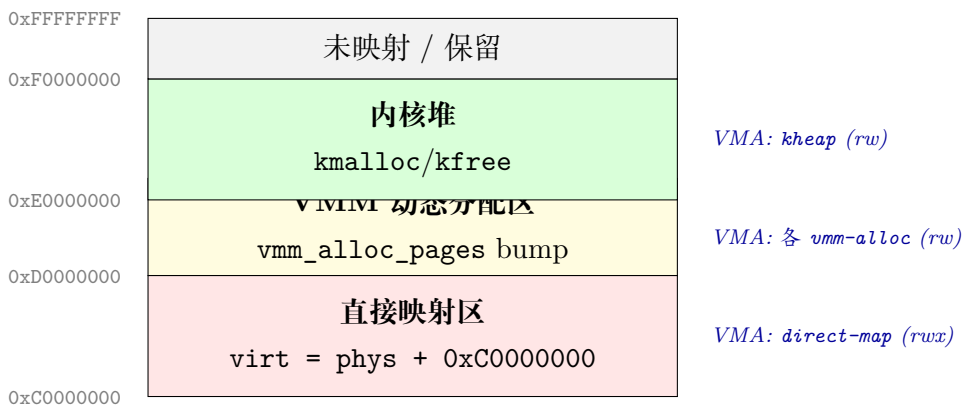

图 12.5: 内核虚拟地址空间与 VMA 覆盖关系

12.14 思考与练习

练习 12.1

实现有序链表后端 vma_list.c

实现一个简单的有序单链表后端（满足 vma_ops_t 接口）：

1. 使用静态数组 node_pool[128]，每个节点包含 vm_area_t 和 next 指针
2. add：遍历链表找到正确位置（按 start 升序），插入前检查重叠
3. find：线性扫描，start <= addr < end 即命中
4. remove：找到 start 匹配的节点，从链表中摘除

在 kconfig.h 中添加 VMA_BACKEND 选项，0=list, 1=rbtree。用 vma test 验证通过。

练习 12.2

#PF handler 中集成 VMA 检查

修改 irq.c 中的 Page Fault handler：

1. 读取 CR2（故障地址）
2. 调用 vma_find(cr2) 获取 VMA
3. 如果返回 NULL：打印 "SEGFAULT: addr=0x... not in any VMA" 并 panic
4. 如果返回非 NULL 但权限不匹配：打印 "PERMISSION DENIED" 并 panic
5. 如果权限匹配：打印 "demand-page candidate"（暂不实现实际分页）

测试：将 0xA0000000 解引用，确认触发 Case 1 的 panic 输出。

练习 12.3

VMA 区域合并

当新增 VMA 与相邻 VMA 满足以下条件时自动合并：

- 新 VMA 的 start == 已有 VMA 的 end（或反之）
- 两者的 flags 完全相同

合并后更新 end（或 start）并移除多余节点。这减少了 VMA 总数，使红黑树更扁平、查找更快。

思考：合并可能影响 `vma_remove(start)` 的语义——如果用户按原始 `start` 移除，但该区域已被合并到更大的 VMA 中，应该怎么处理？

练习 12.4

VMA 统计与碎片分析

给 `vma_ops_t` 新增 `stats` 函数指针，收集以下信息：

1. 已注册 VMA 总数
2. 已覆盖的虚拟地址总量（KiB）
3. VMA 之间最大空洞的大小——即相邻 VMA 之间最大的 `gap`
4. 节点池利用率（`pool_used / VMA_MAX_NODES`）

在 `vma count` 命令中显示这些统计信息。地址空间碎片化可定义为：

$$\text{fragmentation} = 1 - \frac{\max(\text{gap})}{\text{total unmapped space}}$$

Part IV

进程与用户态

Chapter 13

TSS 与 Ring 3：进入用户态

13.1 本章目标

- 理解 x86 特权级（Ring 0 – Ring 3）与特权级切换机制
- 实现 Task State Segment（TSS），配置内核栈指针
- 在 GDT 中添加 TSS 描述符和用户态代码/数据段
- 用 `iret` 从 Ring 0 跳转到 Ring 3 执行用户代码

13.2 背景知识：x86 特权级

13.2.1 TSS 的作用

13.2.2 从 Ring 0 到 Ring 3：`iret`

13.3 GDT 扩展

13.4 TSS 初始化

13.5 跳转到用户态

CPU 状态变化

`iret` 执行后：

- CPL 从 0 变为 3（用户态）
- CS 指向用户代码段，SS 指向用户数据段

- ESP 切换到用户态栈
- 内核页表中 PTE_USER 位控制哪些页用户态可访问

13.6 验证

13.7 本章小结

TSS + iret 让我们能在 Ring 0 和 Ring 3 之间切换。下一章加载 ELF 文件，让用户态执行真正的程序。

Chapter 14

ELF 加载器

14.1 本章目标

- 理解 ELF32 文件格式（ELF header、Program header、Section header）
- 实现 ELF 加载器：解析 ELF header → 加载 `.text`/`.data` 到用户地址空间
- 用 VMA 记录各段映射（代码段 r-x、数据段 rw-）

14.2 背景知识：ELF 格式

14.2.1 ELF 加载流程

14.3 数据结构

14.4 实现详解

14.5 VMA 集成

14.6 验证

14.7 本章小结

ELF 加载器让内核能运行外部编译的程序。下一章实现系统调用，让用户态程序能请求内核服务。

Chapter 15

系统调用

15.1 本章目标

- 理解系统调用的本质：用户态 → 内核态的受控入口

- 实现 `int 0x80` 系统调用门

- 第一批 syscall：`write`、`exit`、`brk`

- 用户态 syscall 封装库

15.2 背景知识：为什么需要系统调用

15.2.1 系统调用约定

15.3 IDT 配置

15.4 Syscall 分发

15.5 第一批系统调用

15.6 用户态封装

15.7 验证

15.8 本章小结

系统调用桥接了用户态和内核态。有了 write/exit，用户程序能输出信息和正常退出。下一章实现进程管理（PCB、fork、exec）。

Chapter 16

进程管理

16.1 本章目标

- 设计进程控制块（PCB / `task_struct`）
- 每个进程拥有独立的页表和 VMA 树
- 实现 `fork`（COW + VMA 复制）、`exec`、`waitpid`

16.2 背景知识：进程抽象

16.3 PCB 数据结构

16.4 进程地址空间

16.5 fork：进程复制

16.6 exec：加载新程序

16.7 waitpid：等待子进程

16.8 验证

16.9 本章小结

进程管理让内核能同时运行多个独立程序。下一章实现调度器，让这些进程分时共享 CPU。

Chapter 17

调度器

17.1 本章目标

- 实现时间片轮转（Round-Robin）调度
- 实现上下文切换：寄存器保存/恢复 + CR3 切换
- 利用 IRQ0（PIT 定时器）触发调度

17.2 背景知识：CPU 调度

17.3 上下文切换

CPU 状态变化

上下文切换后：

- CR3 指向新进程的页目录 → 地址空间切换
- TSS.ESP0 指向新进程的内核栈 → 中断时用正确的栈
- 通用寄存器恢复为新进程上次被切走时的状态

17.4 Round-Robin 调度器

17.5 PIT 定时器配置

17.6 验证

17.7 本章小结

调度器让多个进程分时运行。至此”进程与用户态”里程碑完成。下一部分进入文件系统。

Part V

文件系统

Chapter 18

VFS：虚拟文件系统

18.1 本章目标

- 设计 VFS 抽象层：统一 `open`/`read`/`write`/`close` 接口
- 实现文件描述符表（per-process fd table）
- 为后续具体文件系统（ramfs、FAT32）提供挂载框架

18.2 背景知识：为什么需要 VFS

18.3 VFS 数据结构

18.4 文件描述符表

18.5 系统调用集成

18.6 验证

18.7 本章小结

VFS 是文件系统的”可插拔接口”。下一章实现第一个具体文件系统 ramfs，让用户程序能读取打包在内核镜像中的文件。

Chapter 19

initrd / ramfs：内存文件系统

19.1 本章目标

- 实现 ramfs（内存中的简易文件系统）
- 将初始文件打包进内核镜像（initrd）
- ELF 加载器通过 VFS 读取文件，而非硬编码内存地址

19.2 背景知识：initrd

19.3 Ramfs 实现

19.4 GRUB Module 加载

19.5 验证

19.6 本章小结

有了 ramfs，用户程序可以通过标准文件系统接口访问文件。这为将来移植更复杂的文件系统（FAT32、ext2）和用户态工具链奠定基础。

Part VI

附录

附录 A

工具链速查

A.1 构建命令

命令	用途
`make iso`	编译内核 + 打包 GRUB ISO
`make run`	QEMU 启动 ISO
`make debug`	QEMU -S 等待 GDB（需 `DEBUG=1`）
`make gdb`	启动 GDB 并连接 QEMU
`make clean`	清理构建产物
`make compdb`	生成 `compile_commands.json`

A.2 QEMU 常用选项

选项	含义
`-serial stdio`	串口输出到终端
`-m 256`	分配 256MiB 内存
`-S`	启动时暂停 CPU
`-s`	开启 gdbstub（端口 1234）
`-no-reboot`	Triple Fault 时不自动重启
`-d int`	输出中断日志

A.3 GDB 常用命令

命令	用途
`b kmain`	在 kmain 设断点
`c`	继续运行
`n / s`	单步（不进入/进入函数）
`si / ni`	指令级单步
`info reg`	查看寄存器
`bt`	调用栈回溯
`x/10i $eip`	反汇编当前位置
`p/x var`	十六进制打印变量
`x/32wx $esp`	查看栈内存

A.4 Shell 命令速查

命令	用途
`cls`	清屏
`cmds`	列出所有命令
`free`	物理内存用量
`mmap`	Multiboot2 内存映射
`pmm state`	PMM 状态
`pmm alloc N`	分配 N 个物理页
`vmm state`	VMM 状态
`vmm lookup ADDR`	虚拟 → 物理地址翻译
`heap status`	堆统计
`heap test`	堆 selftest
`vma list`	列出所有 VMA
`vma find ADDR`	查找地址所属 VMA
`vma test`	VMA selftest
`shutdown`	关机

附录 B

x86 架构速查

B.1 寄存器

B.1.1 通用寄存器（32 位）

寄存器	惯用用途	调用约定
EAX	累加器、返回值	caller-saved
EBX	基址	callee-saved
ECX	计数器	caller-saved
EDX	数据、I/O	caller-saved
ESI	源索引	callee-saved
EDI	目的索引	callee-saved
EBP	栈帧基指针	callee-saved
ESP	栈指针	特殊

B.1.2 段寄存器

寄存器	用途
CS	代码段选择子（隐含 CPL）
DS	默认数据段
SS	栈段
ES	附加数据段
FS / GS	通用附加段（Linux 用 GS 做 per-CPU / TLS）

B.1.3 控制寄存器

寄存器	关键位
CR0	bit 0 (PE): 保护模式
	bit 31 (PG): 开启分页
CR2	Page Fault 时的故障虚拟地址
CR3	页目录物理地址
CR4	bit 4 (PSE): 4MiB 大页支持
	bit 7 (PGE): 全局页

B.2 GDT 条目格式

B.3 IDT 条目格式

B.4 PDE / PTE 格式

`bit 0` P (Present) —1= 有效，0= 访问触发 #PF

`bit 1` R/W —1= 读写，0= 只读

`bit 2` U/S —1= 用户态可访问，0= 仅内核

`bit 3` PWT —写透缓存策略

`bit 4` PCD —禁用缓存

`bit 5` A (Accessed) —CPU 访问后自动置 1

`bit 6` D (Dirty) —CPU 写入后自动置 1（仅 PTE）

`bit 7` PS/PAT —PDE: 4MiB 大页; PTE: PAT

`bit 8` G (Global) —TLB 不随 CR3 切换而刷新

`bit 9--11` AVL —软件可用位

`bit 12--31` 页帧物理地址高 20 位

B.5 Page Fault 错误码

`bit 0` P —0= 页不存在，1= 权限违规

`bit 1` W/R —0= 读操作，1= 写操作

`bit 2` U/S —0= 内核态，1= 用户态

`bit 3` RSVD —页表条目保留位被置位

bit 4 I/D —取指令引起

B.6 中断向量分配

向量	名称	说明
0	#DE	除零
3	#BP	断点（int3）
6	#UD	无效操作码
8	#DF	Double Fault
13	#GP	General Protection
14	#PF	Page Fault
32–47	IRQ0–15	硬件中断（PIC remap 后）
128	—	系统调用（int 0x80）